

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 _____

姓 名 _____

专 业 _____

授课老师 _____

一、实验内容

本实验使用 Verilog HDL 在 Vivado 2016 上设计并实现一款支持 54 条 MIPS 指令的 32 位 CPU。CPU 采用指令存储器（也即 IMEM）与数据存储器（也即 DMEM）分离的**哈佛结构**，以单周期数据通路为主体，完成取指、译码、运算、访存、寄存器写回、分支跳转和异常处理等功能。大多数普通的运算指令在单个指令周期内完成，乘除法模块通过 busy 和 done 信号进行多周期控制，并在运算期间保持 PC 不变。

CPU 主要由 PC、通用寄存器堆、指令译码器、组合逻辑控制器、ALU、IMEM、DMEM、HI/LO 寄存器、乘法器、除法器 and 简化的 CP0 部件组成。所实现的指令包括算术与逻辑运算、比较与移位、立即数运算、数据装载与存储、条件分支、无条件跳转、乘除运算、HI/LO 读写以及 CP0 异常处理等类型。

测试工作包括单条指令测试、边界数据测试、指令序列测试和学校提供的综合测试程序测试。**前仿真阶段将 CPU 运行结果与标准结果进行 txt_compare；**后仿真阶段在综合后的网表上继续进行功能和时序行为验证。

下板阶段使用学校提供的 COE 文件初始化指令存储器 IP 核，通过 icf.xdc 完成 Nexys4 DDR 开发板的时钟、复位和七段数码管引脚约束。工程经过综合、实现并生成 Bitstream 后下载至开发板，将 CPU 的 PC 值输出到七段数码管，用于观察 PC 地址的变化过程。

二、CPU 数据通路设计

CPU 的取指过程是 54 条指令固定的开始流程，由 PC 和 IMEM 共同完成，PC 给出当前指令的字节地址，经过基址转换并舍去最低两位后得到 `imem_word_addr[10:0]`，IMEM 根据该地址输出 32 位指令。然后 CPU 按照 MIPS 的格式规范将指令拆分为 OP、Rsc、Rtc、IM_15_11、sa、FUNC、Imm16 和 target 等字段。Decoder 根据指令编码识别具体指令，Controller 再结合译码结果、比较结果及忙碌状态产生控制信号。

寄存器堆根据 Rsc 和 Rtc 读取 Rs、Rt 两个 32 位操作数，目的寄存器由 `rdc_sel` 控制的多路选择器确定。ALU 的两个输入分别由 M3、M4 选择，具体运算由 ALUC 控制。运算结果、数据存储器读出数据、NPC、HI、LO、CP0 读出数据、CLZ 结果和 LUI 结果通过 M2 选择后写回寄存器堆。下一条指令地址由 M1 选择，可以来自 NPC、分支目标地址、跳转目标地址、寄存器内容、异常返回地址或异常入口地址。

具体到不同的指令类型，普通算术、逻辑、移位、分支和跳转指令主要通过组合数据通路完成；Load 指令利用 `load_busy` 保持 PC 并等待数据写回，乘除法指令则利用 `mdu_busy` 在运算期间保持当前 PC。这样既保留了单周期 CPU 的主要数据通路结构，也能配合数据存储器和乘除法器的实际时序。

下面就开始按照不同的指令类型来给出各指令的**数据通路图和指令流程图**：

（1）寄存器算术与逻辑指令

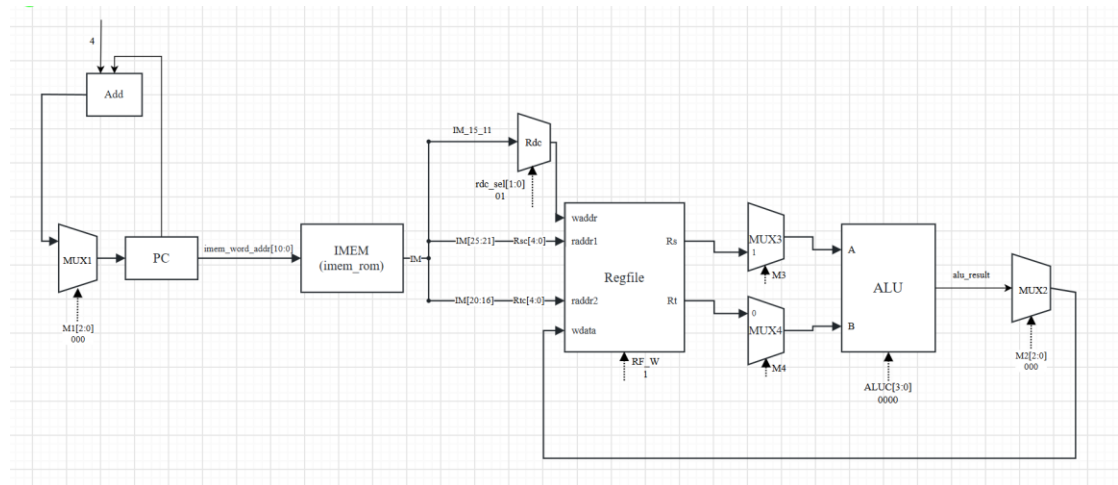
ADD、ADDU、SUB、SUBU、AND、OR、XOR、NOR、SLT、SLTU 均读取寄存器 `R[rs]` 和 `R[rt]`。M3 和 M4 分别选择 Rs 与 Rt 作为 ALU 的两个输入，ALU

根据 ALUC 完成加、减、按位逻辑或大小比较。运算结果经过 M2 写回指令 rd 字段指定的寄存器。

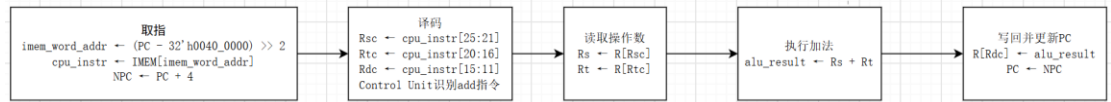
这些指令的数据通路基本相同，主要区别在 ALU 执行的运算。SLT 按照有符号数比较，SLTU 按照无符号数比较。当前实现中 ADD 与 ADDU 使用相同的加法数据通路，SUB 与 SUBU 使用相同的减法数据通路，未另外设置溢出异常处理。

①Add

数据通路图：

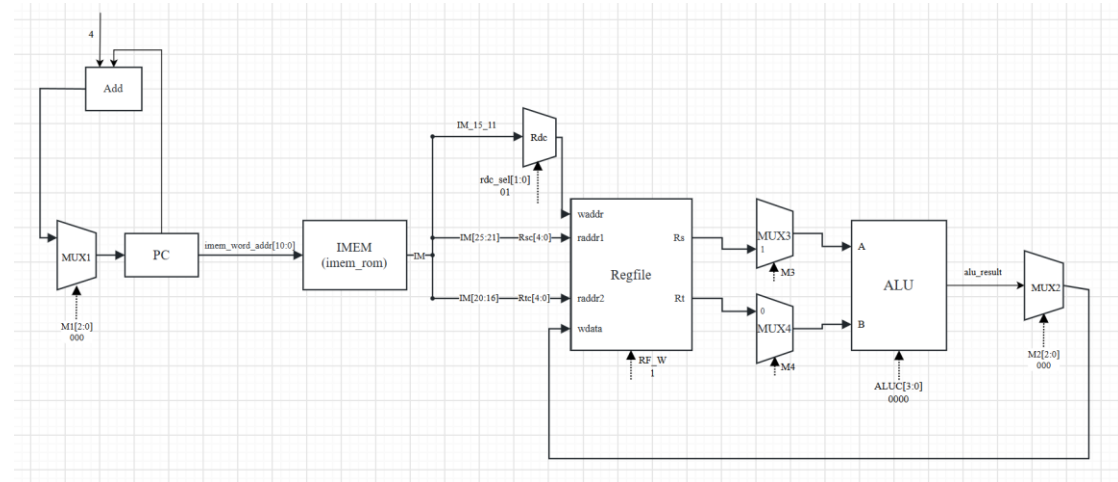


指令流程图：

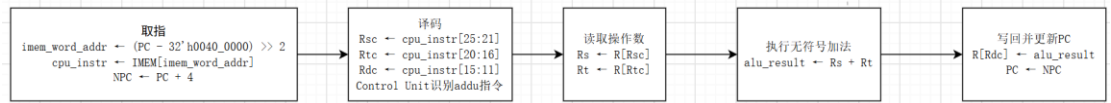


②Addu

数据通路图：

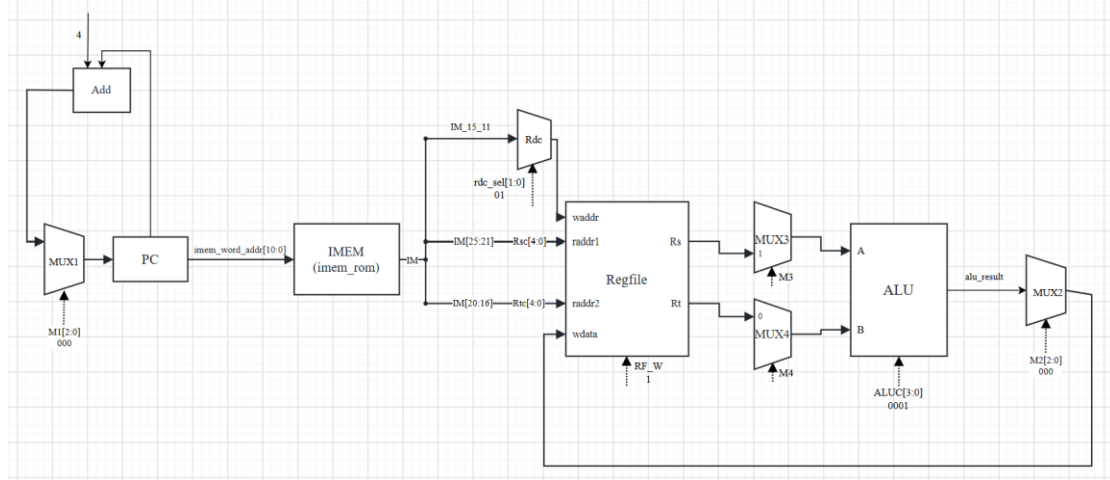


指令流程图：

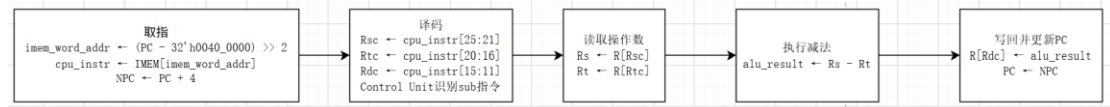


③ sub

数据通路图：

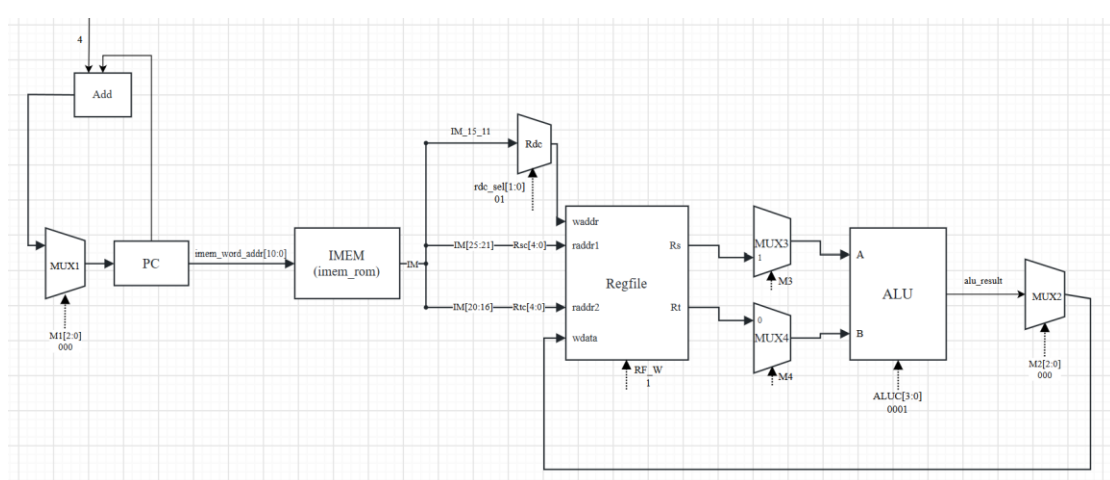


指令流程图：

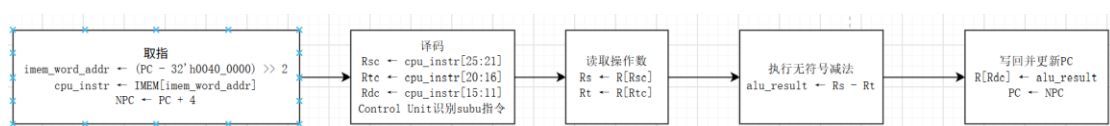


④ subu

数据通路图：

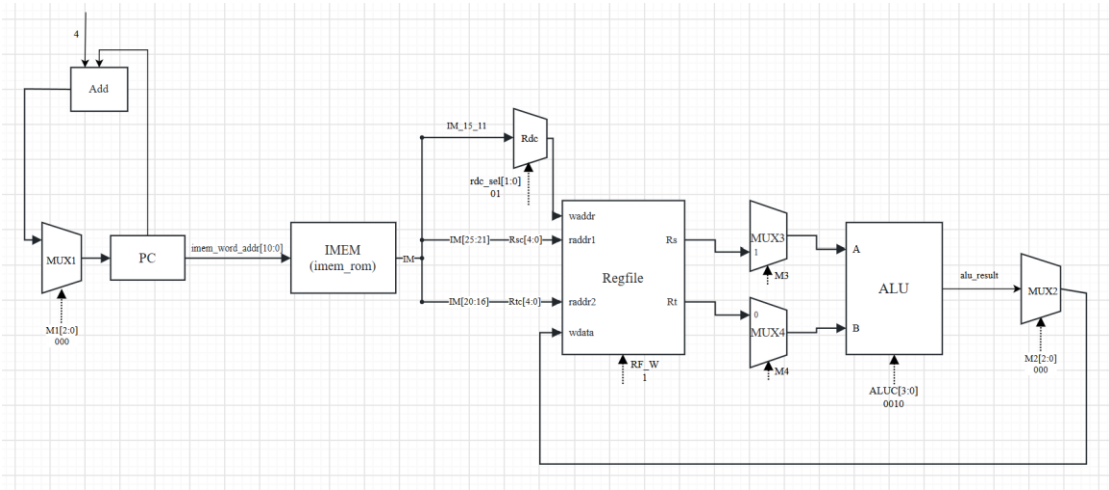


指令流程图：

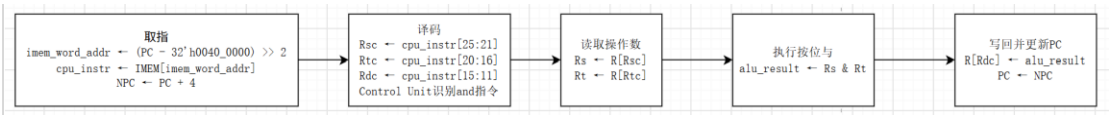


⑤ And

数据通路图：

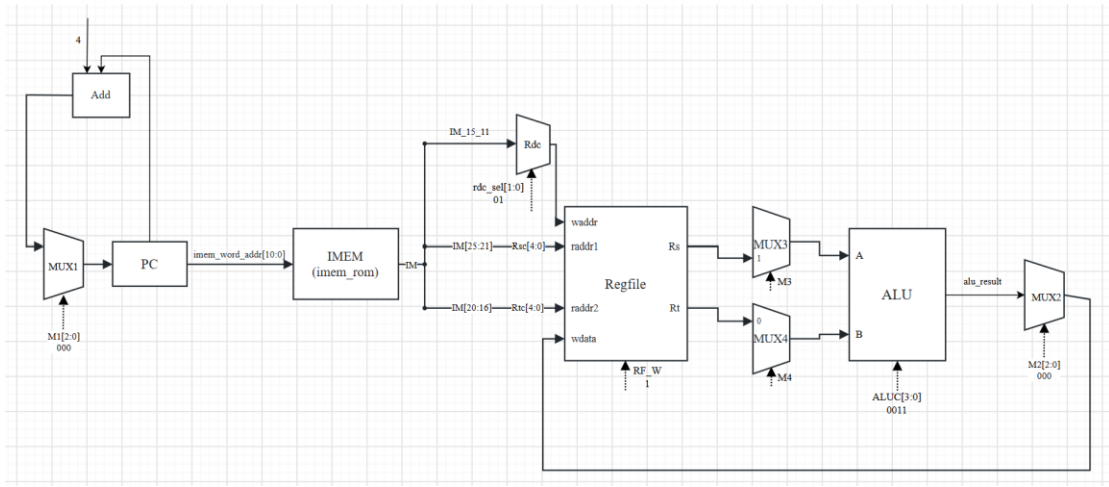


指令流程图：

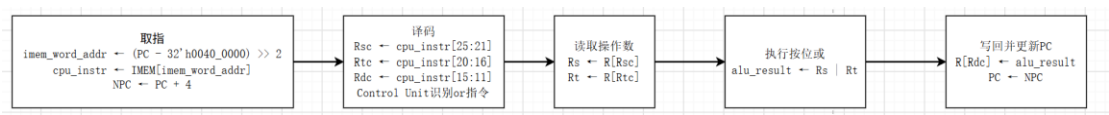


⑥ Or

数据通路图：

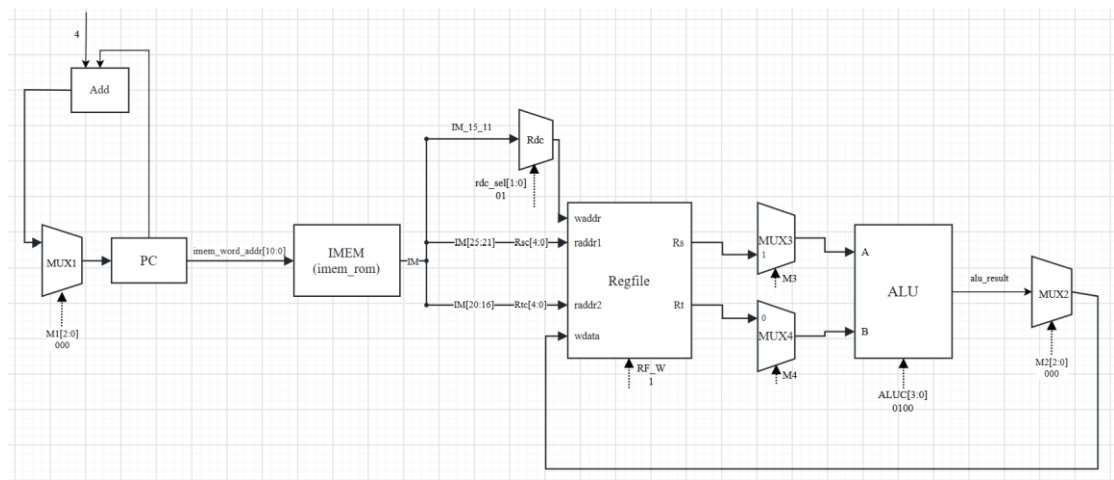


指令流程图：

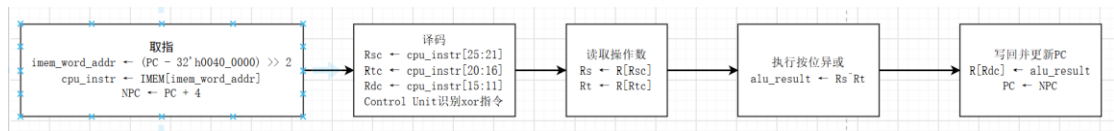


⑦ xor

数据通路图：

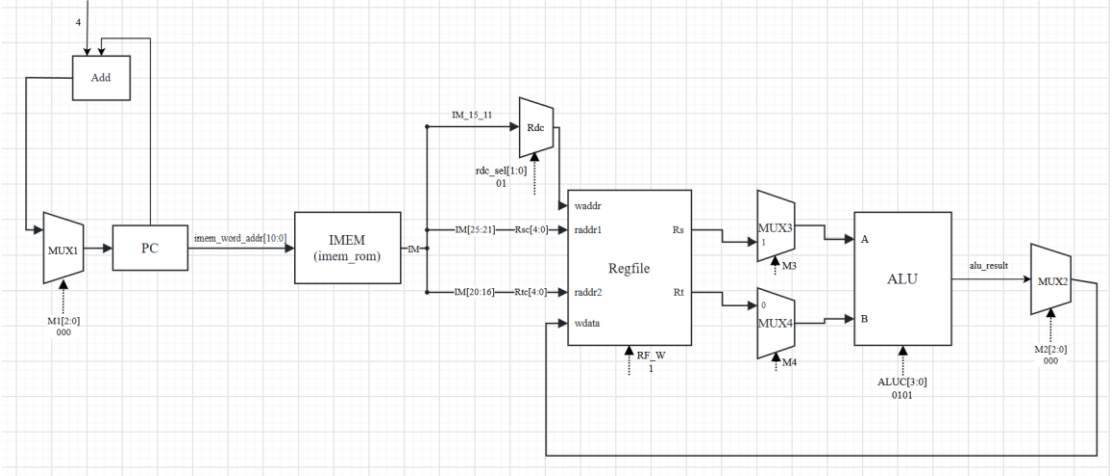


指令流程图：

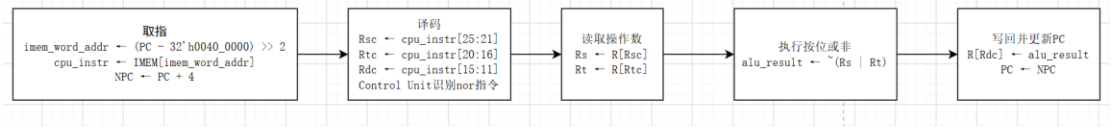


⑧ nor

数据通路图：

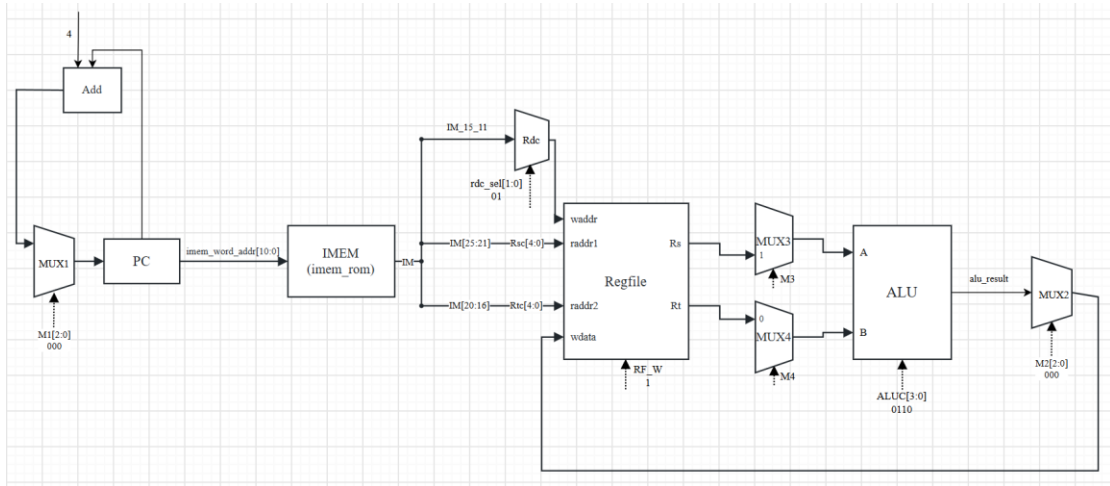


指令流程图：

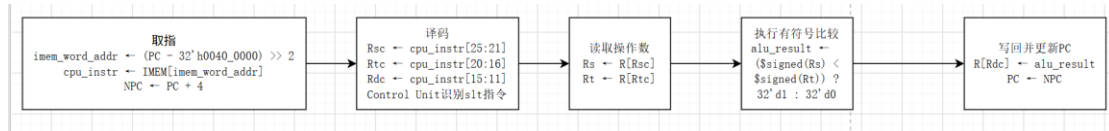


⑨ SLT

数据通路图：

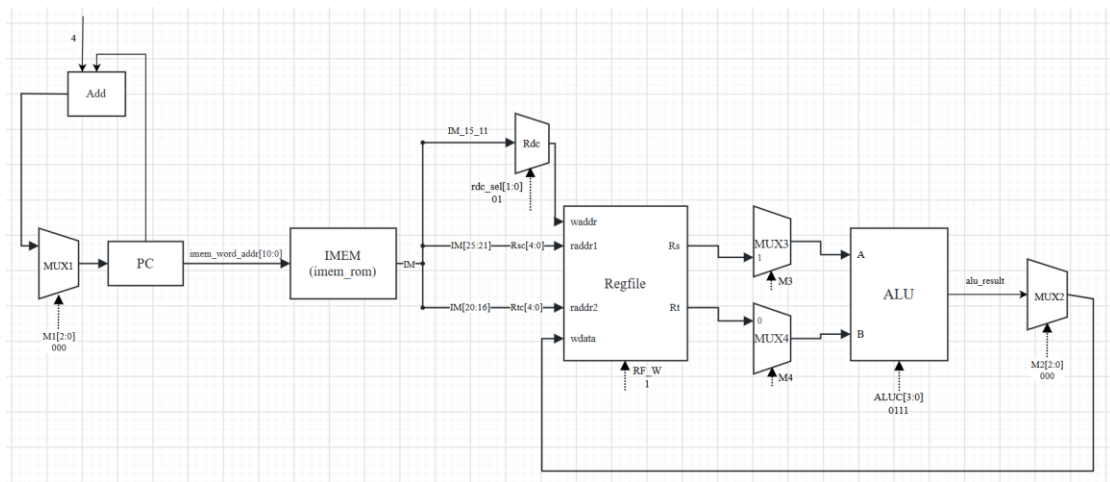


指令流程图：

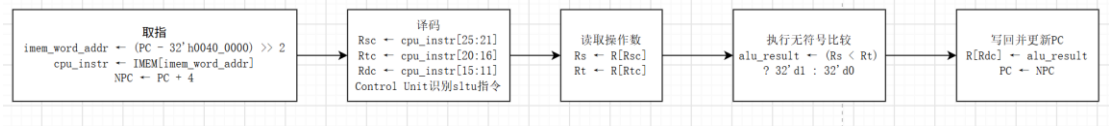


⑩ SLTU

数据通路图：



指令流程图：



(2) 移位指令

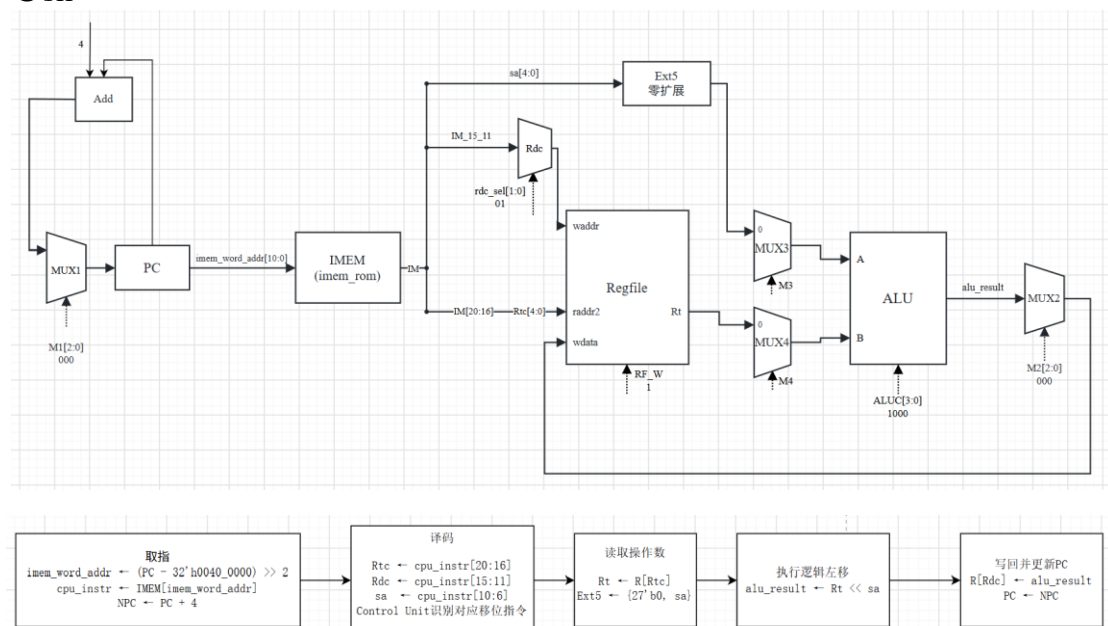
SLL、SRL、SRA、SLLV、SRLV、SRAV 均以 $R[rt]$ 作为被移位的 32 位数据，并将移位结果写入 $R[rd]$ 。立即移位指令从指令的 $sa[4:0]$ 字段直接取得移位数，经过 Ext5 零扩展后送入 ALU；可变移位指令则读取 $R[rs]$ ，使用 $Rs[4:0]$ 作为移位数。

SLL 与 SLLV 执行逻辑左移，SRL 与 SRLV 执行逻辑右移，SRA 与 SRAV 执行算术右移。六条指令的主要区别是

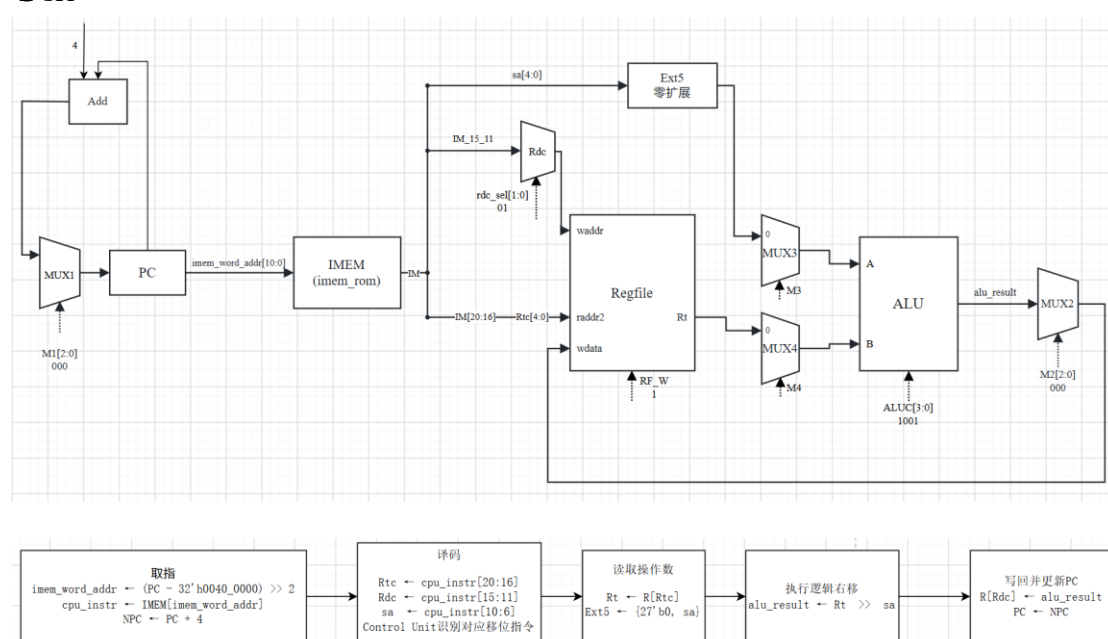
移位数来源（加 V 的是 **Variable**，从数字可变的寄存器里拿值）和右移时高位的填充方式（**left logic/right logic/right arthimatics**）。

下面各指令都默认第一张是数据通路图，第二张是指令流程图：

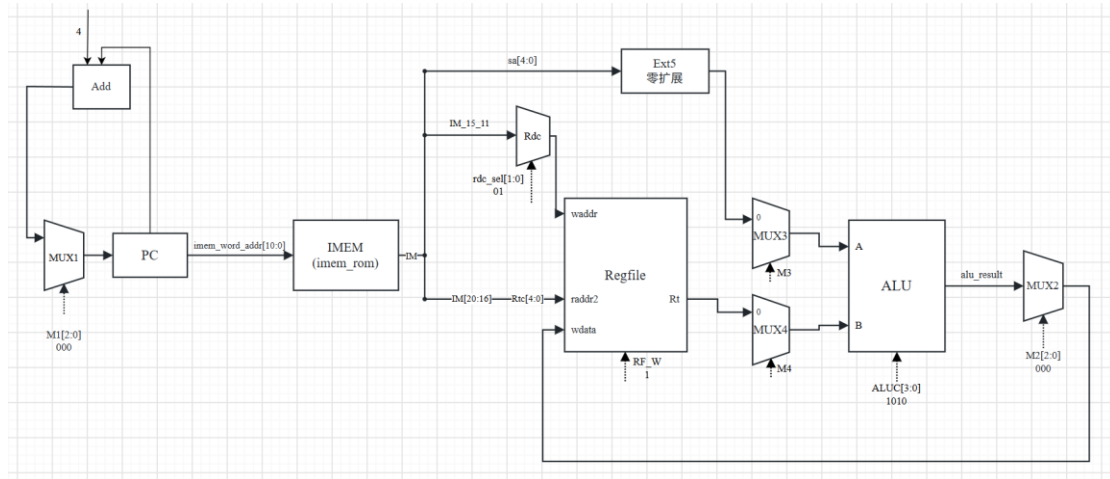
① sll



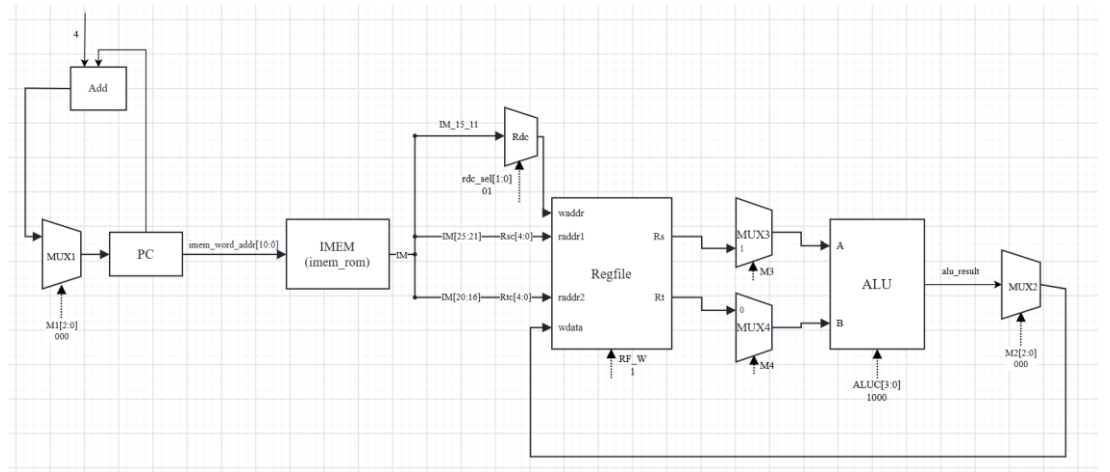
② srl



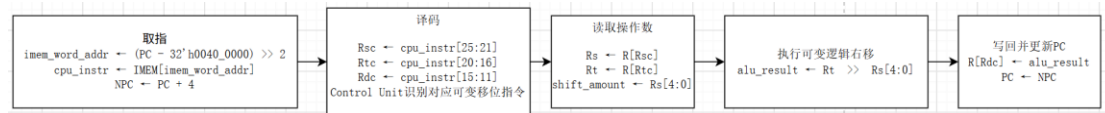
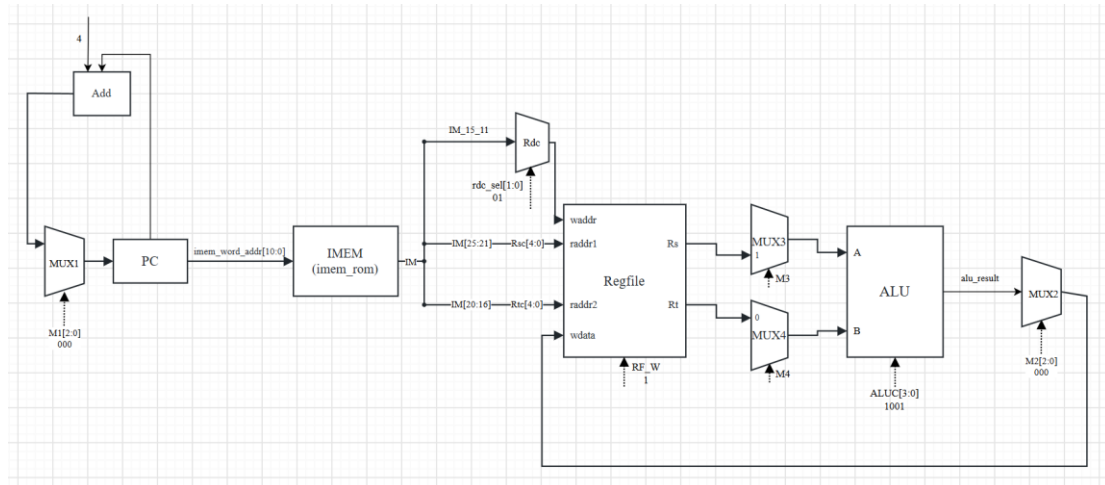
③ sra



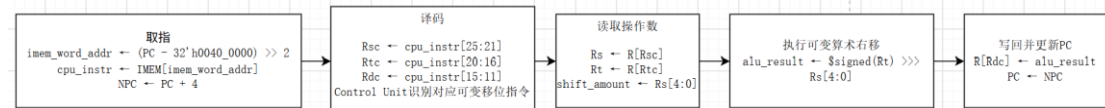
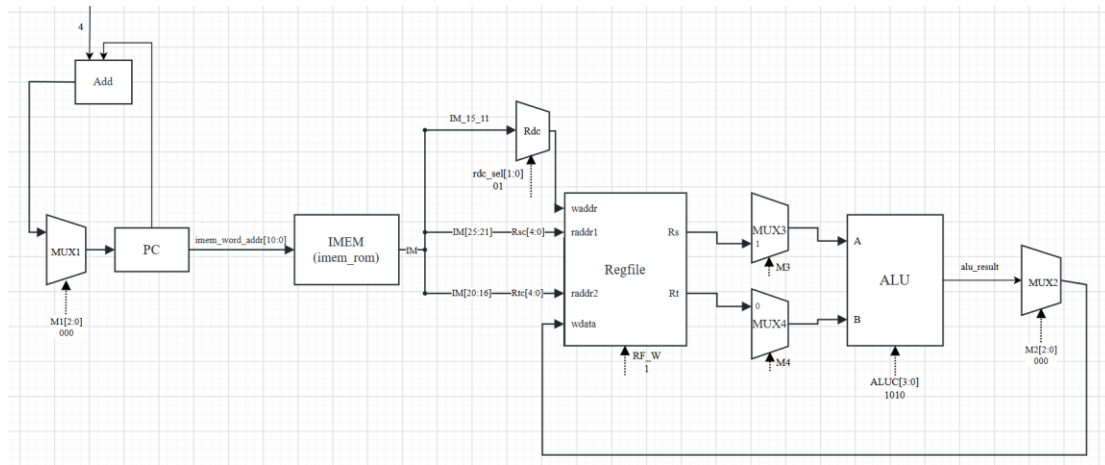
④ sllv



⑤ srlv



⑥ srav

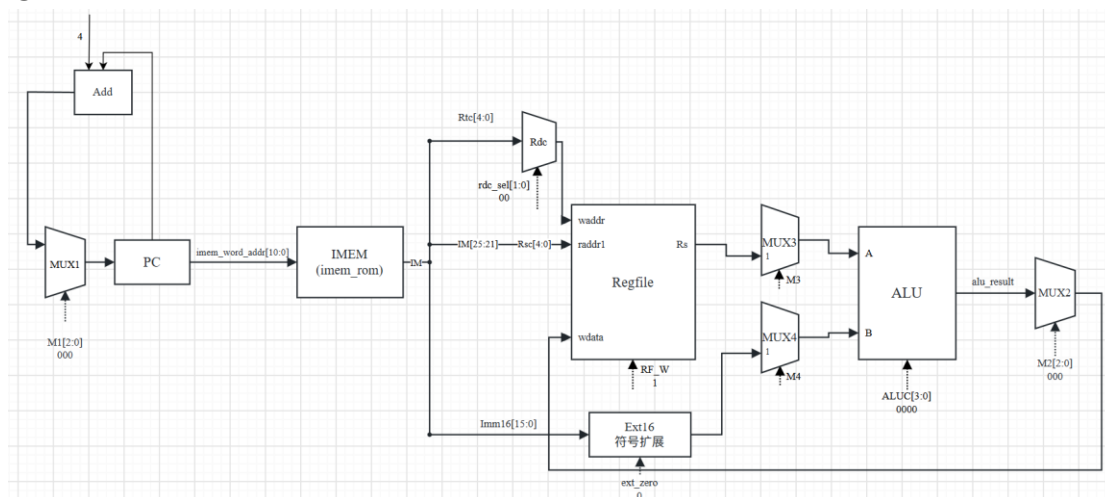


(3) 立即数指令

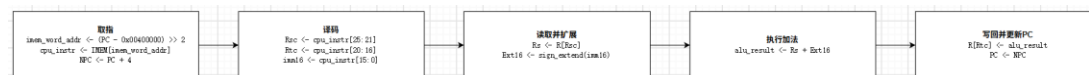
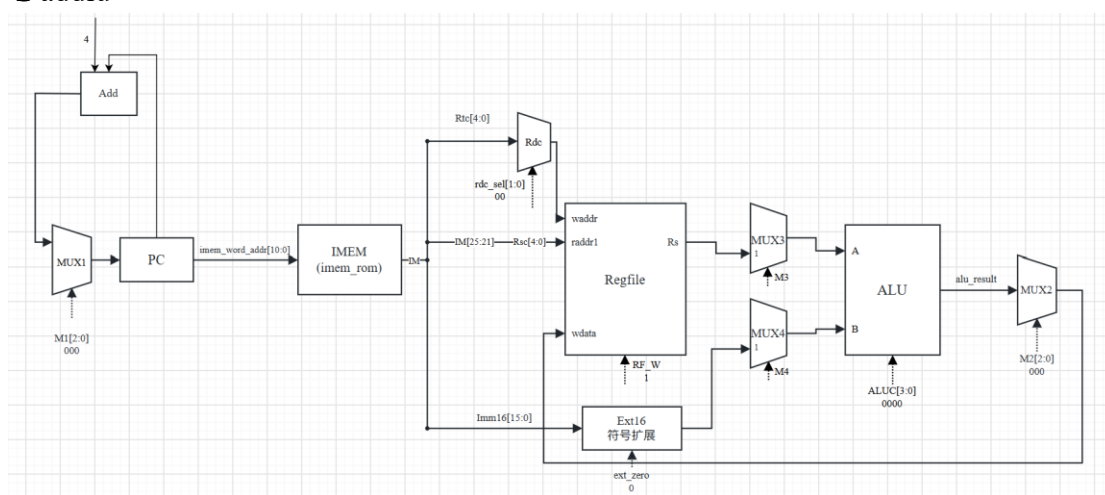
ADDI、ADDIU、SLTI、SLTIU、ANDI、ORI、XORI、LUI 使用指令中的 16 位立即数。ADDI、ADDIU、SLTI、SLTIU 对立即数进行符号扩展，ANDI、ORI、XORI 对立即数进行零扩展。扩展后的立即数经 M4 送入 ALU，与 R[rs]进行算术、比较或逻辑运算，结果写入 R[rt]。

LUI 不需要读取 R[rs]，而是将 Imm16 放在结果的高 16 位、低 16 位补零，再通过 M2 直接写入 R[rt]。

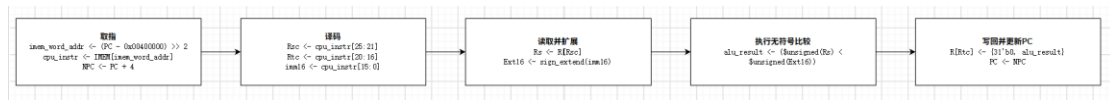
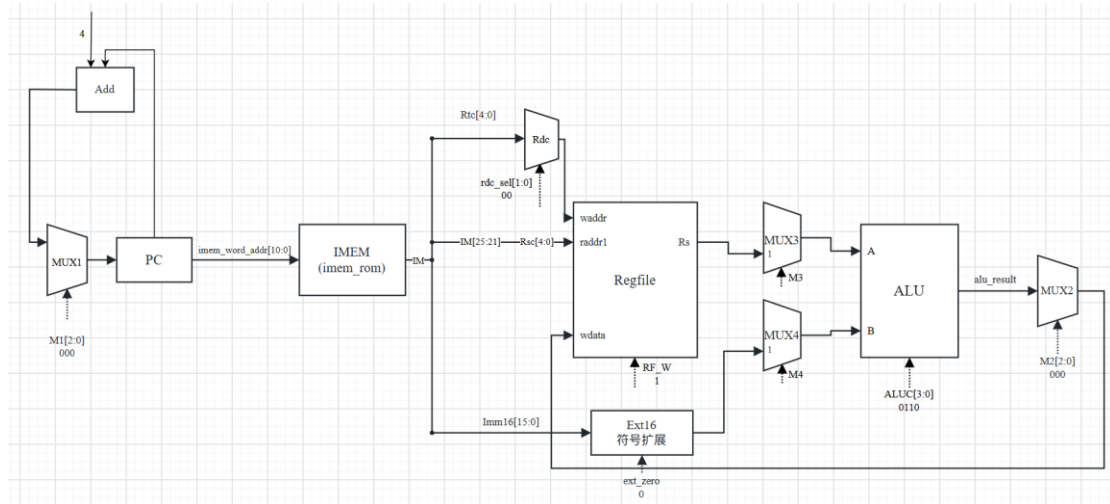
①addi



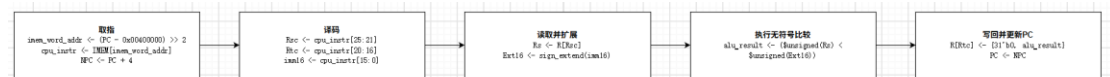
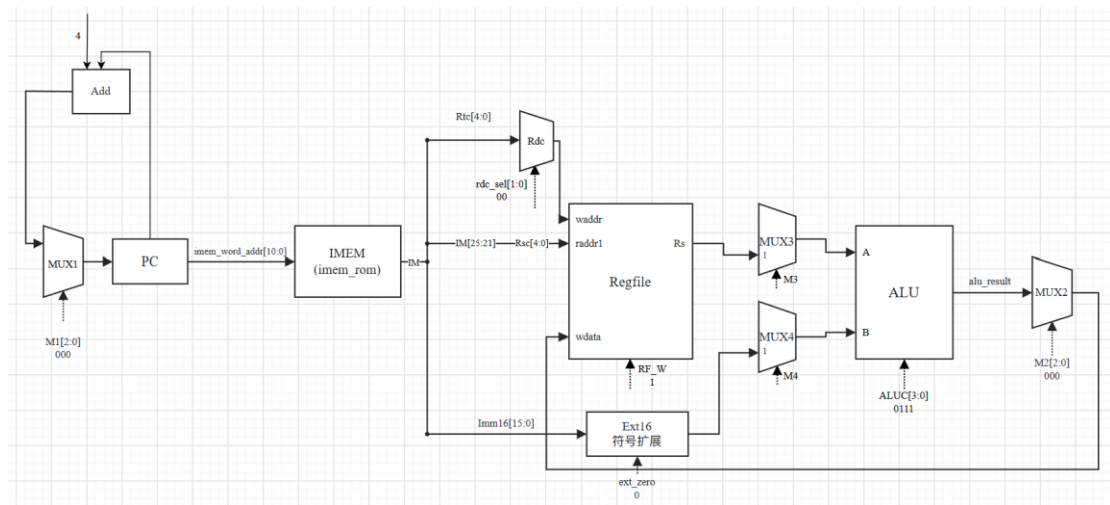
②addiu



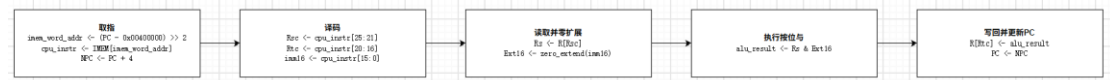
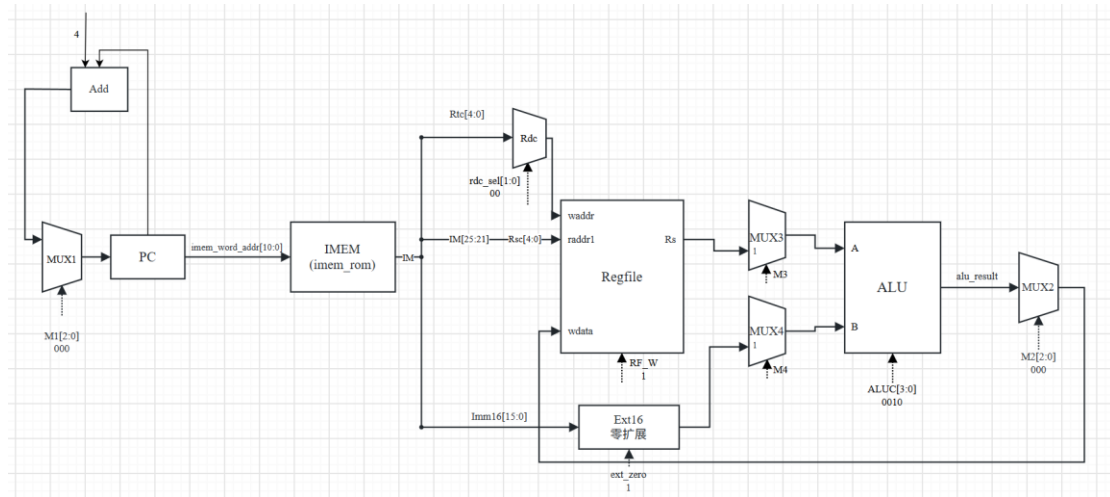
③ slti



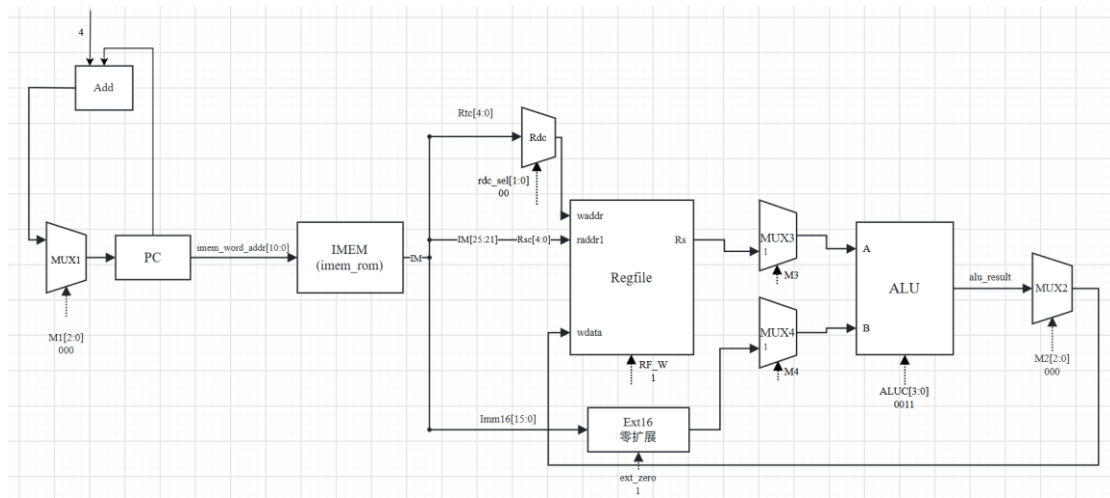
④ sltiu



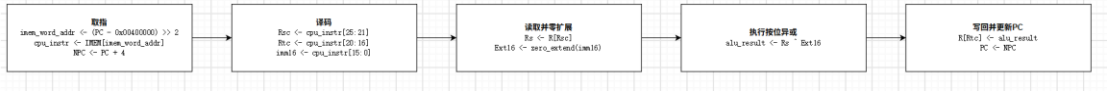
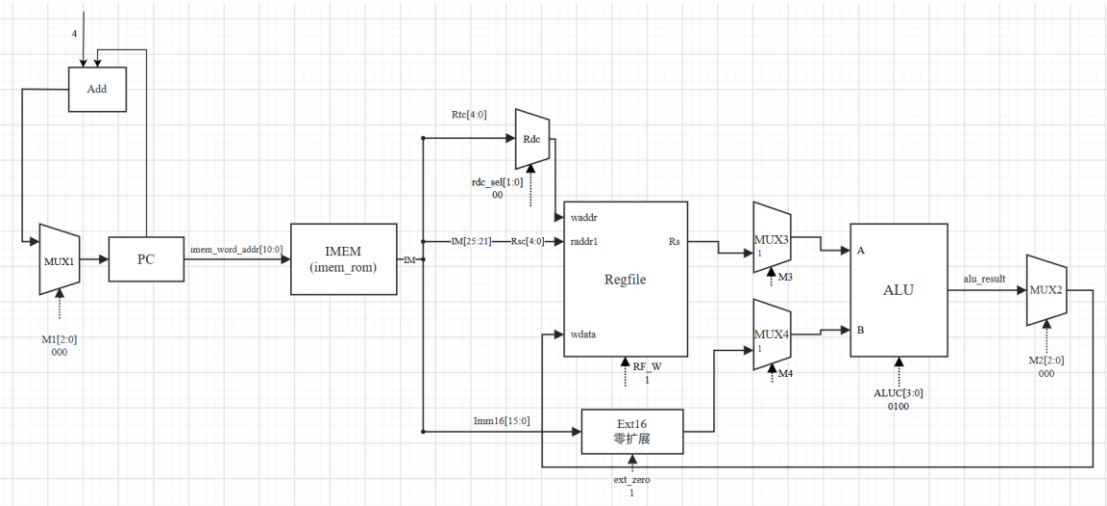
⑤ andi



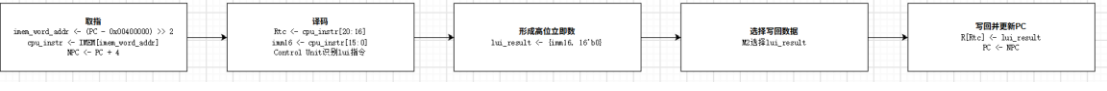
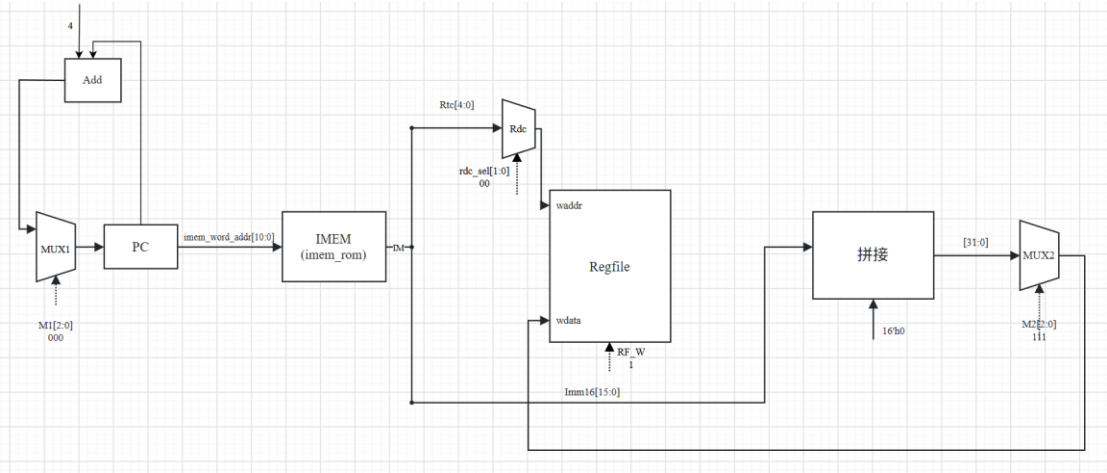
⑥ ori



⑦ xori



⑧ lui



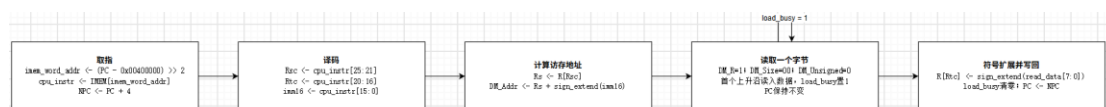
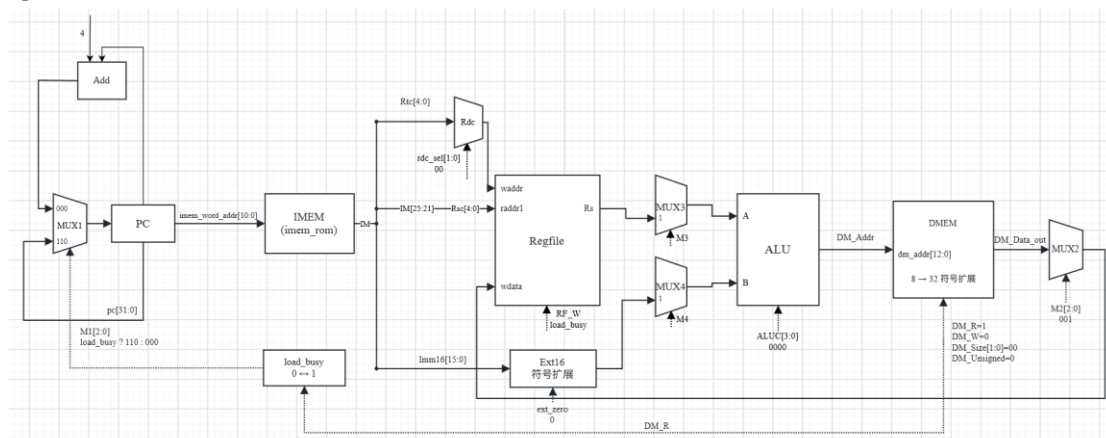
(4) 访存指令

LB(load byte)、LH(load half word)、LW(load word)、LBU、LHU、SB、SH、SW 首先读取基址寄存器 R[rs]，将 16 位偏移量进行符号扩展，然后由 ALU 计算有效地址： $DM_Addr = R[rs] + SignExt(Imm16)$

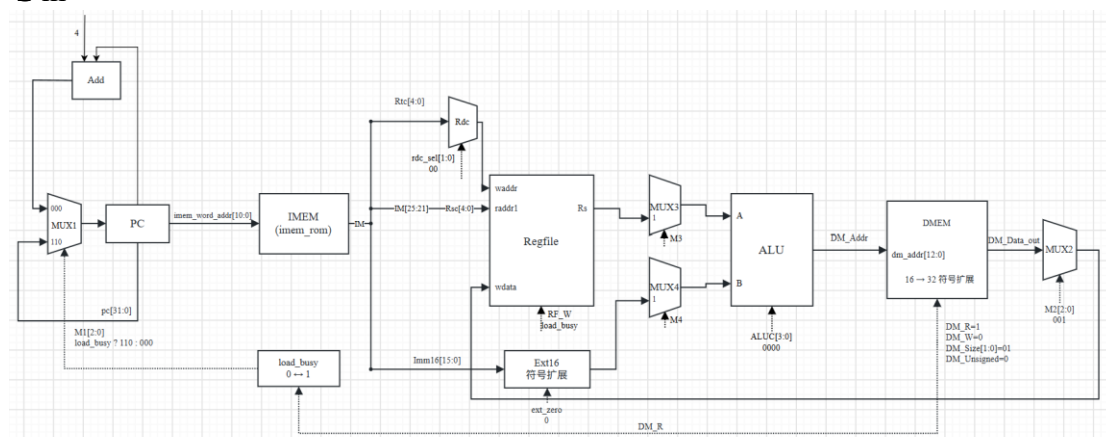
Load 指令根据有效地址从 DMEM 读取数据，并通过 M2 写入 R[rt]。LB 和 LH 分别对读出的字节和半字进行符号扩展，LBU 和 LHU 进行零扩展，LW 直接读出 32 位字。由于当前数据存储器的读取结果在时钟沿后更新，Load 指令使用 load_busy 保持 PC，并在随后一个时钟沿完成寄存器写回。

Store 指令同样利用 ALU 计算地址，但不写寄存器。R[rt]作为 DM_Data_in 送入 DMEM，SB、SH、SW 分别写入一个字节、两个字节和四个字。DM_Size 用于指定访问宽度，DM_R 和 DM_W 用于区分读写操作。

①lb

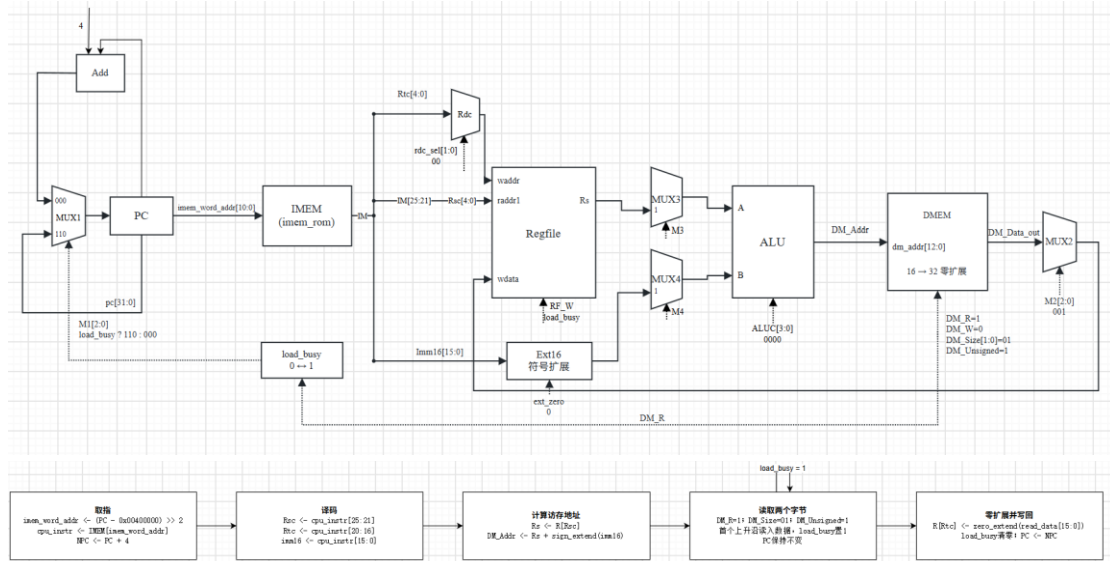


②lh

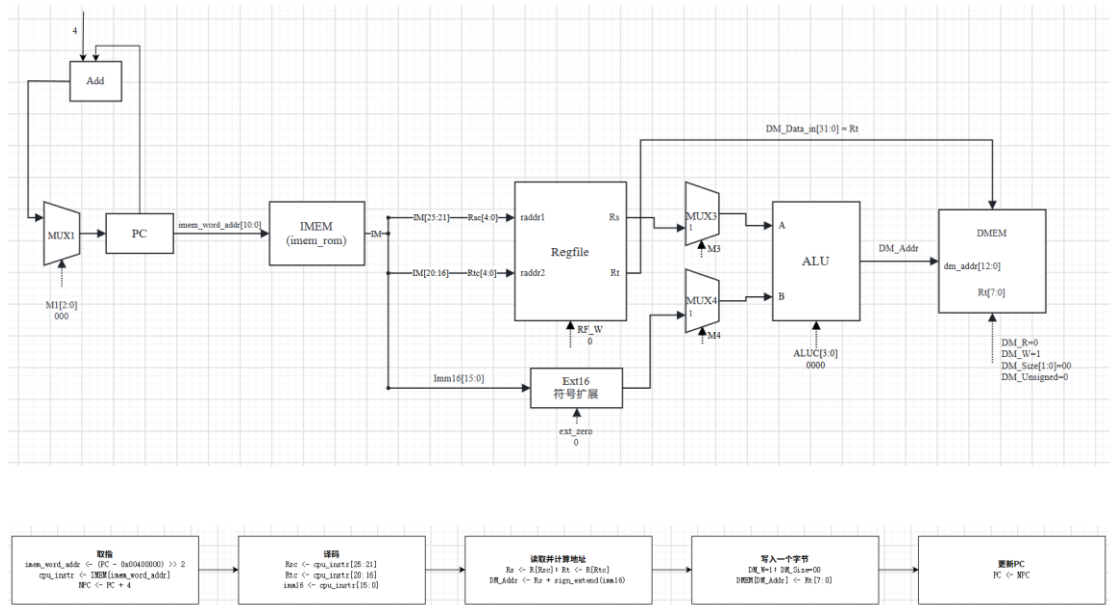


[illegible][illegible]

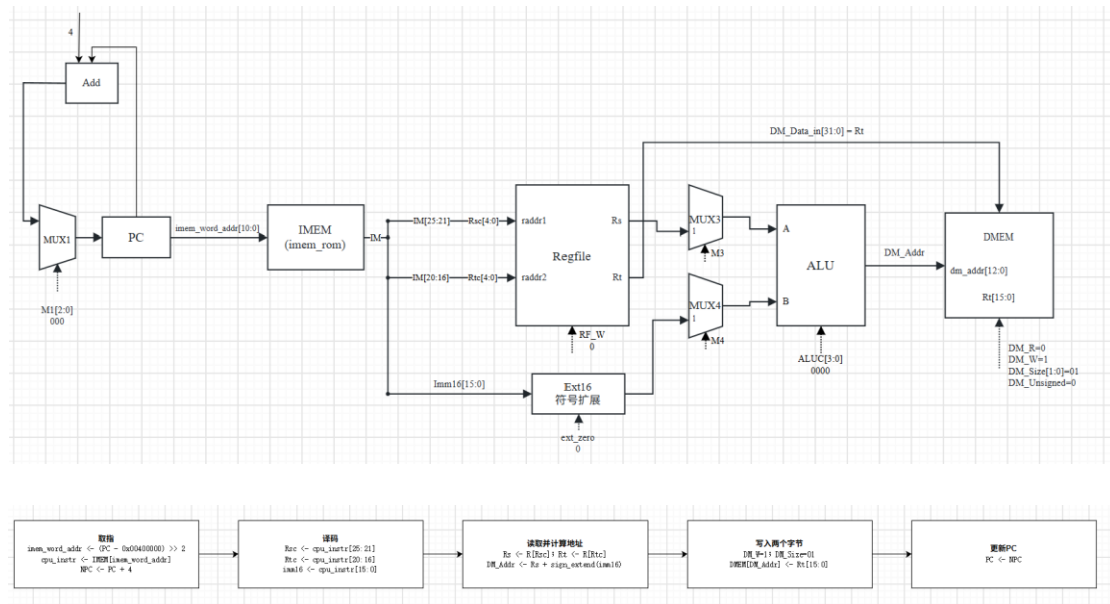
⑤ lhu



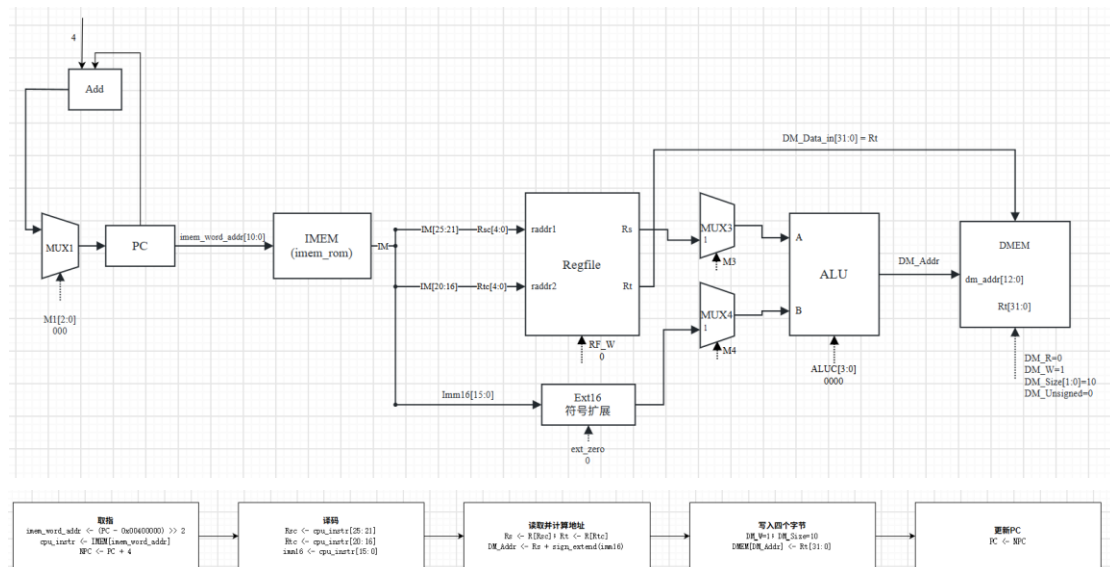
⑥ sb



⑦sh



⑧sw

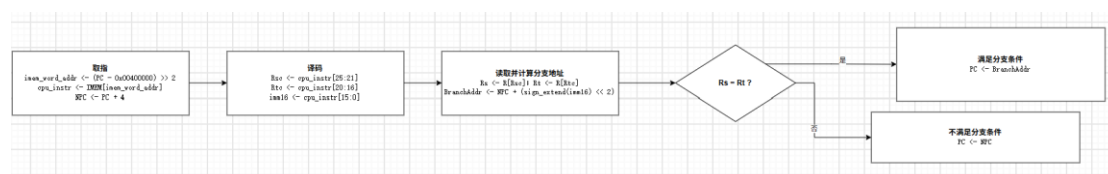
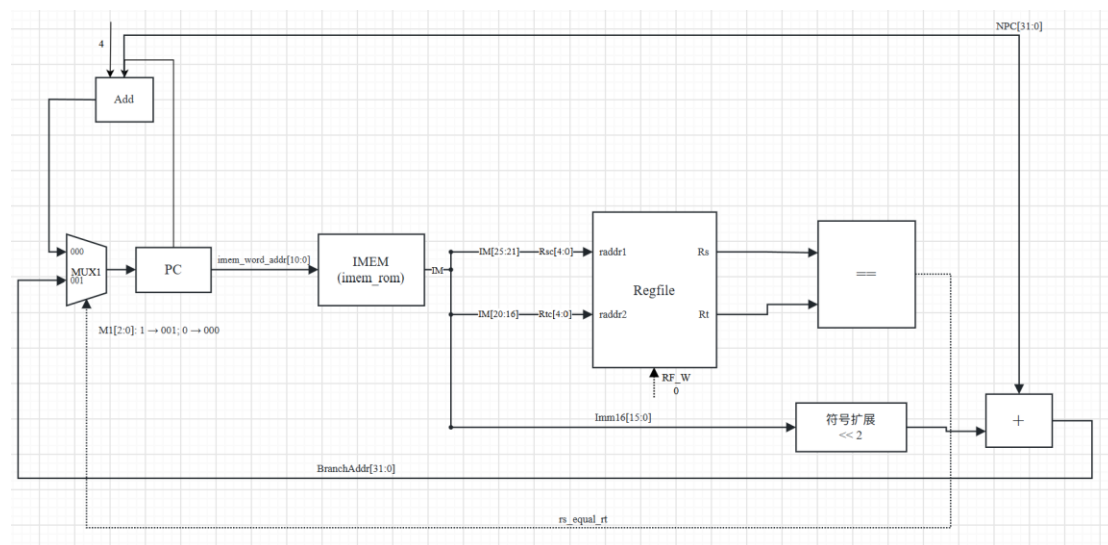


(5) 分支指令

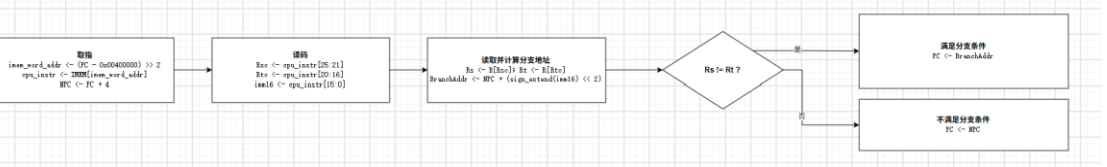
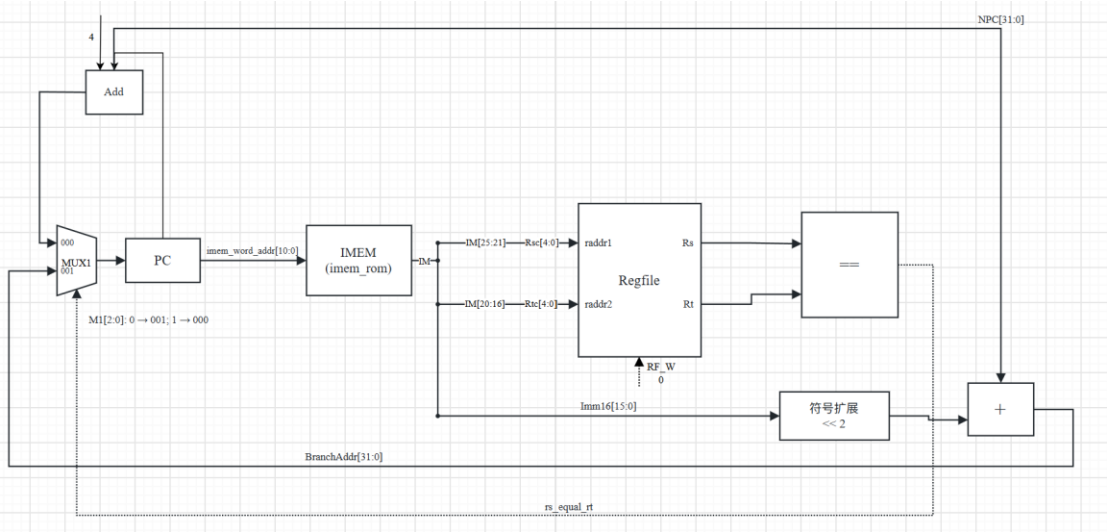
BEQ、BNE、BGEZ 均使用 PC 相对寻址。分支目标地址由 NPC 与经过符号扩展、左移两位后的立即数相加得到： $\text{BranchAddr} = \text{NPC} + (\text{SignExt}(\text{Imm16}) \ll 2)$

BEQ 和 BNE 读取 R[rs]与 R[rt]并判断二者是否相等；BGEZ 读取 R[rs]并检查其最高位，从而判断该数是否为非负数。当分支条件成立时，M1 选择 BranchAddr 作为下一条指令地址，否则继续选择 NPC。三条指令只改变 PC，不写入通用寄存器。

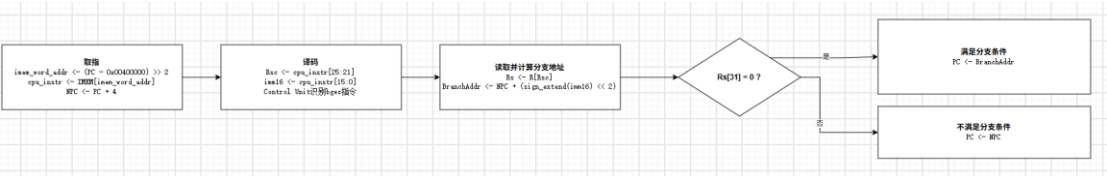
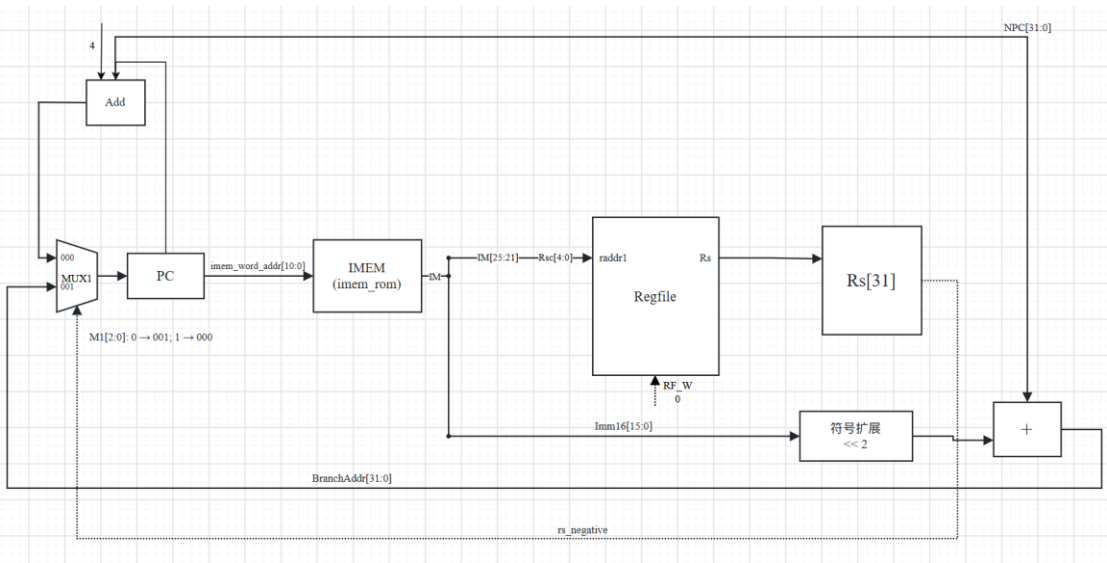
①BEQ



②BNE



③BGEZ

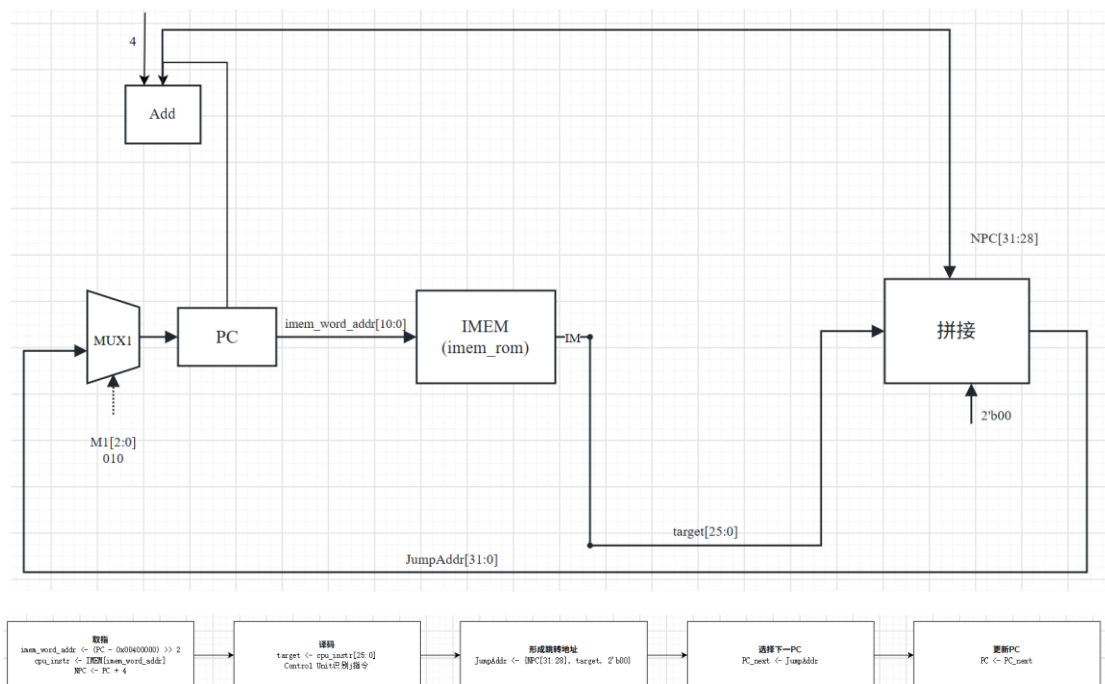


（6）跳转指令

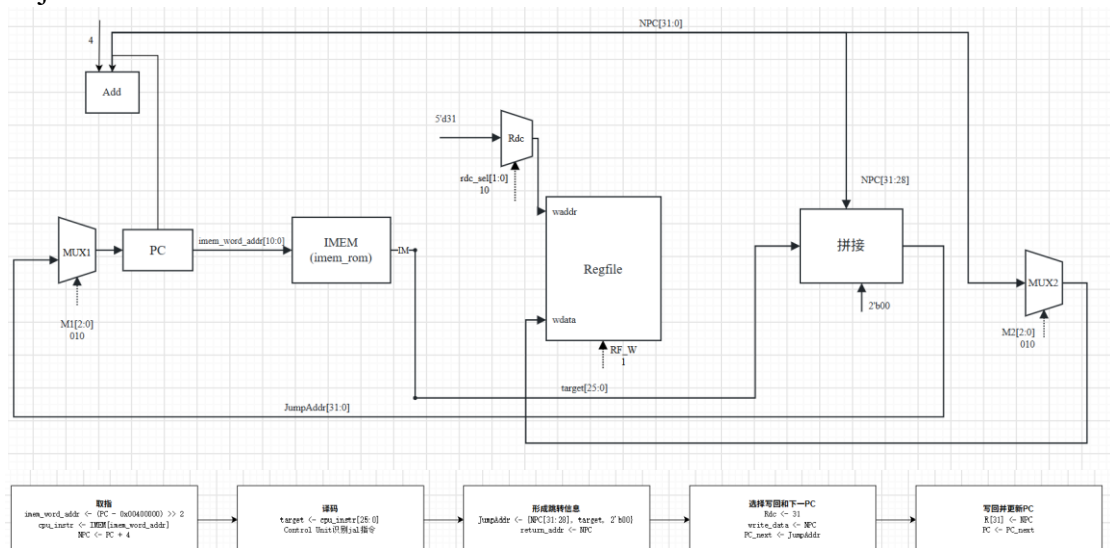
J 和 JAL 使用指令中的 26 位 **target** 字段，将其左移两位后与 NPC[31:28]拼接形成跳转目标地址。JR 和 JALR 从 R[rs]读取完整的 32 位目标地址，并由 M1 将该地址送入 PC。

JAL 和 JALR 在跳转的同时还需要保存返回地址。两条指令均将 NPC 作为链接地址写入寄存器，其中 JAL 固定写入 31 号寄存器，JALR 优先写入 rd 字段指定的寄存器；在 rd 为 0 时选择 31 号寄存器（因为 **MIPS 不改变 0 号寄存器**）。

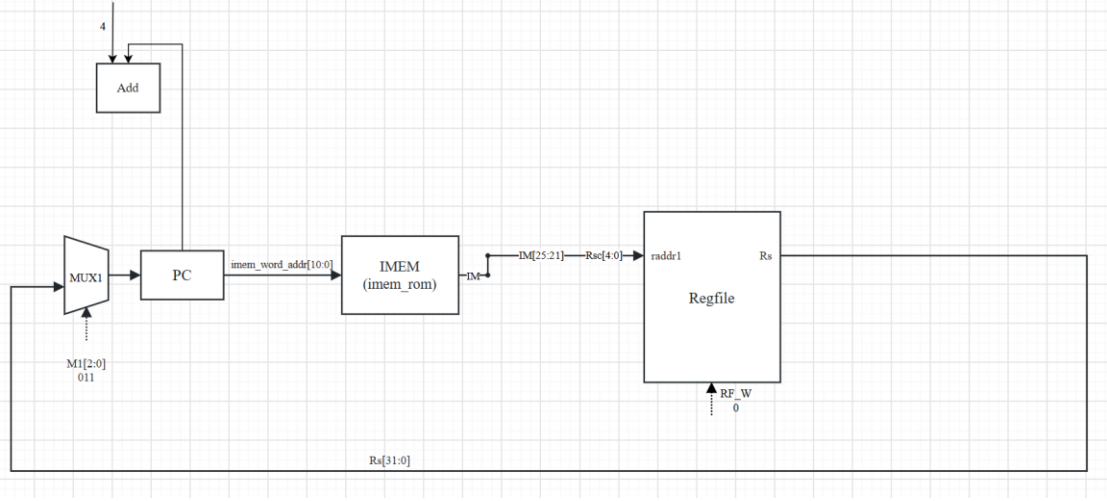
①j



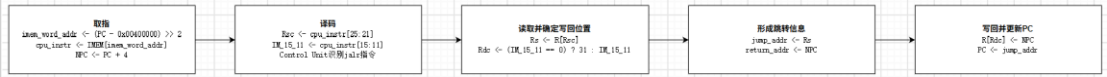
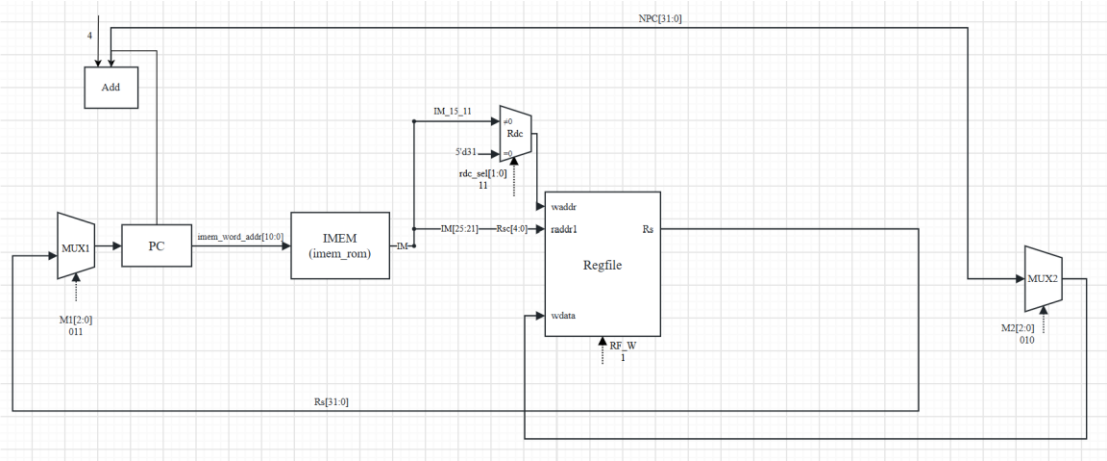
②jal



③jr



④jalr

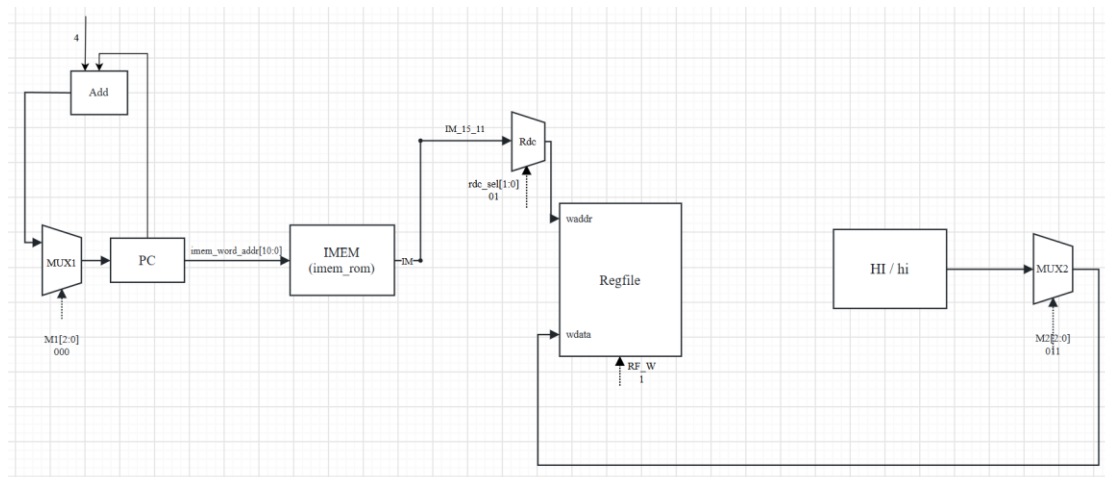


(7) HI/LO 及乘除法指令

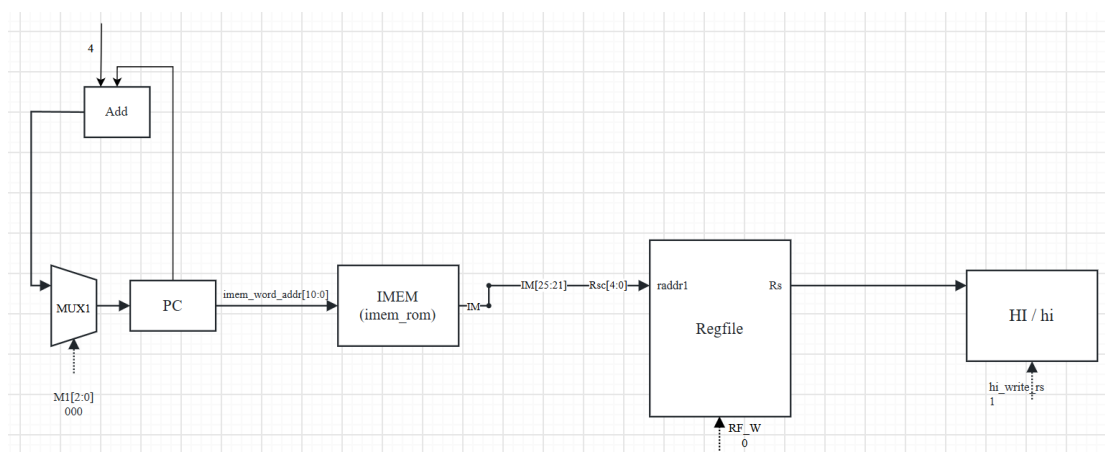
MFHI、MTHI、MFLO、MTLO、MULT、MULTU、DIV、DIVU 使用 HI 和 LO 两个专用寄存器。MFHI 和 MFLO 分别读取 HI 或 LO，并通过 M2 将数据写入 R[rd]；MTHI 和 MTLO 则把 R[rs]写入对应的专用寄存器。

乘除法指令读取 R[rs]和 R[rt]作为两个操作数。MULT 和 MULTU 分别执行有符号和无符号乘法，64 位乘积的高 32 位写入 HI，低 32 位写入 LO。DIV 和 DIVU 分别执行有符号和无符号除法，余数写入 HI，商写入 LO。运算期间 mdu_busy 保持 PC 不变，运算完成后再更新 HI、LO 并继续执行下一条指令。

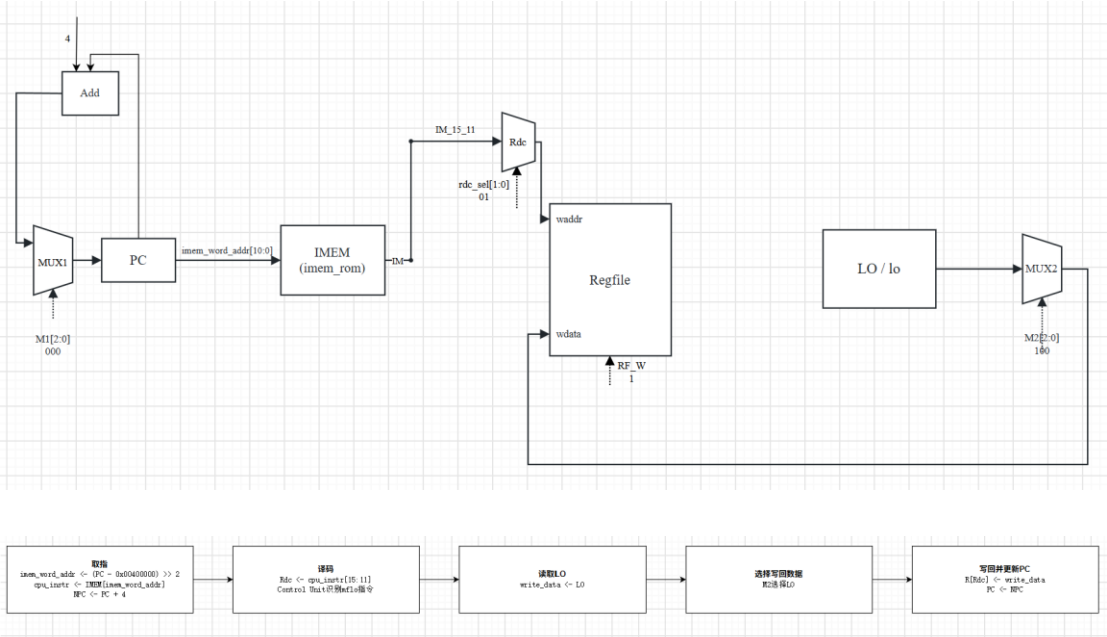
①mfhi



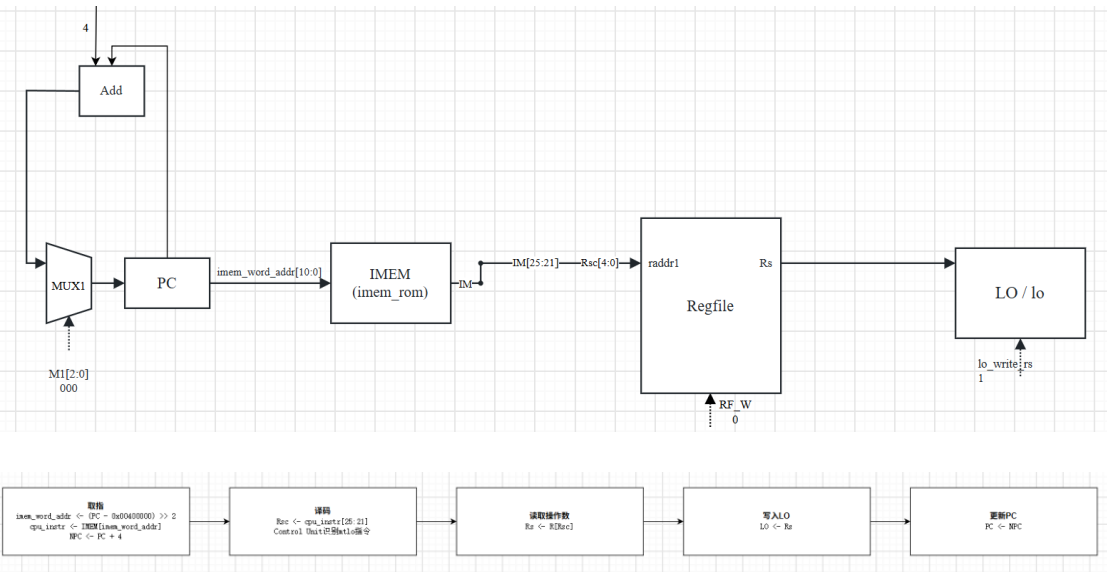
②mthi



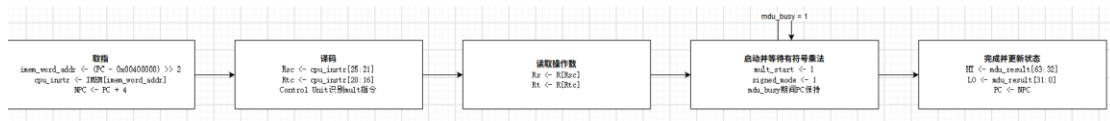
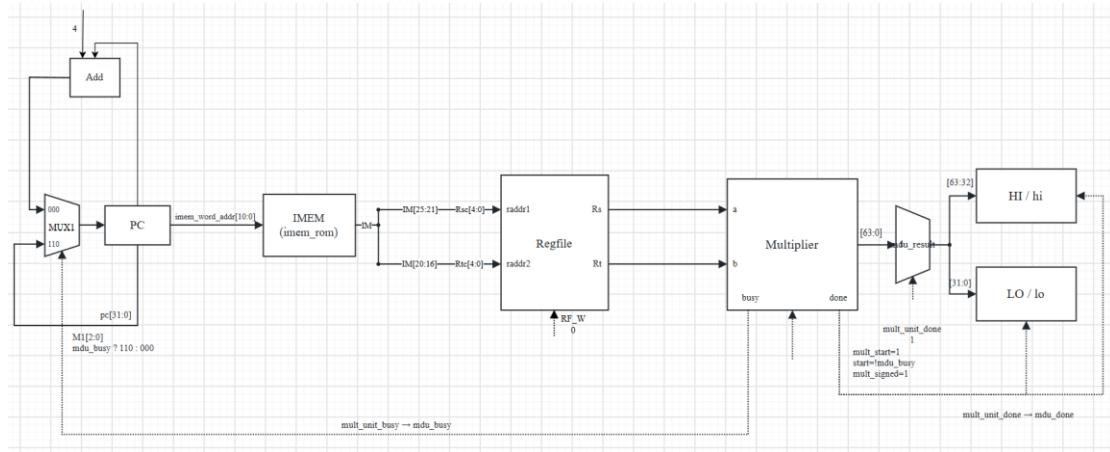
③mflo



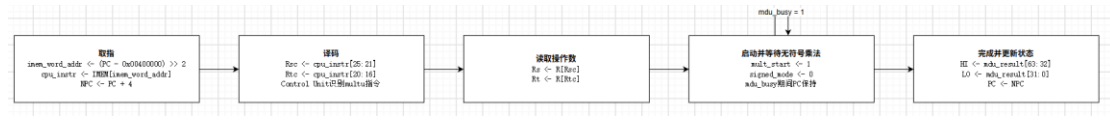
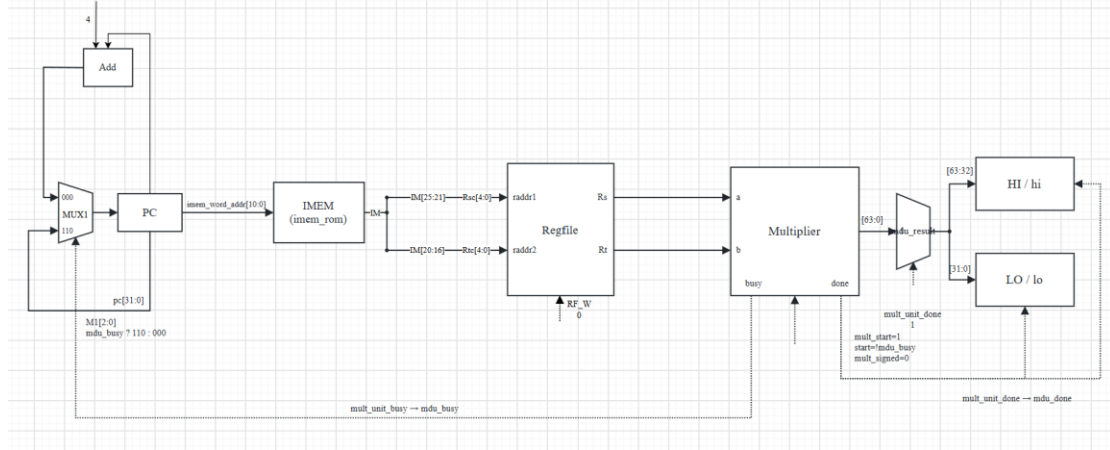
④mtlo



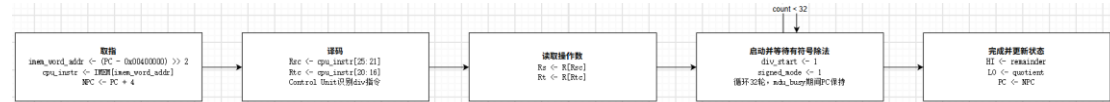
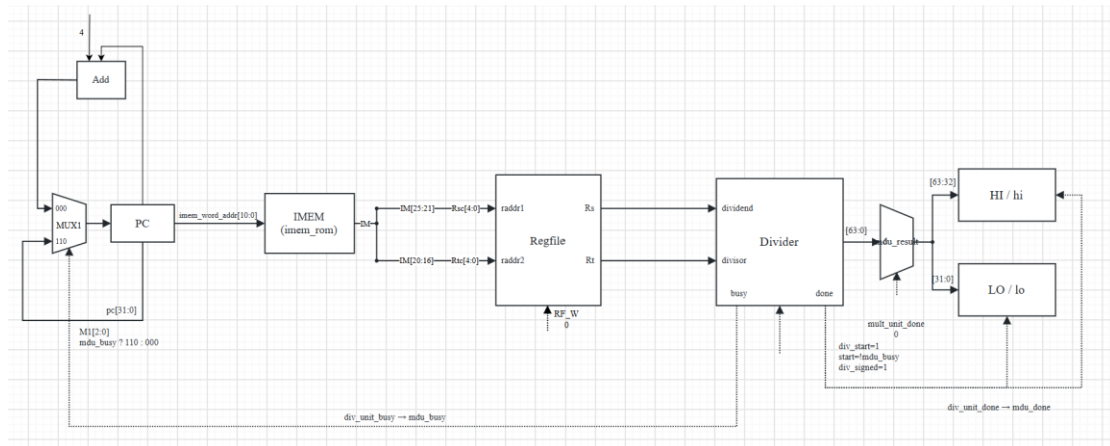
⑤ mult



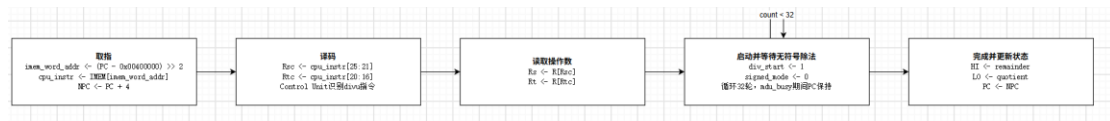
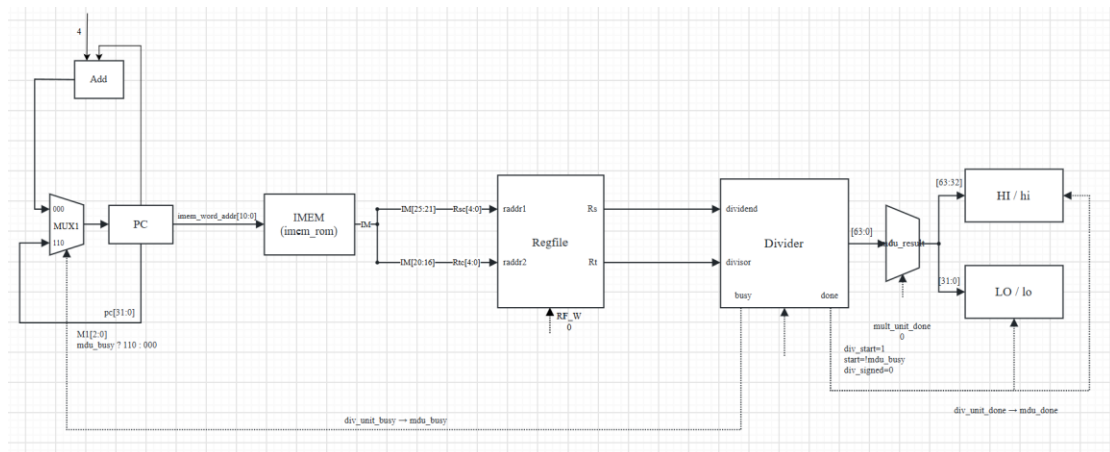
⑥ multu



⑦div



⑧divu



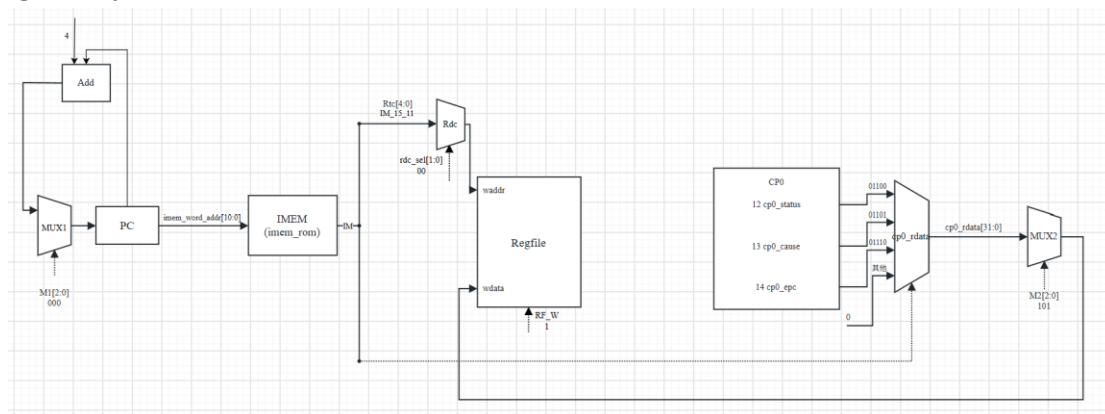
(8) CP0、异常与特殊指令

MFC0 通过 IM_15_11 选择 CP0 中的 Status、Cause 或 EPC 寄存器，并将读出的数据写入 R[rt]；MTC0 则将 R[rt]写入相应的 CP0 寄存器。

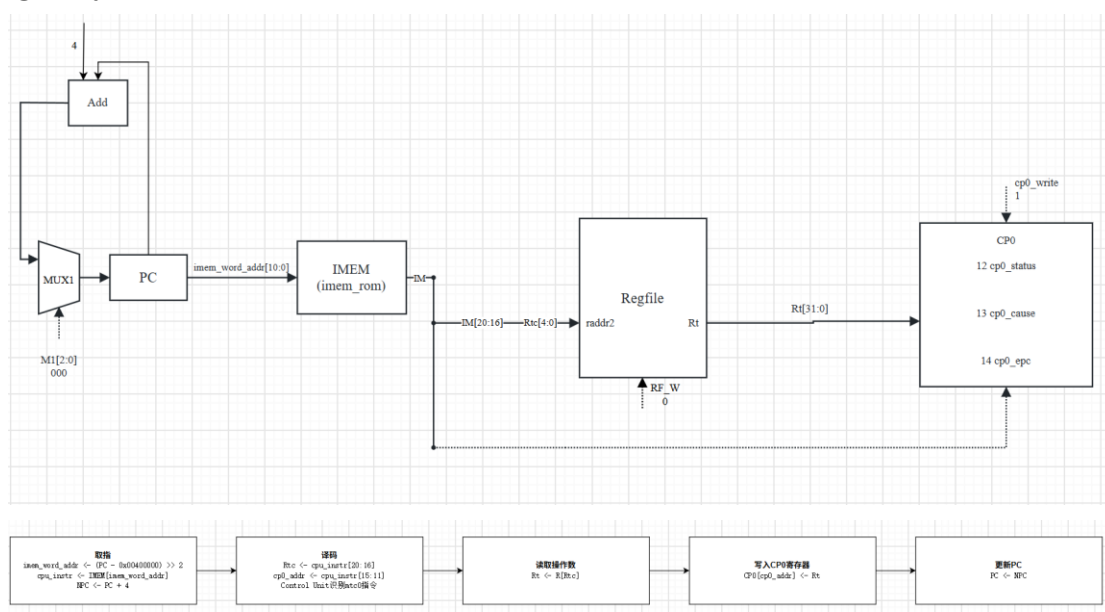
SYSCALL、BREAK、TEQ 用于产生异常。异常发生时，CPU 记录异常原因和当前 PC，将 Status 寄存器左移五位，并通过 M1 跳转至异常入口地址。TEQ 只有在 R[rs]与 R[rt]相等时才触发异常。ERET 用于异常返回，CPU 将 Status 右移五位，并把 cp0_epc+4 作为新的 PC。

CLZ 统计 R[rs]从最高位开始连续出现的零的数量，并将 32 位计数结果写入 R[rd]。该指令不使用 ALU 的常规算术结果，而是通过独立的 CLZ 组合逻辑生成写回数据。

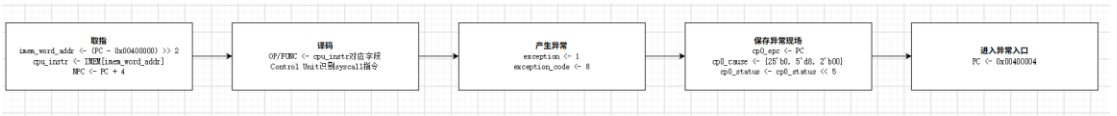
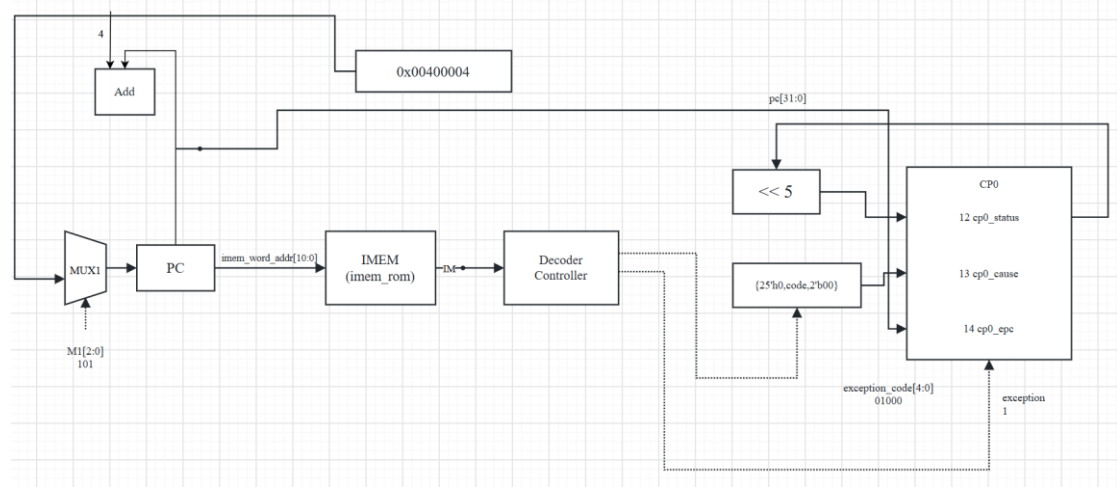
①MFC0



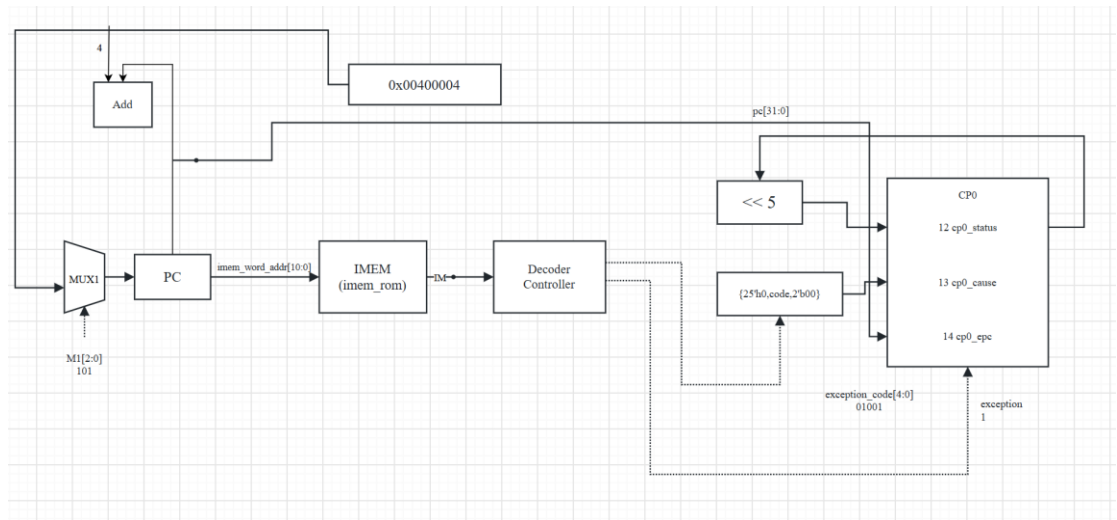
②mtc0

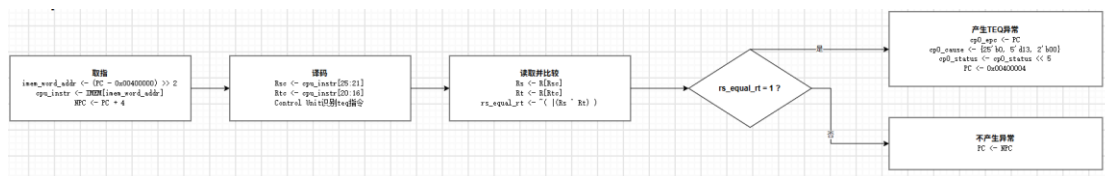
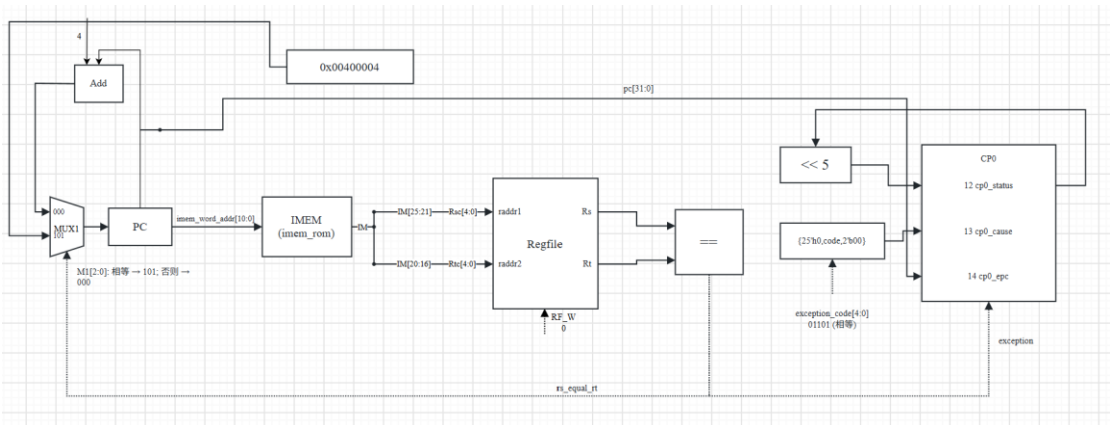
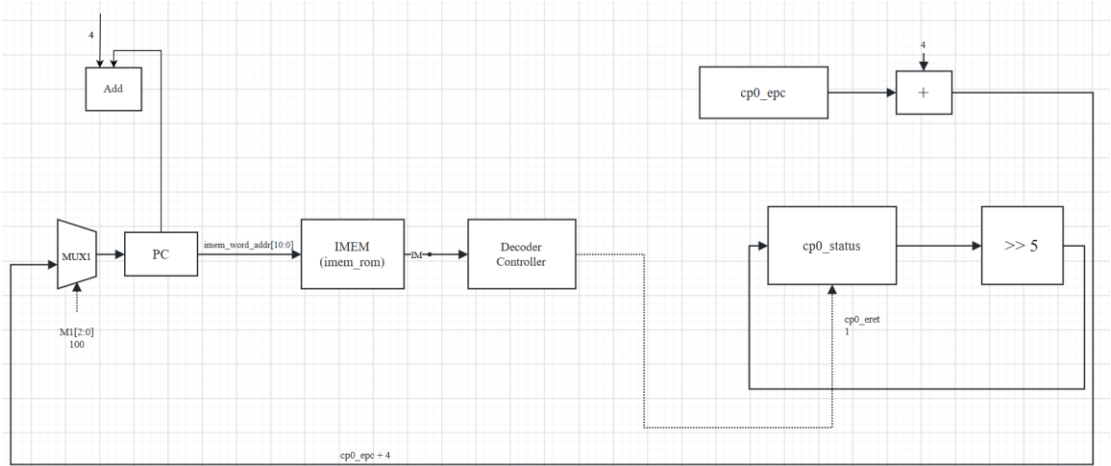


③ syscall

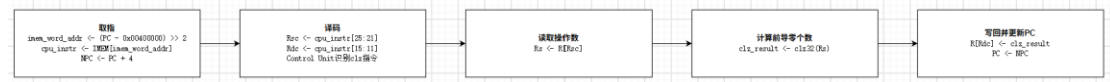
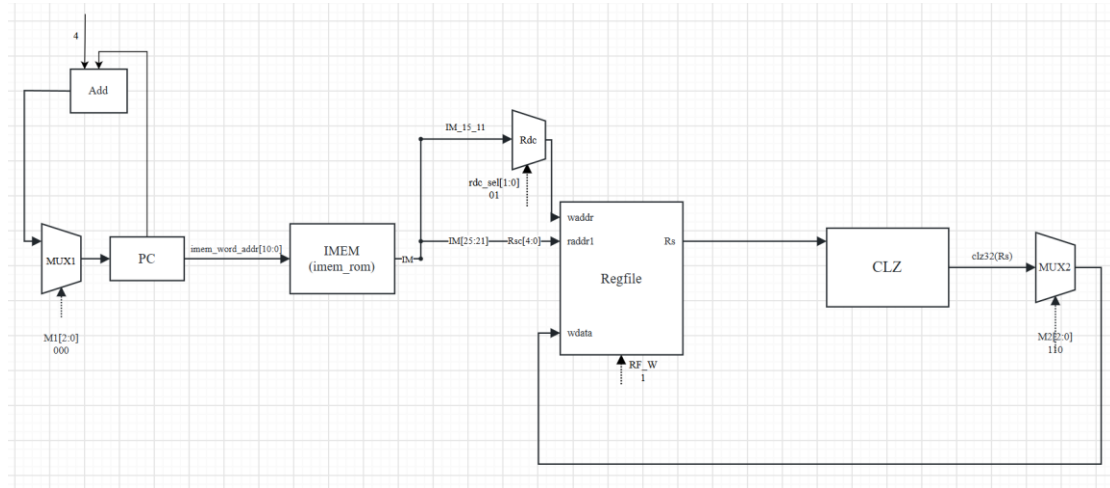


④ break

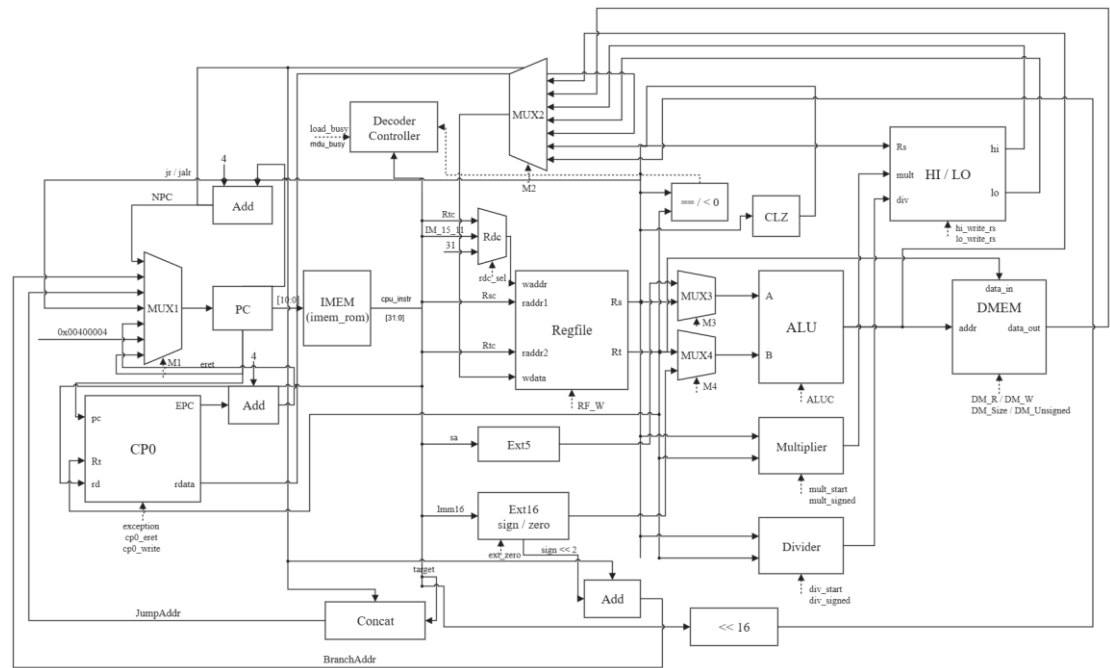




⑦ clz



54 条总数据通路图：



三、CPU 控制部件设计

(1) 根据指令流程图, 编排指令取指到执行的操作时间表。

[illegible]

上表每一行代表一条指令，第一张图各列表示 CPU 数据通路中的功能部件及其数据来源，第二张图各列则列出该指令对应的主要控制信号取值。左侧表格中，空白表示该指令不使用相应部件。比如 ADD 指令中，PC 取 NPC，NPC 和 IMEM 均以当前 PC 为输入，RegFile 写回数据来自 ALU，而 ALU 的 A、B 端分别取 rs 和 rt。表头中带 A、B 的部件表示其两个主要输入端，例如 ALU、CMP、ADD、JUMP、MULT 和 DIV；DMEMA、DMEMD 分别表示数据存储器地址和写入数据，Ext5、Ext16、Ext18 则表示不同的立即数扩展通路。这样沿某一指令行从左向右读取，可以很方便地看出该指令执行时经过的操作流程。

第二张几个关于 MUX 的图给出了 M1、M2、M3、M4、ext_zero、rdc_sel、ALUC、寄存器写使能、存储器读写控制以及乘除法、HI/LO、CP0 和异常处理等控制信号的取值。其中多位控制信号例如 M2=000 表示寄存器写回数据选择 ALU，ALUC=0001 表示 ALU 执行减法；条件分支和异常指令则在表中保留判断条件。因此这两张表结合可以作为写表达式的依据。对于只由指令译码决定的控制信号，只需逐列找出该控制位为 1 的所有指令，并将相应译码项逻辑或即可得到布尔表达式；对于 BEQ、BNE、BGEZ、TEQ 等条件指令，还需将译码项与比较条件相与后再进行逻辑或。load_busy 和 mdu_busy 等少数为了乘除法的多周期状态则作为额外的时序条件加入控制逻辑。

(2) 根据指令操作时间表，写出每个控制信号的逻辑表达式。

$rs_equal_rt = (Rs == Rt)$
 $rs_negative = Rs[31]$
 $M1[2] = (mdu_busy + load_busy) + \neg(mdu_busy + load_busy) \cdot (i_eret + i_syscall + i_break + i_teq \cdot rs_equal_rt)$
 $M1[1] = (mdu_busy + load_busy) + \neg(mdu_busy + load_busy) \cdot (i_j + i_jal + i_jr + i_jalr)$
 $M1[0] = \neg(mdu_busy + load_busy) \cdot (i_beq \cdot rs_equal_rt + i_bne \cdot \neg rs_equal_rt + i_bgez \cdot \neg rs_negative + i_jr + i_jalr + i_syscall + i_break + i_teq \cdot rs_equal_rt)$
 $M2[2] = i_mflo + i_mfc0 + i_clz + i_lui$
 $M2[1] = i_jal + i_jalr + i_mfhi + i_clz + i_lui$
 $M2[0] = i_lb + i_lbu + i_lh + i_lhu + i_lw + i_mfhi + i_mfc0 + i_lui$
 $M3 = \neg(i_sll + i_srl + i_sra)$
 $M4 = i_addi + i_addiu + i_slli + i_slliu + i_andi + i_ori + i_xori + i_lb + i_lbu + i_lh + i_lhu + i_lw + i_sb + i_sh + i_sw$
 $ext_zero = i_andi + i_ori + i_xori$
 $rdc_sel[1] = i_jal + i_jalr$
 $rdc_sel[0] = i_add + i_addu + i_sub + i_subu + i_and + i_or + i_xor + i_nor + i_slt + i_sltu + i_sll + i_srl + i_sra + i_sllv + i_srlv + i_sra + i_mfhi + i_mflo + i_clz + i_jalr$
 $ALUC[3] = i_sll + i_sllv + i_srl + i_srlv + i_sra + i_sra + i_sra$
 $ALUC[2] = i_xor + i_xori + i_nor + i_slt + i_slli + i_slliu + i_slliu$
 $ALUC[1] = i_and + i_andi + i_or + i_ori + i_slt + i_slli + i_slliu + i_slliu + i_sra + i_sra + i_sra$
 $ALUC[0] = i_sub + i_subu + i_or + i_ori + i_nor + i_sltu + i_slliu + i_srl + i_srlv$
 $RF_W_raw = i_add + i_addu + i_sub + i_subu + i_and + i_or + i_xor + i_nor + i_slt + i_sltu + i_sll + i_srl + i_sra + i_sllv + i_srlv + i_sra + i_addi + i_addiu + i_slli + i_slliu + i_andi + i_ori + i_xori + i_lui + i_jal + i_jalr + i_mfhi + i_mflo + i_mfc0 + i_clz + (i_lb + i_lbu + i_lh + i_lhu + i_lw) \cdot load_busy$
 $RF_W = [i_add + i_addu + i_sub + i_subu + i_and + i_or + i_xor + i_nor + i_slt + i_sltu + i_sll + i_srl + i_sra + i_sllv + i_srlv + i_sra + i_addi + i_addiu + i_slli + i_slliu + i_andi + i_ori + i_xori + i_lui + i_jal + i_jalr + i_mfhi + i_mflo + i_mfc0 + i_clz + (i_lb + i_lbu + i_lh + i_lhu + i_lw) \cdot load_busy] \cdot \neg mdu_busy$
 $DM_R_raw = i_lb + i_lbu + i_lh + i_lhu + i_lw$
 $DM_W_raw = i_sb + i_sh + i_sw$
 $DM_R = (i_lb + i_lbu + i_lh + i_lhu + i_lw) \cdot \neg mdu_busy$
 $DM_W = (i_sb + i_sh + i_sw) \cdot \neg mdu_busy$
 $DM_CS = (i_lb + i_lbu + i_lh + i_lhu + i_lw + i_sb + i_sh + i_sw) \cdot \neg mdu_busy$
 $DM_Size[1] = \neg(i_lb + i_lbu + i_sb + i_lh + i_lhu + i_sh)$
 $DM_Size[0] = i_lh + i_lhu + i_sh$
 $DM_Unsigned = i_lbu + i_lhu$
 $mult_start = i_mult + i_multu$
 $mult_signed = i_mult$
 $cpu_multiplier.start = (i_mult + i_multu) \cdot \neg mdu_busy$

```

div_start = i_div + i_divu
div_signed = i_div
cpu_divider.start = (i_div + i_divu) · ¬mdu_busy
hi_write_rs_raw = i_mthi
hi_write_rs = i_mthi · ¬mdu_busy
lo_write_rs_raw = i_mtlo
lo_write_rs = i_mtlo · ¬mdu_busy
cp0_write_raw = i_mtc0
cp0_write = i_mtc0 · ¬mdu_busy
cp0_eret_raw = i_eret
cp0_eret = i_eret · ¬mdu_busy
exception_raw = i_syscall + i_break + i_teq · rs_equal_rt
exception = (i_syscall + i_break + i_teq · rs_equal_rt) · ¬mdu_busy
exception_code[4] = 0
exception_code[3] = i_syscall + i_break + i_teq · rs_equal_rt
exception_code[2] = i_teq · rs_equal_rt
exception_code[1] = 0
exception_code[0] = i_break + i_teq · rs_equal_rt
mdu_busy = mult_unit_busy + div_unit_busy
mdu_done = mult_unit_done + div_unit_done
load_busy_next = ¬load_busy · (i_lb + i_lbu + i_lh + i_lhu + i_lw) · ¬mdu_busy
Regfile 写入条件 = [i_add + i_addu + i_sub + i_subu + i_and + i_or + i_xor + i_nor +
i_slt + i_sltu + i_sll + i_srl + i_sra + i_sllv + i_srlv + i_srav + i_addi + i_addiu + i_slti
+ i_sltiu + i_andi + i_ori + i_xori + i_lui + i_jal + i_jalr + i_mfhi + i_mflo + i_mfc0 +
i_clz + (i_lb + i_lbu + i_lh + i_lhu + i_lw) · load_busy] · ¬mdu_busy · (Rdc ≠ 0)
Rsc = IM[25:21]
Rtc = IM[20:16]
Ext5 = {27'b0, sa}
Ext16 = ext_zero · ZeroExt(Imm16) + ¬ext_zero · SignExt(Imm16)
Ext18 = SignExt({Imm16, 2'b00})
BranchAddr = NPC + Ext18
JumpAddr = {NPC[31:28], target, 2'b00}
ALU_A = M3 · Rs + ¬M3 · Ext5
ALU_B = M4 · Ext16 + ¬M4 · Rt
Rdc = ¬rdc_sel[1] · ¬rdc_sel[0] · Rtc + ¬rdc_sel[1] · rdc_sel[0] · IM_15_11 +
rdc_sel[1] · ¬rdc_sel[0] · 5'd31 + rdc_sel[1] · rdc_sel[0] · [(IM_15_11 == 0) ? 5'd31 :
IM_15_11]
Regfile 写回数据 Rd = ¬M2[2] · ¬M2[1] · ¬M2[0] · ALU_Result +
¬M2[2] · ¬M2[1] · M2[0] · DM_Data_out + ¬M2[2] · M2[1] · ¬M2[0] · NPC +
¬M2[2] · M2[1] · M2[0] · HI + M2[2] · ¬M2[1] · ¬M2[0] · LO +
M2[2] · ¬M2[1] · M2[0] · cp0_rdata + M2[2] · M2[1] · ¬M2[0] · clz_result +
M2[2] · M2[1] · M2[0] · {Imm16, 16'h0000}
PC_next = ¬M1[2] · ¬M1[1] · ¬M1[0] · NPC +
¬M1[2] · ¬M1[1] · M1[0] · BranchAddr + ¬M1[2] · M1[1] · ¬M1[0] · JumpAddr +

```

$\neg M1[2] \cdot M1[1] \cdot M1[0] \cdot Rs + M1[2] \cdot \neg M1[1] \cdot \neg M1[0] \cdot (cp0_epc + 4) +$
 $M1[2] \cdot \neg M1[1] \cdot M1[0] \cdot 32'h00400004 + M1[2] \cdot M1[1] \cdot \neg M1[0] \cdot PC$
 PC 在 clk 下降沿更新。

Regfile 在 clk 上升沿且 RF_W=1、Rdc≠0 时写入。

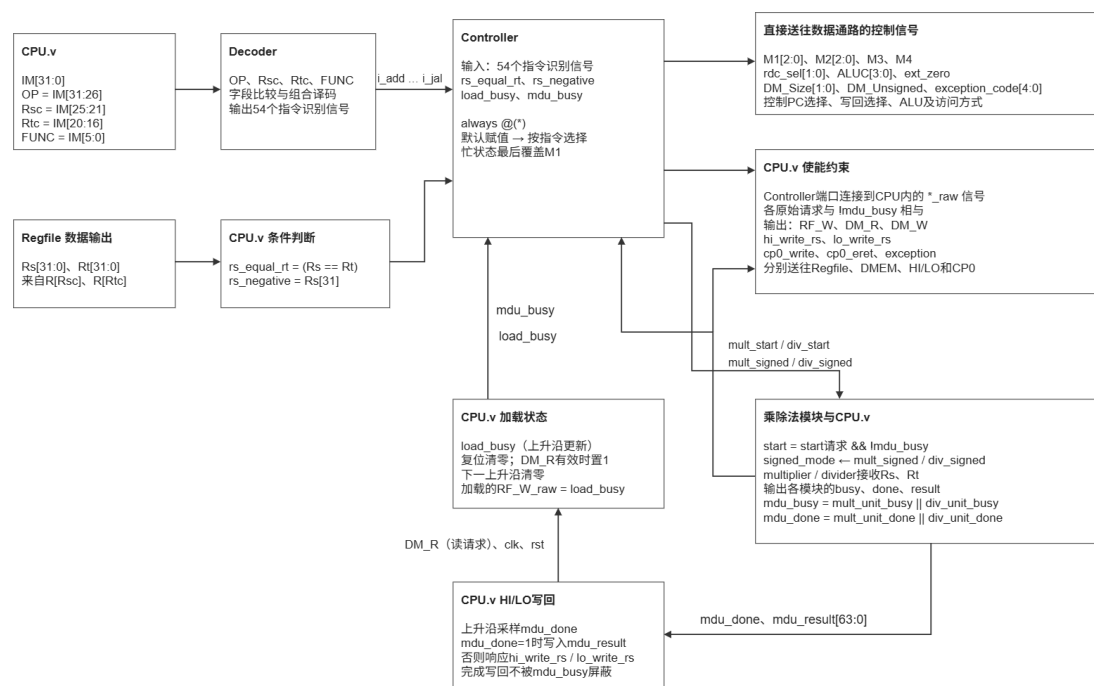
DMEM 在 clk_in 上升沿根据 DM_R、DM_W 完成同步读取或写入。

HI、LO 在 clk 上升沿由 MTHI、MTLO 直接写入或者在 mdu_done 有效时写入乘除法结果。

CP0 中的 Status、Cause 和 EPC 在 clk 上升沿根据 MTC0、异常或 ERET 更新。

(3) 根据逻辑表达式，完成控制部件设计。

控制器的整体逻辑图如下：



下面结合代码对控制器进行分析：

① Decoder 译码实现

```
assign i_add = (OP == 6'b000000 && FUNC == 6'b000000);
assign i_addu = (OP == 6'b000000 && FUNC == 6'b100001);
assign i_sub = (OP == 6'b000000 && FUNC == 6'b100010);
assign i_subu = (OP == 6'b000000 && FUNC == 6'b100011);
assign i_and = (OP == 6'b000000 && FUNC == 6'b101000);
assign i_or = (OP == 6'b000000 && FUNC == 6'b101001);
assign i_xor = (OP == 6'b000000 && FUNC == 6'b101010);
assign i_nor = (OP == 6'b000000 && FUNC == 6'b101011);
assign i_slt = (OP == 6'b000000 && FUNC == 6'b101100);
assign i_sltu = (OP == 6'b000000 && FUNC == 6'b101101);
assign i_sll = (OP == 6'b000000 && FUNC == 6'b000000);
assign i_srl = (OP == 6'b000000 && FUNC == 6'b000010);
assign i_sra = (OP == 6'b000000 && FUNC == 6'b000011);
assign i_sllv = (OP == 6'b000000 && FUNC == 6'b000100);
assign i_srlv = (OP == 6'b000000 && FUNC == 6'b000110);
assign i_srav = (OP == 6'b000000 && FUNC == 6'b000111);
assign i_jr = (OP == 6'b000000 && FUNC == 6'b001000);
assign i_jalr = (OP == 6'b000000 && FUNC == 6'b001001);
assign i_syscall = (OP == 6'b000000 && FUNC == 6'b001100);
assign i_break = (OP == 6'b000000 && FUNC == 6'b001101);
assign i_mfhi = (OP == 6'b000000 && FUNC == 6'b010000);
assign i_mfhi = (OP == 6'b000000 && FUNC == 6'b010001);
assign i_mflo = (OP == 6'b000000 && FUNC == 6'b010010);
assign i_mflo = (OP == 6'b000000 && FUNC == 6'b010011);
assign i_mtlo = (OP == 6'b000000 && FUNC == 6'b010011);
assign i_mult = (OP == 6'b000000 && FUNC == 6'b011000);
assign i_multu = (OP == 6'b000000 && FUNC == 6'b011001);
assign i_div = (OP == 6'b000000 && FUNC == 6'b011010);
assign i_divu = (OP == 6'b000000 && FUNC == 6'b011011);
assign i_teq = (OP == 6'b000000 && FUNC == 6'b110100);
assign i_mfc0 = (OP == 6'b010000 && Rsc == 5'b000000);
assign i_mtc0 = (OP == 6'b010000 && Rsc == 5'b001000);
assign i_eret = (OP == 6'b010000 && Rsc == 5'b100000 && FUNC == 6'b011000);
assign i_clz = (OP == 6'b011100 && FUNC == 6'b100000);
assign i_bgez = (OP == 6'b000001 && Rtc == 5'b000001);
```

```
assign i_beq = (OP == 6'b000100);
assign i_bne = (OP == 6'b000101);
assign i_addi = (OP == 6'b001000);
assign i_addiu = (OP == 6'b001001);
assign i_slti = (OP == 6'b001010);
assign i_sltiu = (OP == 6'b001011);
assign i_andi = (OP == 6'b001100);
assign i_ori = (OP == 6'b001101);
assign i_xori = (OP == 6'b001110);
assign i_lui = (OP == 6'b001111);
assign i_lb = (OP == 6'b100000);
assign i_lh = (OP == 6'b100001);
assign i_lw = (OP == 6'b100010);
assign i_lbu = (OP == 6'b100011);
assign i_lhu = (OP == 6'b100100);
assign i_sb = (OP == 6'b101000);
assign i_sh = (OP == 6'b101001);
assign i_sw = (OP == 6'b101010);
assign i_j = (OP == 6'b000010);
assign i_jal = (OP == 6'b000011);
```

Decoder 负责将指令编码转换为 54 个相互独立的指令识别信号,为 Controller 实现上一问得到的逻辑表达式提供基本译码项。译码过程采用赋值实现,不包含时序逻辑。R 型指令主要同时比较 OP 和 FUNC 字段,例如 ADD 在 OP=000000 且 FUNC=100000 时产生 i_add=1;立即数、访存和普通跳转指令大多只需判断 OP,例如 LW 在 OP=100011 时产生 i_lw=1。

对于编码形式较特殊的扩展指令,还需要进一步判断其他字段。例如 BGEZ 除判断 OP=000001 外,还要求 Rtc=00001;MFC0 和 MTC0 通过 Rsc 区分读写 CP0 操作;ERET 同时检查 OP、Rsc 和 FUNC。这样 Decoder 最终把原始 32 位指令转换为 i_add、i_lw、i_bgez、i_mfc0 等一位有效信号,使 Controller 可以直接按照前面得到的控制逻辑表达式进行组合。

②Controller 组合逻辑实现

Controller 使用 always @(*)实现组合控制逻辑。进入组合块后,首先为各控制信号设置默认值,例如 M1=000 表示 PC 默认选择 NPC, M2=000 表示寄存器写回默认选择 ALU 结果, M3=1、M4=0 分别使 ALU 的两个输入默认选择 Rs 和 Rt,同时将各写使能和功能请求信号默认清零。随后根据 Decoder 产生的 54 个指令识别信号修改对应控制量。

```

128 | if (i_add || i_addu) begin
129 |     RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0000;
130 | end else if (i_sub || i_subu) begin...
132 | end else if (i_and) begin...
134 | end else if (i_or) begin...
136 | end else if (i_xor) begin...
138 | end else if (i_nor) begin...
140 | end else if (i_slt) begin...
142 | end else if (i_sltu) begin...
144 | end else if (i_sll || i_srl || i_sra) begin...
149 | end else if (i_sllv || i_srlv || i_srav) begin...
154 | end else if (i_addi || i_addiu) begin...
156 | end else if (i_slti) begin...
158 | end else if (i_sltiu) begin...
160 | end else if (i_andi || i_ori || i_xori) begin...
165 | end else if (i_lui) begin...
167 | end else if (i_lb || i_lbu || i_lh || i_lhu || i_lw) begin...
169 | end else if (i_sb || i_sh || i_sw) begin...
171 | end else if (i_jal) begin...
173 | end else if (i_jalr) begin...
175 | end else if (i_mfhi) begin...
177 | end else if (i_mflo) begin...
179 | end else if (i_mfc0) begin...
181 | end else if (i_clz) begin...
183 | end

```

```

185 | DM_R = i_lb || i_lbu || i_lh || i_lhu || i_lw;
186 | DM_W = i_sb || i_sh || i_sw;
187 | if (i_lb || i_lbu || i_sb) DM_Size = 2'b00;
188 | else if (i_lh || i_lhu || i_sh) DM_Size = 2'b01;
189 | else DM_Size = 2'b10;
190 | DM_Unsigned = i_lbu || i_lhu;
191 |
192 | mult_start = i_mult || i_multu;
193 | mult_signed = i_mult;
194 | div_start = i_div || i_divu;
195 | div_signed = i_div;
196 | hi_write_rs = i_mthi;
197 | lo_write_rs = i_mtlo;
198 | cp0_write = i_mtc0;
199 | cp0_eret = i_eret;
200 |
201 | exception = i_break || i_syscall || (i_teq && rs_equal_rt);
202 | if (i_syscall) exception_code = 5'b01000;
203 | else if (i_break) exception_code = 5'b01001;
204 | else if (i_teq && rs_equal_rt) exception_code = 5'b01101;
205 |
206 | if ((i_beq && rs_equal_rt) || (i_bne && !rs_equal_rt) ||
207 |     (i_bgez && !rs_negative)) M1 = 3'b001;
208 | else if (i_j || i_jal) M1 = 3'b010;
209 | else if (i_jr || i_jalr) M1 = 3'b011;
210 | else if (i_eret) M1 = 3'b100;
211 | else if (exception) M1 = 3'b101;
212 | if (mdu_busy || load_busy) M1 = 3'b110;

```

左图展现了一般情况,例如 ADD 和 ADDU 执行时置 RF_W=1、rdc_sel=01、ALUC=0000,表示 ALU 执行加法并将结果写回 rd; SUB 和 SUBU 只需将 ALUC 改为 0001,只需要按照前述表格进行对应控制信号的赋值即可。

右图展现了不适合逐条写在主要的 if-else 中的控制信号是怎么按照逻辑表达式进行组合的。DM_R 由所有 Load 指令相或得到,DM_W 由 SB、SH、SW 相或得到;mult_start、div_start 分别由两类乘除法指令产生,mult_signed 和 div_signed 用于区分有符号和无符号运算;cp0_write 和 cp0_eret 则分别由 MTC0 和 ERET 直接产生。这部分与上一问写出的布尔逻辑表达式基本是一一对应的。分支和异常控制还需要加入运行时条件。BREAK、SYSCALL 以及满足 Rs==Rt 的 TEQ 产生 exception,并分别设置异常码 9、8 和 13;BEQ、BNE 和 BGEZ 则分别结合 rs_equal_rt 和 rs_negative 决定 M1 是否选择分支地址。J/JAL、JR/JALR、ERET 和异常分别对应 M1 的其他编码。最后,当 mdu_busy 或 load_busy 有效时,源码再次将 M1 强制赋值为 110,使 PC 保持当前值。

③CPU 中的状态反馈与使能约束

```
wire rs_equal_rt = (Rs == Rt);  
wire rs_negative = Rs[31];  
assign RF_W = RF_W_raw && !mdu_busy;  
assign DM_R = DM_R_raw && !mdu_busy;  
assign DM_W = DM_W_raw && !mdu_busy;  
assign hi_write_rs = hi_write_rs_raw && !mdu_busy;  
assign lo_write_rs = lo_write_rs_raw && !mdu_busy;  
assign cp0_write = cp0_write_raw && !mdu_busy;  
assign cp0_eret = cp0_eret_raw && !mdu_busy;  
assign exception = exception_raw && !mdu_busy;
```

Controller 产生的部分控制信号首先以*_raw 形式送回 CPU,再结合当前运行状态形成真正送往各功能部件的控制信号。CPU 根据寄存器输出产生 rs_equal_rt=(Rs==Rt)和 rs_negative=Rs[31], 供 BEQ、BNE、BGEZ 和 TEQ 等条件指令使用。对于需要改变 CPU 状态的控制信号,则统一与!mdu_busy 相&, 例如寄存器写使能由 RF_W_raw 得到最终的 RF_W, DMEM 读写信号由 DM_R_raw、DM_W_raw 得到最终的 DM_R、DM_W, HI/LO、CP0 和异常控制也采用相同的方式。这样在乘除法模块处于忙状态时,可以阻止当前指令重复修改寄存器、存储器或其他状态。

```
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        hi <= 32'h0;  
        lo <= 32'h0;  
        load_busy <= 1'b0;  
        cp0_status <= 32'h0;  
        cp0_cause <= 32'h0;  
        cp0_epc <= 32'h0;  
    end else begin  
        if (load_busy)  
            load_busy <= 1'b0;  
        else if (DM_R)  
            load_busy <= 1'b1;  
  
        if (mdu_done) begin  
            hi <= mdu_result[63:32];  
            lo <= mdu_result[31:0];  
        end else if (hi_write_rs) begin  
            hi <= Rs;  
        end else if (lo_write_rs) begin  
            lo <= Rs;  
        end  
    end  
end
```

load_busy 是 CPU 内部用于配合同步数据存储器的状态寄存器。复位时其值为 0; 当 Load 指令产生有效的 DM_R 后,在时钟上升沿将 load_busy 置 1,使 Controller 在该指令尚未完成时保持当前 PC; 下一上升沿又将其清 0,同时 Load 指令利用此时的 load_busy 使能寄存器写回,从而将已经读出的 DMEM 数据写入目的寄存器。

HI 和 LO 的写入也在同一个上升沿时序块中完成。当乘法器或除法器产生 mdu_done 时,将 64 位 mdu_result 的高 32 位写入 HI、低 32 位写入 LO; 若当前没有乘除法结果需要写回,则 MTHI 和 MTLO 可以分别通过 hi_write_rs 和 lo_write_rs 将 Rs 直接写入 HI 或 LO。这样组合控制器负责产生控制请求,而需要保持状态的 Load 等待和乘除法结果写回由 CPU 中的时序逻辑完成。

④ CP0 及异常状态控制

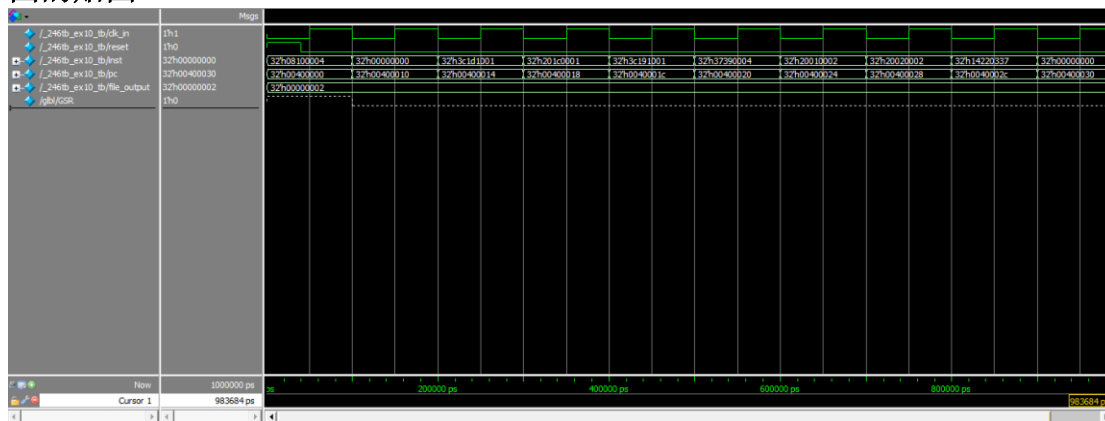
```
if (cp0_write && IM_15_11 == 5'd12) cp0_status <= Rt;
if (cp0_write && IM_15_11 == 5'd13) cp0_cause <= Rt;
if (cp0_write && IM_15_11 == 5'd14) cp0_epc <= Rt;
if (exception) begin
    cp0_status <= cp0_status << 5;
    cp0_cause <= {25'h0, exception_code, 2'b00};
    cp0_epc <= pc;
end else if (cp0_eret) begin
    cp0_status <= cp0_status >> 5;
```

CPU 中实现了 Status、Cause 和 EPC 三个与异常处理有关的 CP0 寄存器。执行 MTC0 时，cp0_write 有效，并根据指令中的 IM_15_11 字段选择写入 12 号 Status、13 号 Cause 或 14 号 EPC 寄存器，写入数据来自 Rt。发生 BREAK、SYSCALL 或满足条件的 TEQ 异常时，exception 有效，CPU 将当前 PC 保存到 cp0_epc，将 exception_code 写入 Cause 寄存器对应字段，同时将 Status 左移 5 位保存当前状态。Controller 在同一条异常指令中令 M1 选择异常入口地址，使程序转入异常处理程序。

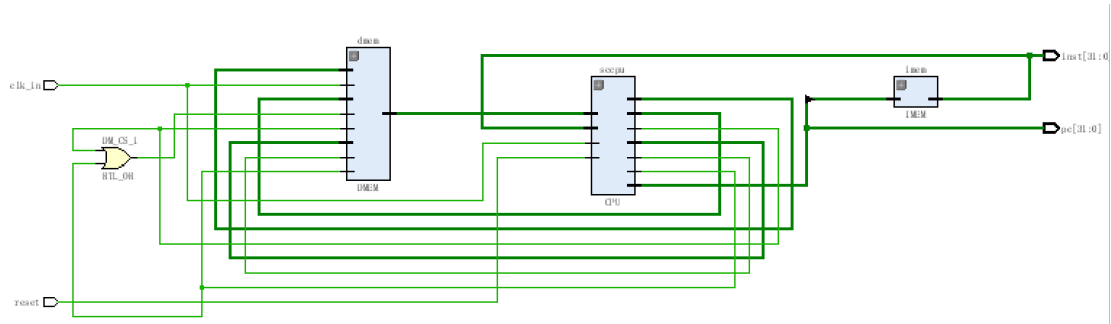
执行 ERET 时，cp0_eret 有效，CPU 将 Status 右移 5 位恢复异常发生前的状态；与此同时 Controller 令 M1 选择 cp0_epc+4 作为下一条指令地址，从而返回到异常指令之后继续执行。这样，异常的识别和 PC 选择由组合控制逻辑完成，而 EPC、Cause 和 Status 等状态的保存与恢复由 CPU 中的时序逻辑完成。

四、CPU 前仿真测试结果

(1) 对 CPU 前仿真的实验内容和结果进行描述，并给出 modelsim 前仿真波形图的贴图。



由前仿真波形可见，复位结束后 CPU 从 PC=00400000 开始执行，此时取出的指令为 08100004；执行该跳转指令后，PC 变为 00400010。随后 PC 依次变化为 00400014、00400018、0040001c 等地址，对应的 inst 也随着 PC 同步改变，说明 CPU 能够连续完成取指和指令执行。执行到 PC=0040002c 的 14220337 指令后，下一 PC 为 00400030，分支控制也能够按照当前条件正确选择下一地址。总体上，前仿真中 PC 和指令序列随时钟稳定变化，CPU 能够正常运行测试程序。



上图为前仿真的 RTL 图，可观察顶层 `sccomp_dataflow`。由图可见，整个 CPU 系统主要由 CPU、IMEM 和 DMEM 三个模块组成。CPU 输出当前程序地址 `pc[31:0]`，经过指令存储器地址转换后送入 IMEM，IMEM 输出 32 位指令并返回 CPU 完成取指；同时 CPU 通过 `DM_Addr`、`DM_Data_in`、`DM_R`、`DM_W`、`DM_Size` 和 `DM_Unsigned` 等信号与 DMEM 连接，实现字、半字和字节数据的读写。DMEM 读取的数据再通过 `DM_Data_out` 返回 CPU。

图中 `clk_in` 同时连接 CPU 和数据存储器，为各时序部件提供统一时钟，`reset` 用于系统复位；右侧的 `pc[31:0]` 和 `inst[31:0]` 作为顶层观察信号输出，正好与前面的 ModelSim 波形中观察的 PC 和当前指令相对应。由 RTL 结构图可以确认，CPU、指令存储器和数据存储器之间的主要端口及连接关系与设计的数据通路一致。

综合 ModelSim 波形与 RTL 结构图可以看出，前仿真阶段既验证了 CPU 在时钟驱动下的实际执行结果，也验证了综合前各 RTL 模块之间的数据和控制连接关系。

(2) 对单条指令、网站前仿真所用测试代码 `coe` 文件等测试的结果进行比对。

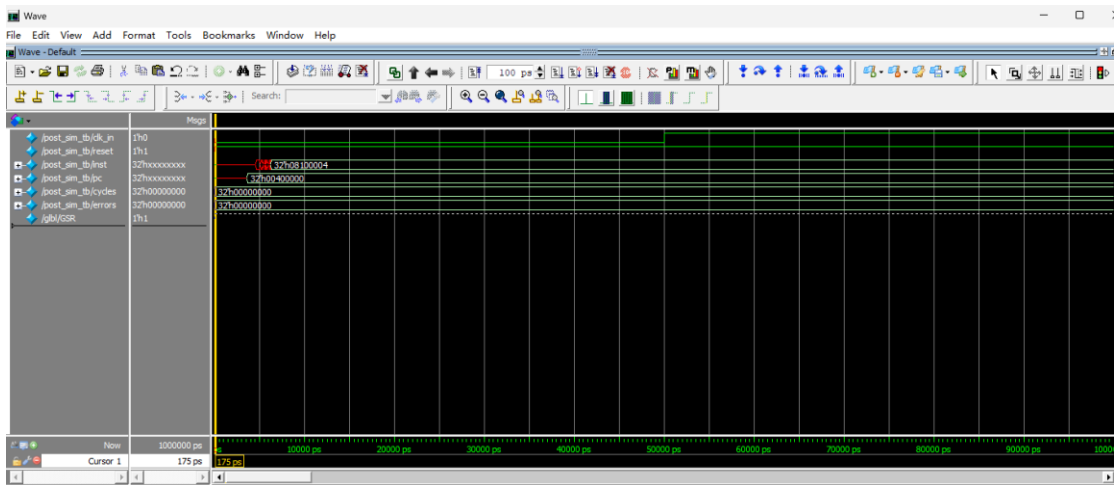
前仿真采用课程提供的 `coe` 作为测试程序，使用 `testbench_cpu54_single.v` 进行测试，并以课程给出的 `_246tb_ex10_result.txt` 作为标准结果文件。仿真过程中按执行状态输出当前 PC、当前指令以及 32 个通用寄存器的值，再将本设计产生的 `output.txt` 与标准结果逐行进行比较。标准结果文件共有 35836 行有效数据。每个 CPU 状态恰好占 34 行，其中包括 1 行 `pc`、1 行 `instr` 以及 `regfile0~regfile31` 共 32 行寄存器状态，因此 $34 \times 1054 = 35836$ ，即标准结果一共记录了 1054 个 CPU 执行状态。文件最后一行正好是最后一个状态中的 `regfile31`，说明 35836 已经覆盖标准结果文件的全部有效内容。因此比较命令中的 `--max_line 35836` 表示从第 1 行一直检查到标准结果末尾，对完整的 1054 组状态进行比对。

```
C:\Users\t1502\Desktop\前仿真对比>txt_compare --file1 _246tb_ex10_result.txt --file2 output.txt --max_line 35836 --display detailed
比较结果输出:
=====
在指定检查条件下完全一致.
=====
```

如上图使用 `txt_compare` 进行比较后，程序输出在指定检查条件下完全一致。标准结果最初记录 `PC=00400000`、`instr=08100004`，随后程序跳转到 `00400010` 继续执行，与测试程序的控制流一致。最终完整 35836 行均未发现差异，说明本设计在这 1054 个检查点上的 PC 值、当前指令以及 32 个通用寄存器状态均与标准结果一致，从而通过了完整测试程序的前仿真结果比对，网站结果为 AC。

五、CPU 后仿真测试结果

(1) 对 CPU 后仿真的实验内容和结果进行描述，并给出 modelsim 后仿真波形图的贴图。

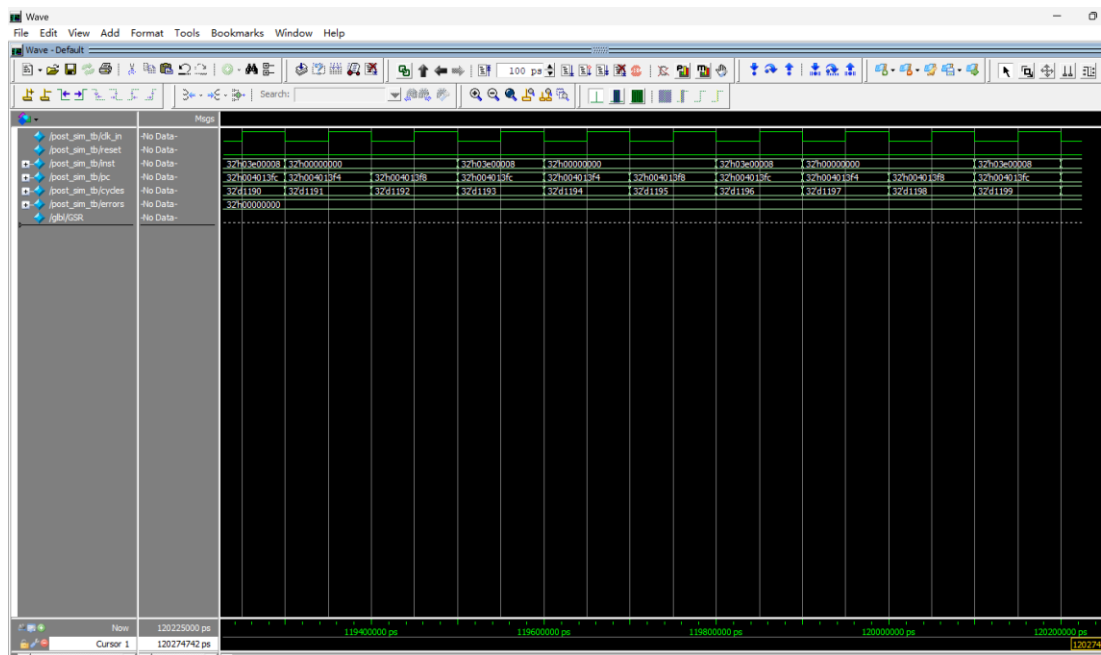


后仿真在完成 CPU 综合后进行，指令存储器采用 IP 核实现。在 Vivado 中选择 Run Post-Synthesis Timing Simulation 并调用 ModelSim，对综合后的 CPU 网表进行时序仿真。测试平台自行产生 clk_in 和 reset 信号。由于后仿真中需要考虑综合后逻辑的门延迟和连线延迟，因此时钟周期不能设置过小，本次测试采用 100 ns 的时钟周期，并在检查点之后留出一定时间等待信号稳定。

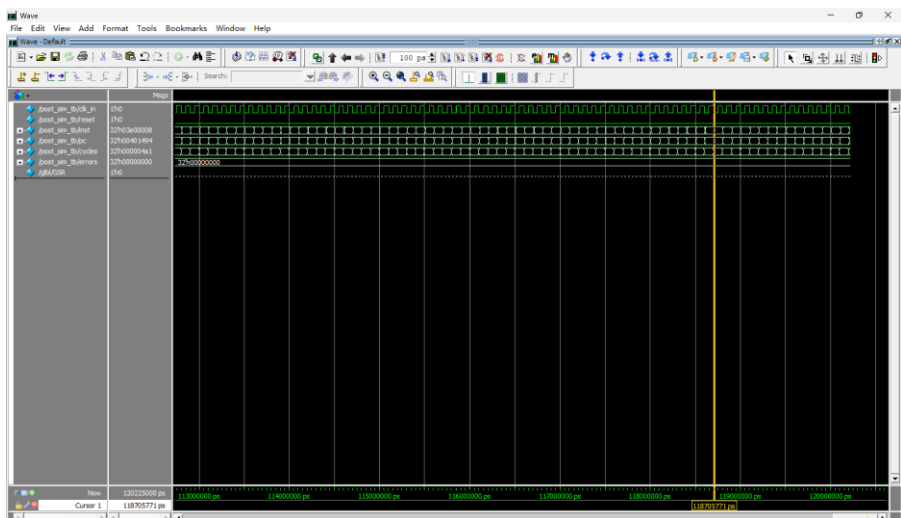
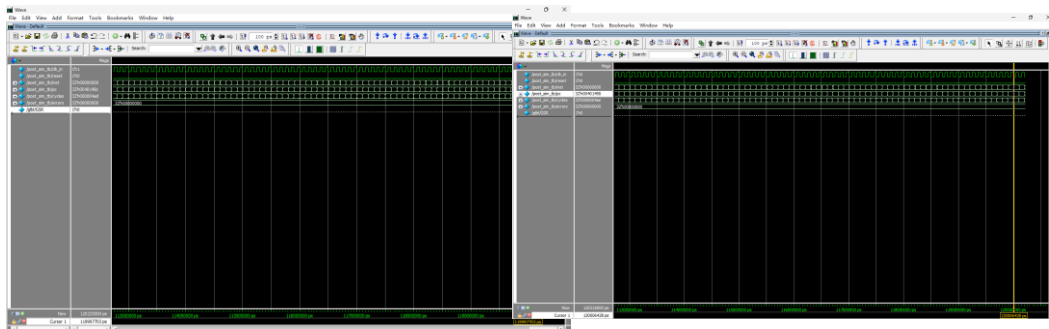
上图为前 0-100ns 时的波形图，可以看到后仿真相比前仿真（也即 Run Behavioral Simulation）增加了时序延迟，下面是波形区别：

观察项	前仿真	后仿真
pc 更新	基本与时钟下降沿对齐	在下降沿之后稍晚更新
inst 更新	看起来与 pc 同时变化	随 pc 更新后，再经过取指电路延迟才稳定
总线边界	比较整齐	部分边界出现短暂交错
指令执行顺序	00400000 → 00400010 → 00400014...	相同

原因是两种仿真模拟的对象不同。前仿真运行 RTL，延时的逻辑通常在同一仿真时刻的若干调度步骤内完成，因此画面上像是立即变化；后仿真运行综合后的门级网表，并加载 SDF 延时。时钟经过输入缓冲和时钟网络，寄存器输出也需要时间；新 PC 再经过地址计算、ROM 取指，才能得到稳定的 inst。因此实际变化过程是在时钟下降沿时钟传播、PC 寄存器输出更新，指令地址变化，ROM 输出新的 inst 并稳定。



与前面那张截取后仿真开始时 0-100ns 的波形不同，图中截取了仿真接近结束时的一段波形，可以看到 cycles 由 1190 连续增加至 1199，pc 和 inst 仍随时钟正常变化，复位结束后 CPU 能够正常取指并更新 PC。在 testbench 中，每个检查周期均在时钟边沿之后留出 20 ns 用于等待综合后网表信号稳定，因此图中显示的是传播延迟结束后的稳定结果。



同线下验收要求，上面三张图展示了地址最终按照 2026 新版 COE 的 pc 值 0040148c-00401490-00401494 三者不断循环的过程，在 tcl 输入 **run 1ms** 即可。

```

VSIM 2> run -all
# POST_SIM_TRACE cycle=10 pc=00400034 inst=20010003
# POST_SIM_TRACE cycle=100 pc=0040019c inst=143f02db
# POST_SIM_TRACE cycle=300 pc=00400434 inst=20040000
# POST_SIM_TRACE cycle=1000 pc=00401414 inst=335b00ff
# POST_SIM_TRACE cycle=1100 pc=004013fc inst=03e00008
# POST_SIM_TRACE cycle=1200 pc=004013f4 inst=00000000
# POST_SIM_PASS cycles=1200 pc=004013f4 inst=00000000
# ** Note: $finish : ../../MIPS_54_CPU.srsrcs/sim_1/new/post_sim_tb.v(52)
# Time: 120225 ns Iteration: 0 Instance: /post_sim_tb
# 1
# Break in Module post_sim_tb at ../../MIPS_54_CPU.srsrcs/sim_1/new/post_sim_tb.v line 52

```

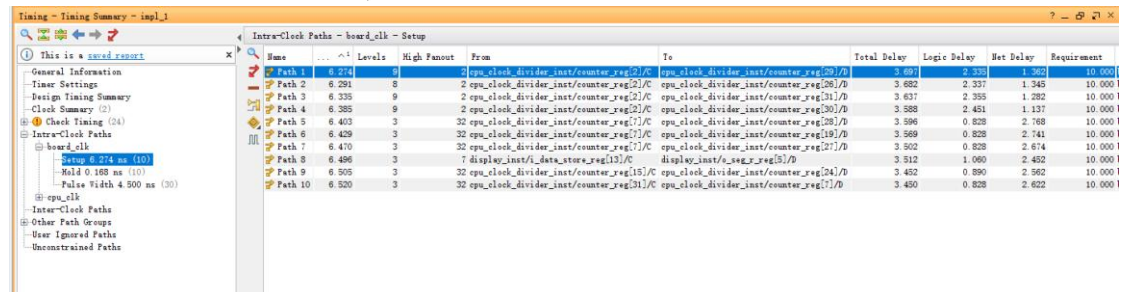
为方便长时间运行时发现明显异常，testbench 中另外设置了 cycles 和 errors 两个辅助观察量，并在第 10、100、300、1000、1100 和 1200 个周期打印当前 PC 和指令。执行 run -all 后，仿真运行至第 1200 个检查周期并输出 POST_SIM_PASS。这里的 POST_SIM_PASS 是设计 testbench 增加的辅助检查结果，用于确认仿真过程中没有出现未知值以及预设的首跳错误；后仿真的主要判断依据仍然是 ModelSim 中的综合后时序波形。

(2) 对时序分析的结果进行分析，并截图。



在完成综合与实现后，使用 Vivado 的 Report Timing Summary 对设计进行静态时序分析，分别检查建立时间、保持时间和时钟脉宽约束。从分析结果可见，建立时间的最坏裕量 WNS 为 6.274 ns，总负裕量 TNS 为 0.000 ns，建立时间违例端点数为 0；保持时间的最坏裕量 WHS 为 0.053 ns，总保持时间负裕量 THS 为 0.000 ns，保持时间违例端点数同样为 0。此外，最坏脉宽裕量 WPWS 为 4.500 ns，总脉宽负裕量 TPWS 为 0.000 ns，也不存在脉宽违例。

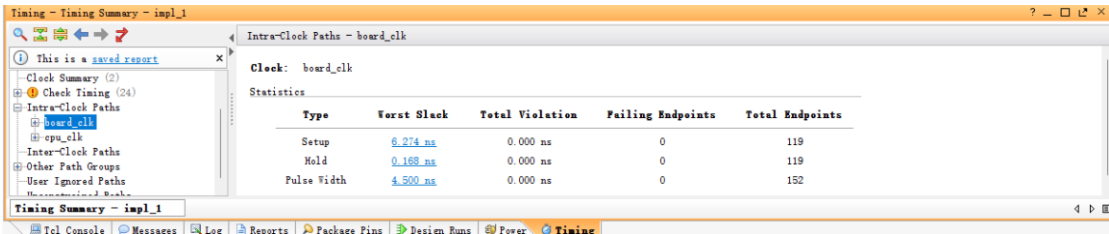
其中 WNS 和 WHS 均为正值，说明即使在最不利的建立时间和保持时间路径上，数据仍能在时钟要求的时间范围内正确到达并保持稳定；TNS 和 THS 均为 0，表明不存在任何累积的建立时间或保持时间违例。Vivado 最终给出“All user specified timing constraints are met”，说明当前设计在所设置的时钟约束下满足时序要求，可以进行后续下板验证。



为分析总体 WNS 的来源，对 board_clk 时钟域的 Setup 路径进行展开。由图可见，该时钟域最坏建立时间裕量为 6.274 ns，对应 Path 1。该路径位于 cpu_clock_divider_inst 内部，从分频计数器的一个寄存器节点经过组合逻辑到达另一计数器寄存器，共经过 9 级逻辑，总路径延迟为 3.697 ns，其中逻辑延迟为 2.335 ns、网络延迟为 1.362 ns。在 10.000 ns 的时钟周期约束下，该路径仍具有

6.274 ns 的正裕量，因此不存在建立时间违例。

同时可以看出，当前 board_clk 时钟域中的最坏路径主要来自时钟分频器的 32 位计数逻辑，而不是 54 条指令 CPU 核心本身。原因是 CPU 核心使用经过大倍率分频后的 cpu_clk 运行，而分频器、显示及部分同步逻辑仍工作在较快的 board_clk 时钟域。因此总体 Timing Summary 中的 WNS=6.274 ns 主要反映的是板级外围逻辑在 board_clk 下的最坏建立时间情况。



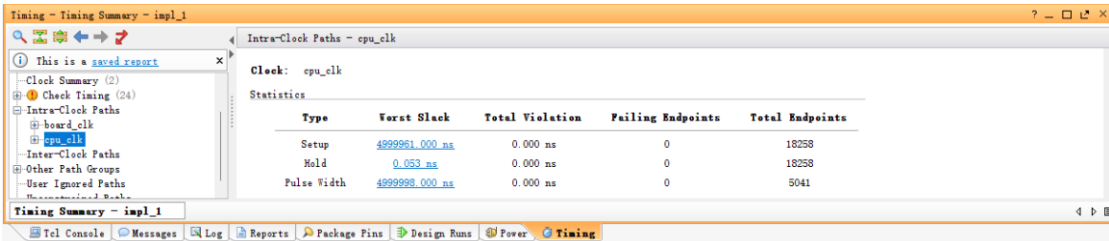
Timing Summary - impl_1

Intra-Clock Paths - board_clk

Clock: board_clk

Type	Worst Slack	Total Violation	Failing Endpoints	Total Endpoints
Setup	6.274 ns	0.000 ns	0	119
Hold	0.168 ns	0.000 ns	0	119
Pulse Width	4.500 ns	0.000 ns	0	152

进一步对不同的时钟域进行分析。board_clk 对应开发板输入时钟及显示、分频等外围逻辑所在的高速时钟域。该时钟域共有 119 个建立/保持时间分析端点，其最坏建立时间裕量为 6.274 ns，最坏保持时间裕量为 0.168 ns，最坏脉宽裕量为 4.500 ns，三类检查的违例总量及失败端点数均为 0。因此，板载时钟域内的寄存器间数据传输能够满足当前时钟约束要求。



Timing Summary - impl_1

Intra-Clock Paths - cpu_clk

Clock: cpu_clk

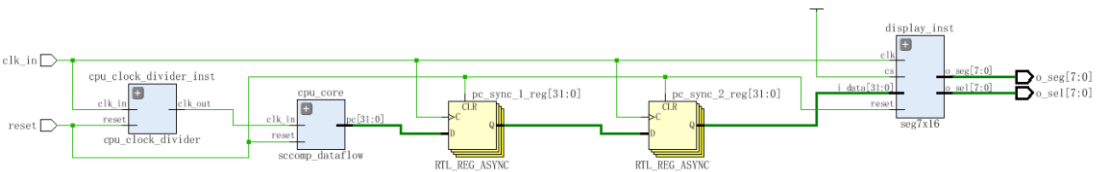
Type	Worst Slack	Total Violation	Failing Endpoints	Total Endpoints
Setup	4999961.000 ns	0.000 ns	0	18258
Hold	0.053 ns	0.000 ns	0	18258
Pulse Width	4999999.000 ns	0.000 ns	0	5041

cpu_clk 为 CPU 核心实际使用的分频时钟。设计中的时钟分频器设置 HALF_PERIOD_CYCLES=500000，使 CPU 工作时钟远低于板载输入时钟，以便人眼下板时能够稳定观察 PC 的运行过程。因此该时钟域的建立时间裕量达到 4999961.000 ns，而最坏保持时间裕量仍为 0.053 ns。虽然 Setup 裕量很大，但其根本原因是 CPU 时钟周期被人为大幅拉长，并不代表组合逻辑本身具有如此高的运行性能。该时钟域的 Total Violation 和 Failing Endpoints 同样均为 0，说明 CPU 核心在实际使用的分频时钟下能够满足建立时间和保持时间要求。

Check Timing Report 中还报告了少量不完整约束的节点，主要集中在 display_inst 的七段数码管扫描寄存器，包括 3 个 no_clock 节点和 6 个未约束最大延迟端点。这些节点属于板级显示扫描逻辑，并不位于 CPU 核心的数据执行通路中。

六、CPU 下板结果

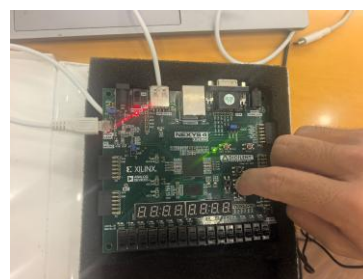
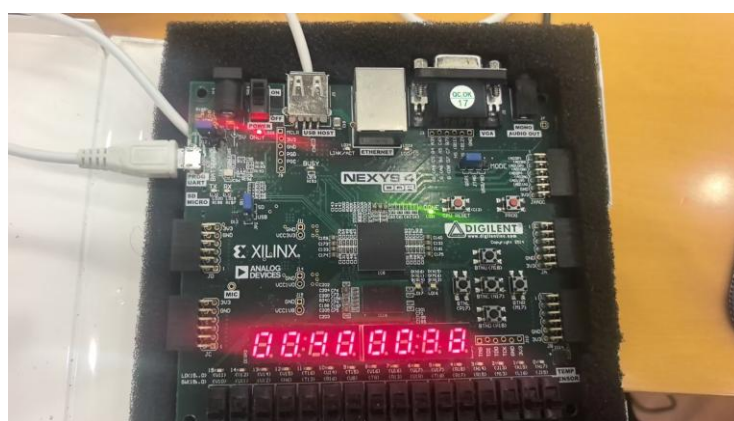
在下板前，先给出顶层设计的 RTL 结构：



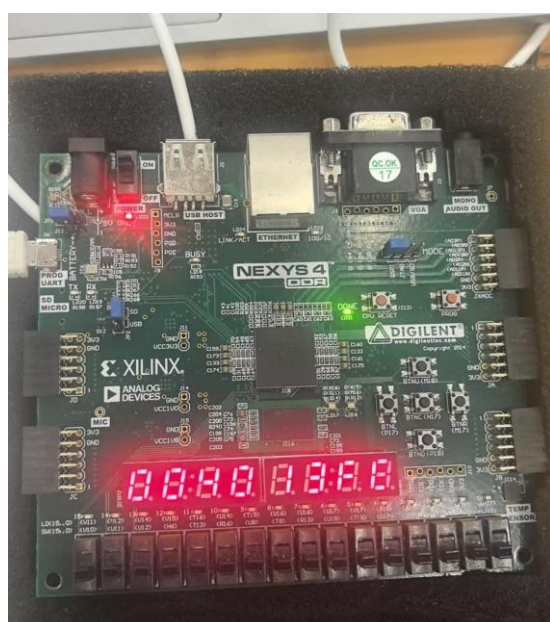
整个系统以 `clk_in` 和 `reset` 作为外部输入，其中 `cpu_clock_divider_inst` 首先对板载输入时钟进行分频，产生较低频率的 CPU 工作时钟，并送入 `cpu_core`。`cpu_core` 对应前述 `sccomp_dataflow` 模块，内部包含 CPU、IMEM 和 DMEM，是完成 54 条指令取指、译码、执行、访存和写回的主要部分。

CPU 输出当前程序计数器 `pc[31:0]`，由于 CPU 核心使用分频后的时钟，而数码管显示模块仍工作在原始板载时钟域，因此 PC 信号在送往显示模块之前依次经过 `pc_sync_1_reg` 和 `pc_sync_2_reg` 两级寄存器同步。这样可以使跨时钟域传递的 PC 值在显示时更加稳定，减小直接跨时钟域采样可能造成的不稳定现象。同步后的 32 位 PC 送入 `display_inst`，即 `seg7x16` 模块，由该模块完成八位数码管的动态扫描和十六进制数据显示，并最终输出 `o_seg[7:0]` 和 `o_sel[7:0]` 控制信号。从 RTL 结构可以看出，上板设计并没有改变 CPU 核心的数据通路和指令执行逻辑，而是在核心外增加了时钟分频、跨时钟域同步和数码管显示三个外围功能。

下面是 bit 流下板结果：



下载 bit 流后，数码管显示 CPU 当前的 32 位 PC 值。复位后 PC 从 MIPS 程序起始地址 00400000 开始运行，并随分频后的 CPU 时钟逐步变化；按照 `icf.xdc` 中的引脚约束，按下 N17 复位键后可重新回到 00400000。上板顶层中，PC 经两级寄存器同步后送入 `seg7x16` 模块，因此可以直接通过 8 位七段数码管观察程序执行地址的变化。



分频器将 CPU 工作时钟降低到适合人眼观察的频率，因此可以通过数码管清楚观察 PC 随程序执行不断变化。CPU 从 00400000 开始运行，在完成测试程序主体后进入末尾循环，在 004013F4、004013F8 和 004013FC 三个地址之间循环。根据测试 COE 对应的程序内容，004013F8 处为 00000000，作为 NOP 顺序执行到 004013FC；004013FC 处为 03E00008，即 JR \$31，而此前 004013F0 处的 JAL 已经将返回地址 004013F4 写入 \$31，因此 JR 又跳回 004013F4，随后经两条 NOP 再次到达 004013FC。标准仿真结果中也可以观察到 004013F8 → 004013FC → 004013F4 → 004013F8 的重复序列，与下板时数码管的循环显示一致，说明程序末尾控制流、JAL/JR 跳转以及 PC 显示均工作正常。

七、心得体会及建议

54 条指令的数据通路比 31 条时复杂了不少，刚开始还是打算按照原来的方法一条条补，到了 MULT、DIV、HI/LO、CP0 和几种字节访存都加进来以后，draw.io 里的线很快就挤在一起了。很多地方要考虑 MUX 增加哪个输入、控制信号怎么编码、最后和 Controller 里的变量能不能一一对应。总图前后改过很多次，单条指令图也跟着反复调整。最后决定把每条指令实际经过的部件和数据来源重新整理成操作时间表，再由表格反推控制信号。

学校给的综合测试最后一共比对 35836 行结果，这一点帮了很大忙，结合 tb 文件相当于给了一个可以实时检查和调试的工具，能看到到底是哪个寄存器的值出了问题。前面调试时 ModelSim 主要用来观察某一段 PC、指令和控制信号，等到程序基本能跑以后，再用 txt_compare 把 output.txt 和标准结果完整比较。最后 35836 行全部一致就说明网站上一定也能 AC 了。

到了综合以后，遇到的问题开始和前面的功能仿真不太一样（后仿真比前仿真麻烦，要考虑实际的硬件资源）。place_design 阶段曾经直接报过 DSP 级联资源无法布线，前面的 Verilog 逻辑明明可以正常仿真，放到具体 FPGA 资源上却过不了实现。乘法器后来重新调整了一次，才把 Implementation 跑通。时序报告也不是一开始就正常，早期出现过 WNS=-2.718 ns、TNS=-21.856 ns，继续修改以后最终得到 WNS=6.274 ns、TNS=0，Hold 和 Pulse Width 也都没有违例。

下板后 CPU 时钟被分频到肉眼能看清的速度，PC 直接显示在八位数码管上，复位后从 00400000 开始变化。程序主体跑完以后，数码管会在 004013F4、004013F8 和 004013FC 之间反复切换。004013FC 是 JR \$31，前面的 JAL 已经把 004013F4 写进 31 号寄存器，所以程序会重新跳回去。

工程文件本身也很容易出问题。前仿真、后仿真和下板用到的文件并不完全一样，还有 COE、XDC、IP 核生成文件以及不同的 testbench。中间出现过源码改完以后仿真结果还是旧的，最后发现是综合没有重新跑；也遇到过功能代码没有问题，实际加载的测试文件却不是当前要用的版本。

如果实验文档后面还会继续更新，我觉得测试文件和提交流程也可以再写明确一些。尤其是前仿真用什么文件、COE 怎么生成、后仿真需要哪些文件、下板时还要补哪些 XDC 和 IP 核内容，最好固定成一套目录和步骤。

源码: CPU.v

```
`timescale 1ns / 1ps
```

```
module CPU(  
    input clk,  
    input rst,  
    input [31:0] IM,  
    input [31:0] DM_Data_out,  
    output [31:0] pc,  
    output DM_R,  
    output DM_W,  
    output [31:0] DM_Addr,  
    output [31:0] DM_Data_in,  
    output [1:0] DM_Size,  
    output DM_Unsigned  
);  
    wire [5:0] OP = IM[31:26];  
    wire [4:0] Rsc = IM[25:21];  
    wire [4:0] Rtc = IM[20:16];  
    wire [4:0] IM_15_11 = IM[15:11];  
    wire [4:0] sa = IM[10:6];  
    wire [5:0] FUNC = IM[5:0];  
    wire [15:0] Imm16 = IM[15:0];  
    wire [25:0] target = IM[25:0];  
  
    wire [31:0] Rs;  
    wire [31:0] Rt;  
    wire RF_W;  
    wire RF_W_raw;  
    reg [4:0] Rdc;
```

```
reg [31:0] Rd;
```

```
regfile cpu_ref(  
.clk(clk), .rst(rst), .RegWrite(RF_W),  
.raddr1(Rsc), .raddr2(Rtc), .waddr(Rdc), .wdata(Rd),  
.rdata1(Rs), .rdata2(Rt)  
);
```

```
wire [31:0] Ext5 = {27'h0, sa};  
wire [31:0] Ext16_sign = {{16{Imm16[15]}}}, Imm16};  
wire [31:0] Ext16_zero = {16'h0000, Imm16};  
wire [31:0] NPC = pc + 32'd4;  
wire [31:0] BranchAddr = NPC + {{14{Imm16[15]}}}, Imm16, 2'b00};  
wire [31:0] JumpAddr = {NPC[31:28], target, 2'b00};  
reg [31:0] PC_next;
```

```
PC pc_reg(.clk(clk), .rst(rst), .pc_next(PC_next), .pc(pc));
```

```
reg [31:0] hi;  
reg [31:0] lo;  
reg [31:0] cp0_status;  
reg [31:0] cp0_cause;  
reg [31:0] cp0_epc;  
wire [31:0] cp0_rdata = (IM_15_11 == 5'd12) ? cp0_status :  
(IM_15_11 == 5'd13) ? cp0_cause :  
(IM_15_11 == 5'd14) ? cp0_epc : 32'h0;
```

```
wire [63:0] mult_unit_result;  
wire mult_unit_busy;  
wire mult_unit_done;  
wire [63:0] div_unit_result;  
wire div_unit_busy;  
wire div_unit_done;  
wire mdu_busy = mult_unit_busy || div_unit_busy;  
wire mdu_done = mult_unit_done || div_unit_done;  
wire [63:0] mdu_result = mult_unit_done ? mult_unit_result : div_unit_result;  
reg load_busy;
```

```
function [31:0] clz32;  
input [31:0] value;  
integer idx;  
reg found;  
begin  
clz32 = 32'd32; //默认全 0 情况下为 32
```

```

found = 1'b0; //标记为还没找到 1
for (idx = 31; idx >= 0; idx = idx - 1) begin
if (!found && value[idx]) begin//如果还没找到且第 idx 位为 1
clz32 = 31 - idx;//第 idx+1..31 都是 0，故 clz32 是(31-(idx+1))+1=31-idx
found = 1'b1;
end
end
end
endfunction

```

```

wire i_add, i_addu, i_sub, i_subu, i_and, i_or, i_xor, i_nor;
wire i_slt, i_sltu, i_sll, i_srl, i_sra, i_sllv, i_srlv, i_srav;
wire i_jr, i_jalr, i_syscall, i_break, i_mfhi, i_mthi, i_mflo, i_mtlo;
wire i_mult, i_multu, i_div, i_divu, i_teq;
wire i_mfc0, i_mtc0, i_eret, i_clz;
wire i_bgez, i_beq, i_bne;
wire i_addi, i_addiu, i_slti, i_sltiu, i_andi, i_ori, i_xori, i_lui;
wire i_lb, i_lh, i_lw, i_lbu, i_lhu, i_sb, i_sh, i_sw;
wire i_j, i_jal;

```

```

Decoder decoder(
.OP(OP), .Rsc(Rsc), .Rtc(Rtc), .FUNC(FUNC),
.i_add(i_add), .i_addu(i_addu), .i_sub(i_sub), .i_subu(i_subu),
.i_and(i_and), .i_or(i_or), .i_xor(i_xor), .i_nor(i_nor),
.i_slt(i_slt), .i_sltu(i_sltu), .i_sll(i_sll), .i_srl(i_srl),
.i_sra(i_sra), .i_sllv(i_sllv), .i_srlv(i_srlv), .i_srav(i_srav),
.i_jr(i_jr), .i_jalr(i_jalr), .i_syscall(i_syscall), .i_break(i_break),
.i_mfhi(i_mfhi), .i_mthi(i_mthi), .i_mflo(i_mflo), .i_mtlo(i_mtlo),
.i_mult(i_mult), .i_multu(i_multu), .i_div(i_div), .i_divu(i_divu),
.i_teq(i_teq), .i_mfc0(i_mfc0), .i_mtc0(i_mtc0), .i_eret(i_eret),
.i_clz(i_clz), .i_bgez(i_bgez), .i_beq(i_beq), .i_bne(i_bne),
.i_addi(i_addi), .i_addiu(i_addiu), .i_slti(i_slti), .i_sltiu(i_sltiu),
.i_andi(i_andi), .i_ori(i_ori), .i_xori(i_xori), .i_lui(i_lui),
.i_lb(i_lb), .i_lh(i_lh), .i_lw(i_lw), .i_lbu(i_lbu),
.i_lhu(i_lhu), .i_sb(i_sb), .i_sh(i_sh), .i_sw(i_sw),
.i_j(i_j), .i_jal(i_jal)
);

```

```

wire [2:0] M1;
wire [2:0] M2;
wire M3;
wire M4;
wire ext_zero;
wire [1:0] rdc_sel;

```

```

wire [3:0] ALUC;
wire mult_start;
wire mult_signed;
wire div_start;
wire div_signed;
wire hi_write_rs;
wire lo_write_rs;
wire cp0_write;
wire cp0_eret;
wire exception;
wire hi_write_rs_raw;
wire lo_write_rs_raw;
wire cp0_write_raw;
wire cp0_eret_raw;
wire exception_raw;
wire DM_R_raw;
wire DM_W_raw;
wire [4:0] exception_code;
wire rs_equal_rt = (Rs == Rt);
wire rs_negative = Rs[31];

```

```

assign RF_W = RF_W_raw && !mdu_busy;
assign DM_R = DM_R_raw && !mdu_busy;
assign DM_W = DM_W_raw && !mdu_busy;
assign hi_write_rs = hi_write_rs_raw && !mdu_busy;
assign lo_write_rs = lo_write_rs_raw && !mdu_busy;
assign cp0_write = cp0_write_raw && !mdu_busy;
assign cp0_eret = cp0_eret_raw && !mdu_busy;
assign exception = exception_raw && !mdu_busy;

```

```

Controller controller(
.i_add(i_add), .i_addu(i_addu), .i_sub(i_sub), .i_subu(i_subu),
.i_and(i_and), .i_or(i_or), .i_xor(i_xor), .i_nor(i_nor),
.i_slt(i_slt), .i_sltu(i_sltu), .i_sll(i_sll), .i_srl(i_srl),
.i_sra(i_sra), .i_sllv(i_sllv), .i_srlv(i_srlv), .i_srav(i_srav),
.i_jr(i_jr), .i_jalr(i_jalr), .i_syscall(i_syscall), .i_break(i_break),
.i_mfhi(i_mfhi), .i_mthi(i_mthi), .i_mflo(i_mflo), .i_mtlo(i_mtlo),
.i_mult(i_mult), .i_multu(i_multu), .i_div(i_div), .i_divu(i_divu),
.i_teq(i_teq), .i_mfc0(i_mfc0), .i_mtc0(i_mtc0), .i_eret(i_eret),
.i_clz(i_clz), .i_bgez(i_bgez), .i_beq(i_beq), .i_bne(i_bne),
.i_addi(i_addi), .i_addiu(i_addiu), .i_slti(i_slti), .i_sltiu(i_sltiu),
.i_andi(i_andi), .i_ori(i_ori), .i_xori(i_xori), .i_lui(i_lui),
.i_lb(i_lb), .i_lh(i_lh), .i_lw(i_lw), .i_lbu(i_lbu),
.i_lhu(i_lhu), .i_sb(i_sb), .i_sh(i_sh), .i_sw(i_sw),

```

```

.i_j(i_j), .i_jal(i_jal),
.rs_equal_rt(rs_equal_rt), .rs_negative(rs_negative),
.load_busy(load_busy), .mdu_busy(mdu_busy),
.M1(M1), .M2(M2), .M3(M3), .M4(M4), .ext_zero(ext_zero),
.rdc_sel(rdc_sel), .ALUC(ALUC), .RF_W(RF_W_raw),
.DM_R(DM_R_raw), .DM_W(DM_W_raw), .DM_Size(DM_Size), .DM_Unsigned(DM_Unsigned),
.mult_start(mult_start), .mult_signed(mult_signed),
.div_start(div_start), .div_signed(div_signed),
.hi_write_rs(hi_write_rs_raw), .lo_write_rs(lo_write_rs_raw),
.cp0_write(cp0_write_raw), .cp0_eret(cp0_eret_raw),
.exception(exception_raw), .exception_code(exception_code)
);

```

```

wire [31:0] Ext16 = ext_zero ? Ext16_zero : Ext16_sign;
wire [31:0] alu_a = M3 ? Rs : Ext5;
wire [31:0] alu_b = M4 ? Ext16 : Rt;
reg [31:0] alu_result;

```

```

always @(*) begin
case (ALUC)
4'b0000: alu_result = alu_a + alu_b;
4'b0001: alu_result = alu_a - alu_b;
4'b0010: alu_result = alu_a & alu_b;
4'b0011: alu_result = alu_a | alu_b;
4'b0100: alu_result = alu_a ^ alu_b;
4'b0101: alu_result = ~(alu_a | alu_b);
4'b0110: alu_result = ($signed(alu_a) < $signed(alu_b)) ? 32'd1 : 32'd0;
4'b0111: alu_result = (alu_a < alu_b) ? 32'd1 : 32'd0;
4'b1000: alu_result = alu_b << alu_a[4:0];
4'b1001: alu_result = alu_b >> alu_a[4:0];
4'b1010: alu_result = $signed(alu_b) >>> alu_a[4:0];
default: alu_result = 32'h0;
endcase
end

```

```

always @(*) begin
case (rdc_sel)
2'b00: Rdc = Rtc;
2'b01: Rdc = IM_15_11;
2'b10: Rdc = 5'd31;
default: Rdc = (IM_15_11 == 5'd0) ? 5'd31 : IM_15_11;
endcase
end

```

```

always @(*) begin
case (M2)
3'b000: Rd = alu_result;
3'b001: Rd = DM_Data_out;
3'b010: Rd = NPC;
3'b011: Rd = hi;
3'b100: Rd = lo;
3'b101: Rd = cp0_rdata;
3'b110: Rd = clz32(Rs);
default: Rd = {Imm16, 16'h0};
endcase
end

```

```

always @(*) begin
case (M1)
3'b000: PC_next = NPC;
3'b001: PC_next = BranchAddr;
3'b010: PC_next = JumpAddr;
3'b011: PC_next = Rs;
3'b100: PC_next = cp0_epc + 32'd4;
3'b101: PC_next = 32'h0040_0004;
3'b110: PC_next = pc;
default: PC_next = NPC;
endcase
end

```

```

assign DM_Addr = alu_result;
assign DM_Data_in = Rt;

```

```

multiplier cpu_multiplier(
.clk(clk), .rst(rst),
.start(mult_start && !mdu_busy), .signed_mode(mult_signed),
.a(Rs), .b(Rt),
.result(mult_unit_result), .busy(mult_unit_busy), .done(mult_unit_done)
);

```

```

divider cpu_divider(
.clk(clk), .rst(rst),
.start(div_start && !mdu_busy), .signed_mode(div_signed),
.dividend(Rs), .divisor(Rt),
.result(div_unit_result), .busy(div_unit_busy), .done(div_unit_done)
);

```

```

always @(posedge clk or posedge rst) begin
  if (rst) begin
    hi <= 32'h0;
    lo <= 32'h0;
    load_busy <= 1'b0;
    cp0_status <= 32'h0;
    cp0_cause <= 32'h0;
    cp0_epc <= 32'h0;
  end else begin
    if (load_busy)
      load_busy <= 1'b0;
    else if (DM_R)
      load_busy <= 1'b1;

    if (mdu_done) begin
      hi <= mdu_result[63:32];
      lo <= mdu_result[31:0];
    end else if (hi_write_rs) begin
      hi <= Rs;
    end else if (lo_write_rs) begin
      lo <= Rs;
    end

    if (cp0_write && IM_15_11 == 5'd12) cp0_status <= Rt;
    if (cp0_write && IM_15_11 == 5'd13) cp0_cause <= Rt;
    if (cp0_write && IM_15_11 == 5'd14) cp0_epc <= Rt;
    if (exception) begin
      cp0_status <= cp0_status << 5;
      cp0_cause <= {25'h0, exception_code, 2'b00};
      cp0_epc <= pc;
    end else if (cp0_eret) begin
      cp0_status <= cp0_status >> 5;
    end
  end
end
endmodule

```

Controller.v

```

`timescale 1ns / 1ps
module Controller(
  input i_add,
  input i_addu,
  input i_sub,
  input i_subu,

```

input i_and,
input i_or,
input i_xor,
input i_nor,
input i_slt,
input i_sltu,
input i_sll,
input i_srl,
input i_sra,
input i_sllv,
input i_srlv,
input i_srav,
input i_jr,
input i_jalr,
input i_syscall,
input i_break,
input i_mfhi,
input i_mthi,
input i_mflo,
input i_mtlo,
input i_mult,
input i_multu,
input i_div,
input i_divu,
input i_teq,
input i_mfc0,
input i_mtc0,
input i_eret,
input i_clz,
input i_bgez,
input i_beq,
input i_bne,
input i_addi,
input i_addiu,
input i_slti,
input i_sltiu,
input i_andi,
input i_ori,
input i_xori,
input i_lui,
input i_lb,
input i_lh,
input i_lw,
input i_lbu,

```

input i_lhu,
input i_sb,
input i_sh,
input i_sw,
input i_j,
input i_jal,
input rs_equal_rt,
input rs_negative,
input load_busy,
input mdu_busy,
output reg [2:0] M1,
output reg [2:0] M2,
output reg M3,
output reg M4,
output reg ext_zero,
output reg [1:0] rdc_sel,
output reg [3:0] ALUC,
output reg RF_W,
output reg DM_R,
output reg DM_W,
output reg [1:0] DM_Size,
output reg DM_Unsigned,
output reg mult_start,
output reg mult_signed,
output reg div_start,
output reg div_signed,
output reg hi_write_rs,
output reg lo_write_rs,
output reg cp0_write,
output reg cp0_eret,
output reg exception,
output reg [4:0] exception_code
);
always @(*) begin
M1 = 3'b000; // PC_next 选择 NPC
M2 = 3'b000; // Rd 选择 ALU 结果
M3 = 1'b1; // ALU 输入 A 选择 Rs
M4 = 1'b0; // ALU 输入 B 选择 Rt
ext_zero = 1'b0; // Ext16 默认进行符号扩展
rdc_sel = 2'b00; // Rdc 默认选择 rt 字段
ALUC = 4'b0000; // ALU 默认执行加法
RF_W = 1'b0;
DM_R = 1'b0;
DM_W = 1'b0;

```

```
DM_Size = 2'b10; // 默认按 32 位字访问 DMEM
DM_Unsigned = 1'b0;
mult_start = 1'b0;
mult_signed = 1'b0;
div_start = 1'b0;
div_signed = 1'b0;
hi_write_rs = 1'b0;
lo_write_rs = 1'b0;
cp0_write = 1'b0;
cp0_eret = 1'b0;
exception = 1'b0;
exception_code = 5'b00000;
```

```
if (i_add || i_addu) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0000;
end else if (i_sub || i_subu) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0001;
end else if (i_and) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0010;
end else if (i_or) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0011;
end else if (i_xor) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0100;
end else if (i_nor) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0101;
end else if (i_slt) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0110;
end else if (i_sltu) begin
RF_W = 1'b1; rdc_sel = 2'b01; ALUC = 4'b0111;
end else if (i_sll || i_srl || i_sra) begin
RF_W = 1'b1; rdc_sel = 2'b01; M3 = 1'b0;
if (i_sll) ALUC = 4'b1000;
else if (i_srl) ALUC = 4'b1001;
else ALUC = 4'b1010;
end else if (i_sllv || i_srlv || i_srav) begin
RF_W = 1'b1; rdc_sel = 2'b01;
if (i_sllv) ALUC = 4'b1000;
else if (i_srlv) ALUC = 4'b1001;
else ALUC = 4'b1010;
end else if (i_addi || i_addiu) begin
RF_W = 1'b1; M4 = 1'b1; ALUC = 4'b0000;
end else if (i_slti) begin
RF_W = 1'b1; M4 = 1'b1; ALUC = 4'b0110;
end else if (i_sltiu) begin
```

```

RF_W = 1'b1; M4 = 1'b1; ALUC = 4'b0111;
end else if (i_andi || i_ori || i_xori) begin
RF_W = 1'b1; M4 = 1'b1; ext_zero = 1'b1;
if (i_andi) ALUC = 4'b0010;
else if (i_ori) ALUC = 4'b0011;
else ALUC = 4'b0100;
end else if (i_lui) begin
RF_W = 1'b1; M2 = 3'b111;
end else if (i_lb || i_lbu || i_lh || i_lhu || i_lw) begin
RF_W = load_busy; M2 = 3'b001; M4 = 1'b1; ALUC = 4'b0000;
end else if (i_sb || i_sh || i_sw) begin
M4 = 1'b1; ALUC = 4'b0000;
end else if (i_jal) begin
RF_W = 1'b1; rdc_sel = 2'b10; M2 = 3'b010; //JAL 跳转的同时把下一条指令地址 NPC 写入 31
号寄存器，供函数执行结束后返回
end else if (i_jalr) begin
RF_W = 1'b1; rdc_sel = 2'b11; M2 = 3'b010;
end else if (i_mfhi) begin
RF_W = 1'b1; rdc_sel = 2'b01; M2 = 3'b011;
end else if (i_mflo) begin
RF_W = 1'b1; rdc_sel = 2'b01; M2 = 3'b100;
end else if (i_mfc0) begin
RF_W = 1'b1; M2 = 3'b101;
end else if (i_clz) begin
RF_W = 1'b1; rdc_sel = 2'b01; M2 = 3'b110;
end
end

```

```

DM_R = i_lb || i_lbu || i_lh || i_lhu || i_lw;
DM_W = i_sb || i_sh || i_sw;
if (i_lb || i_lbu || i_sb) DM_Size = 2'b00;
else if (i_lh || i_lhu || i_sh) DM_Size = 2'b01;
else DM_Size = 2'b10;
DM_Unsigned = i_lbu || i_lhu;

```

```

mult_start = i_mult || i_multu;
mult_signed = i_mult;
div_start = i_div || i_divu;
div_signed = i_div;
hi_write_rs = i_mthi;
lo_write_rs = i_mtlo;
cp0_write = i_mtc0;
cp0_eret = i_eret;

```

```

exception = i_break || i_syscall || (i_teq && rs_equal_rt);

```

```

if (i_syscall) exception_code = 5'b01000;
else if (i_break) exception_code = 5'b01001;
else if (i_teq && rs_equal_rt) exception_code = 5'b01101;

```

```

if ((i_beq && rs_equal_rt) || (i_bne && !rs_equal_rt) ||
(i_bgez && !rs_negative)) M1 = 3'b001;
else if (i_j || i_jal) M1 = 3'b010;
else if (i_jr || i_jalr) M1 = 3'b011;
else if (i_eret) M1 = 3'b100;
else if (exception) M1 = 3'b101;
if (mdu_busy || load_busy) M1 = 3'b110;
end
endmodule

```

Decoder.v

```

`timescale 1ns / 1ps

```

```

module Decoder(
input [5:0] OP,
input [4:0] Rsc,
input [4:0] Rtc,
input [5:0] FUNC,
output i_add,
output i_addu,
output i_sub,
output i_subu,
output i_and,
output i_or,
output i_xor,
output i_nor,
output i_slt,
output i_sltu,
output i_sll,
output i_srl,
output i_sra,
output i_sllv,
output i_srlv,
output i_srav,
output i_jr,
output i_jalr,
output i_syscall,
output i_break,
output i_mfhi,
output i_mthi,

```

```

output i_mflo,
output i_mtlo,
output i_mult,
output i_multu,
output i_div,
output i_divu,
output i_teq,
output i_mfc0,
output i_mtc0,
output i_eret,
output i_clz,
output i_bgez,
output i_beq,
output i_bne,
output i_addi,
output i_addiu,
output i_slti,
output i_sltiu,
output i_andi,
output i_ori,
output i_xori,
output i_lui,
output i_lb,
output i_lh,
output i_lw,
output i_lbu,
output i_lhu,
output i_sb,
output i_sh,
output i_sw,
output i_j,
output i_jal
);
assign i_add = (OP == 6'b000000 && FUNC == 6'b100000);
assign i_addu = (OP == 6'b000000 && FUNC == 6'b100001);
assign i_sub = (OP == 6'b000000 && FUNC == 6'b100010);
assign i_subu = (OP == 6'b000000 && FUNC == 6'b100011);
assign i_and = (OP == 6'b000000 && FUNC == 6'b100100);
assign i_or = (OP == 6'b000000 && FUNC == 6'b100101);
assign i_xor = (OP == 6'b000000 && FUNC == 6'b100110);
assign i_nor = (OP == 6'b000000 && FUNC == 6'b100111);
assign i_slt = (OP == 6'b000000 && FUNC == 6'b101010);
assign i_sltu = (OP == 6'b000000 && FUNC == 6'b101011);
assign i_sll = (OP == 6'b000000 && FUNC == 6'b000000);

```

```

assign i_srl = (OP == 6'b000000 && FUNC == 6'b000010);
assign i_sra = (OP == 6'b000000 && FUNC == 6'b000011);
assign i_sllv = (OP == 6'b000000 && FUNC == 6'b000100);
assign i_srlv = (OP == 6'b000000 && FUNC == 6'b000110);
assign i_srav = (OP == 6'b000000 && FUNC == 6'b000111);
assign i_jr = (OP == 6'b000000 && FUNC == 6'b001000);
assign i_jalr = (OP == 6'b000000 && FUNC == 6'b001001);
assign i_syscall = (OP == 6'b000000 && FUNC == 6'b001100);
assign i_break = (OP == 6'b000000 && FUNC == 6'b001101);
assign i_mfhi = (OP == 6'b000000 && FUNC == 6'b010000);
assign i_mthi = (OP == 6'b000000 && FUNC == 6'b010001);
assign i_mflo = (OP == 6'b000000 && FUNC == 6'b010010);
assign i_mtlo = (OP == 6'b000000 && FUNC == 6'b010011);
assign i_mult = (OP == 6'b000000 && FUNC == 6'b011000);
assign i_multu = (OP == 6'b000000 && FUNC == 6'b011001);
assign i_div = (OP == 6'b000000 && FUNC == 6'b011010);
assign i_divu = (OP == 6'b000000 && FUNC == 6'b011011);
assign i_teq = (OP == 6'b000000 && FUNC == 6'b110100);
assign i_mfc0 = (OP == 6'b010000 && Rsc == 5'b000000);
assign i_mtc0 = (OP == 6'b010000 && Rsc == 5'b001000);
assign i_eret = (OP == 6'b010000 && Rsc == 5'b10000 && FUNC == 6'b011000);
assign i_clz = (OP == 6'b011100 && FUNC == 6'b100000);
assign i_bgez = (OP == 6'b000001 && Rtc == 5'b000001);
assign i_beq = (OP == 6'b000100);
assign i_bne = (OP == 6'b000101);
assign i_addi = (OP == 6'b001000);
assign i_addiu = (OP == 6'b001001);
assign i_slti = (OP == 6'b001010);
assign i_sltiu = (OP == 6'b001011);
assign i_andi = (OP == 6'b001100);
assign i_ori = (OP == 6'b001101);
assign i_xori = (OP == 6'b001110);
assign i_lui = (OP == 6'b001111);
assign i_lb = (OP == 6'b100000);
assign i_lh = (OP == 6'b100001);
assign i_lw = (OP == 6'b100011);
assign i_lbu = (OP == 6'b100100);
assign i_lhu = (OP == 6'b100101);
assign i_sb = (OP == 6'b101000);
assign i_sh = (OP == 6'b101001);
assign i_sw = (OP == 6'b101011);
assign i_j = (OP == 6'b000010);
assign i_jal = (OP == 6'b000011);
endmodule

```

PC.v

```
`timescale 1ns / 1ps
```

```
module PC(  
    input clk,  
    input rst,  
    input [31:0] pc_next,  
    output reg [31:0] pc  
);  
initial pc = 32'h0040_0000;  
  
always @(negedge clk or posedge rst) begin  
    if (rst) //复位至初值，对应板载 N17  
        pc <= 32'h0040_0000;  
    else //时钟下降沿就更新 PC，这样可以保证在时钟上升沿时 PC 已经是最新的值了  
        pc <= pc_next;  
    end  
endmodule
```

Regfile.v

```
`timescale 1ns / 1ps
```

```
module regfile(  
    input clk,  
    input rst,  
    input RegWrite,  
    input [4:0] raddr1,  
    input [4:0] raddr2,  
    input [4:0] waddr,  
    input [31:0] wdata,  
    output [31:0] rdata1,  
    output [31:0] rdata2  
);  
reg [31:0] array_reg [0:31];  
integer i;  
  
assign rdata1 = array_reg[raddr1];  
assign rdata2 = array_reg[raddr2];  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        for (i = 0; i < 32; i = i + 1)  
            array_reg[i] <= 32'h0;  
    end  
end
```

```

end else begin
array_reg[0] <= 32'h0;
if (RegWrite && waddr != 5'd0) //写入信号有效且地址不为 0 时才写入，MIPS 的 0 号寄存器
固定为 0
array_reg[waddr] <= wdata; //把 wdata 数据写入第 waddr 号寄存器，可以读取，但不能写入
end
end
endmodule

```

IMEM.v

```

`timescale 1ns / 1ps

module IMEM(
input [10:0] im_addr_in,
output [31:0] im_instr_out
);
imem_rom imem_rom_inst(
.a(im_addr_in),
.spo(im_instr_out)
);
endmodule

```

DMEM.v

```

`timescale 1ns / 1ps

module DMEM(
input dm_clk,
input dm_ena,
input dm_r,
input dm_w,
input [12:0] dm_addr,
input [31:0] dm_data_in,
input [1:0] dm_size,
input dm_unsigned, // 读出的数据不满 32 位时，1 表示左边补 0，0 表示左边复制符号位
output [31:0] dm_data_out
);
reg [7:0] dmem0 [0:2047]; // 四个 dmem 数组可以看成四列，这一列存每行第 1 个字节
reg [7:0] dmem1 [0:2047]; // 这一列存每行第 2 个字节
reg [7:0] dmem2 [0:2047]; // 这一列存每行第 3 个字节
reg [7:0] dmem3 [0:2047]; // 这一列存每行第 4 个字节，四列合起来是一个 32 bit 的字
integer i;
wire [10:0] wa = dm_addr[12:2]; // 地址高 11 位选择哪一行，也就是字节地址除以 4
wire [1:0] byte_sel = dm_addr[1:0]; // 地址低 2 位选择这一行中的哪一个字节列
reg [31:0] read_data; // 把读出的字节或半字扩展成 32 位后存在这里

```

```
assign dm_data_out = read_data; // 把整理好的 32 位读取结果送回 CPU
```

```
initial begin // 开始时执行一次，把 2048 行的四列全部清 0，不需要等 2048 个时钟
```

```
read_data = 32'h0000_0000;
```

```
for (i = 0; i < 2048; i = i + 1) begin
```

```
dmem0[i] = 8'h00;
```

```
dmem1[i] = 8'h00;
```

```
dmem2[i] = 8'h00;
```

```
dmem3[i] = 8'h00;
```

```
end
```

```
end
```

```
always @(posedge dm_clk) begin // 只在 dm_clk 从 0 变成 1 时进行一次读写
```

```
if (dm_ena && dm_w && !dm_r) begin // 存储器已启用，并且现在只写不读
```

```
case (dm_size) // 00 执行 SB，01 执行 SH，CPU 正常给 10 时执行 SW
```

```
2'b00: begin // SB 只写 dm_data_in 的最低 8 位，这次用不到高 24 位
```

```
case (byte_sel)
```

```
2'b00: dmem0[wa] <= dm_data_in[7:0];
```

```
2'b01: dmem1[wa] <= dm_data_in[7:0];
```

```
2'b10: dmem2[wa] <= dm_data_in[7:0];
```

```
default: dmem3[wa] <= dm_data_in[7:0];
```

```
endcase
```

```
end
```

```
2'b01: begin // SH 一次写相邻的两个字节
```

```
if (byte_sel[1]) begin // 为 1 就写一行的后两列，否则写前两列
```

```
dmem2[wa] <= dm_data_in[7:0];
```

```
dmem3[wa] <= dm_data_in[15:8];
```

```
end else begin
```

```
dmem0[wa] <= dm_data_in[7:0];
```

```
dmem1[wa] <= dm_data_in[15:8];
```

```
end
```

```
end
```

```
default: begin // SW 把 32 位数据拆成四个字节，写满同一行
```

```
dmem0[wa] <= dm_data_in[7:0];
```

```
dmem1[wa] <= dm_data_in[15:8];
```

```
dmem2[wa] <= dm_data_in[23:16];
```

```
dmem3[wa] <= dm_data_in[31:24];
```

```
end
```

```
endcase
```

```
end
```

```
if (dm_ena && dm_r && !dm_w) begin // 存储器已启用，并且现在只读不写
```

```
case (dm_size)
```

```

2'b00: begin // LB 和 LBU 只读一个字节，再把它扩展成 32 位
case (byte_sel)
2'b00: read_data <= dm_unsigned ? {24'h0, dmem0[wa]} :
{{24{dmem0[wa][7]}}, dmem0[wa]}; // 读第 1 列，LBU 补 0，LB 复制符号位 24 次并补充
2'b01: read_data <= dm_unsigned ? {24'h0, dmem1[wa]} :
{{24{dmem1[wa][7]}}, dmem1[wa]};
2'b10: read_data <= dm_unsigned ? {24'h0, dmem2[wa]} :
{{24{dmem2[wa][7]}}, dmem2[wa]};
default: read_data <= dm_unsigned ? {24'h0, dmem3[wa]} :
{{24{dmem3[wa][7]}}, dmem3[wa]};
endcase
end
2'b01: begin
if (byte_sel[1])
read_data <= dm_unsigned ? {16'h0, dmem3[wa], dmem2[wa]} :
{{16{dmem3[wa][7]}}, dmem3[wa], dmem2[wa]}; // 读第 3、4 列，LHU 补 0，LH 复制符号最
高位 16 次并补充
else
read_data <= dm_unsigned ? {16'h0, dmem1[wa], dmem0[wa]} :
{{16{dmem1[wa][7]}}, dmem1[wa], dmem0[wa]};
end
default: read_data <= {dmem3[wa], dmem2[wa], dmem1[wa], dmem0[wa]}; // LW 整行直接拼接
成 32 位
endcase
end
end
endmodule

```

Multiplier.v

```
`timescale 1ns / 1ps
```

```

module multiplier(
input clk,
input rst,
input start,
input signed_mode,
input [31:0] a,
input [31:0] b,
output reg [63:0] result,
output reg busy,
output reg done
);
reg [2:0] phase; //当前进行的阶段，000-100 分别对应 5 个阶段
reg [63:0] partial [0:31]; //32 个部分积，每个部分积 64 位

```

```

reg [63:0] sum_2 [0:15];//第一轮两两相加后变为 16 个中间和
reg [63:0] sum_3 [0:7];//第二轮两两相加后变为 8 个中间和
reg [63:0] sum_4 [0:3];//第三轮两两相加后变为 4 个中间和
reg [63:0] sum_5 [0:1];//第四轮两两相加后变为 2 个中间和

wire [63:0] signed_a = {{32{a[31]}}}, a};//将 a 扩展为 64 位有符号数，复制符号位 32 次
wire [63:0] neg_signed_a = ~signed_a + 64'd1;//按位取反加 1 得到 -a 的补码表示
wire [32:0] booth_b = {b, 1'b0};//补 1 个 0 开始 Booth
wire [63:0] unsigned_a = {32'h0000_0000, a};//无符号的 a 直接补 32 个 0

integer i;
always @(posedge clk or posedge rst) begin
if (rst) begin //把结果、busy、完成、阶段号和所有部分积、中间和清零
result <= 64'h0;
busy <= 1'b0;
done <= 1'b0;
phase <= 3'd0;
for (i = 0; i < 32; i = i + 1)
partial[i] <= 64'h0;
for (i = 0; i < 16; i = i + 1)
sum_2[i] <= 64'h0;
for (i = 0; i < 8; i = i + 1)
sum_3[i] <= 64'h0;
for (i = 0; i < 4; i = i + 1)
sum_4[i] <= 64'h0;
for (i = 0; i < 2; i = i + 1)
sum_5[i] <= 64'h0;
end else if (done) begin //如果完成了乘法运算，清除 busy 和 done 信号
done <= 1'b0;
busy <= 1'b0;
end else if (start && !busy) begin //如果开始乘法运算且当前不忙，则设置 busy 信号并进入第
0 阶段，也即计算 partial 部分积阶段
busy <= 1'b1;
phase <= 3'd0;
if (signed_mode) begin //MULT
for (i = 0; i < 32; i = i + 1) begin //遍历 32 位的 b，计算每一位对应的部分积
case ({booth_b[i + 1], booth_b[i]})
2'b01: partial[i] <= signed_a << i; //01,则加 a(左移 i 位即乘以 2^i)
2'b10: partial[i] <= neg_signed_a << i; //10,则减 a(左移 i 位即乘以 2^i)
default: partial[i] <= 64'h0; //00 或 11 则部分积为 0
endcase
end
end else begin //MULTU
for (i = 0; i < 32; i = i + 1)

```

```

partial[i] <= b[i] ? (unsigned_a << i) : 64'h0; //如果 b 的第 i 位为 1，则部分积为 a 左移 i 位，否则第 i 位为 0，部分积为 0
end
end else if (busy) begin //busy 已经在上一个分支里变为 1 了，开始进入阶段计算
done <= 1'b0;
case (phase)
3'd0: begin //32 变 16
for (i = 0; i < 16; i = i + 1) //并行得到 16 个中间和
sum_2[i] <= partial[i << 1] + partial[(i << 1) + 1]; //类似二叉树的下标规律，把 2i 和 2i+1 的部分积(类似左右孩子)相加得到第 i 个部分积(类似父节点)
phase <= 3'd1; //进入下一阶段
end
3'd1: begin //16 变 8
for (i = 0; i < 8; i = i + 1)
sum_3[i] <= sum_2[i << 1] + sum_2[(i << 1) + 1];
phase <= 3'd2;
end
3'd2: begin //8 变 4
for (i = 0; i < 4; i = i + 1)
sum_4[i] <= sum_3[i << 1] + sum_3[(i << 1) + 1];
phase <= 3'd3;
end
3'd3: begin //4 变 2
sum_5[0] <= sum_4[0] + sum_4[1];
sum_5[1] <= sum_4[2] + sum_4[3];
phase <= 3'd4;
end
default: begin //2 变 1，即最终结果
result <= sum_5[0] + sum_5[1];
done <= 1'b1; //完成乘法运算，设置 done 信号，标记结果有效，CPU 将在下个上升沿写入 HI 和 LO 寄存器
end
endcase
end
end
endmodule

```

Divider.v

```
`timescale 1ns / 1ps
```

```

module divider(
input clk,
input rst,
input start,

```

```

input signed_mode,
input [31:0] dividend,
input [31:0] divisor,
output reg [63:0] result, //高 32 位为余数 HI, 低 32 位为商 LO
output reg busy,
output reg done
);
reg [5:0] count; //记录已经算出的商的位数
reg quotient_negative; //最终的商是否为负数
reg remainder_negative; //最终的余数是否为负数, 符号和被除数符合应该相同
reg [63:0] work; //前 32 位为余数, 后 32 位逐渐形成商
reg [31:0] divisor_work; //除数的绝对值, 32 轮运算期间保持不变

wire [31:0] dividend_abs = signed_mode && dividend[31] //判断是否为有符号模式且被除数为负数
? (~dividend + 32'd1) : dividend; //如果是, 那么取被除数的绝对值 (也即相反数), 否则直接使用被除数
wire [31:0] divisor_abs = signed_mode && divisor[31]
? (~divisor + 32'd1) : divisor; //同理, 当有符号且除数为负时取除数的绝对值, 用来试减
wire [31:0] quotient_value = quotient_negative //完成无符号运算后决定最终的商是否为负数, 如果是, 那么还原商的补码, 否则直接使用商(work 的低 32 位)
? (~work[31:0] + 32'd1) : work[31:0];
wire [31:0] remainder_value = remainder_negative //同理, 决定最终的余数是否为负数, 如果是, 那么还原余数的补码, 否则直接使用余数(work 的高 32 位)
? (~work[63:32] + 32'd1) : work[63:32];

always @(posedge clk or posedge rst) begin
if (rst) begin //复位时清空所有寄存器
result <= 64'h0;
busy <= 1'b0;
done <= 1'b0;
count <= 6'd0;
quotient_negative <= 1'b0;
remainder_negative <= 1'b0;
work <= 64'h0;
divisor_work <= 32'h0;
end else if (done) begin //当除法运算完成后, 清空 busy 和 done 信号
done <= 1'b0;
busy <= 1'b0;
end else if (start && !busy) begin //开始且不忙时, 初始化除法运算
busy <= 1'b1; //记录忙, 下一轮开始执行 busy 分支
count <= 6'd0; //初始化计数器
quotient_negative <= signed_mode && (dividend[31] ^ divisor[31]); //判断最终的商是否为负数, 只有当有符号模式且被除数和除数符号不同才为负数

```

```

remainder_negative <= signed_mode && dividend[31]; //判断最终的余数是否为负数，只有当
有符号模式且被除数为负数才为负数（因为余数的符号和被除数相同）
work <= {32'h0000_0000, dividend_abs}; //初始化 work 寄存器，前 32 位为余数，后 32 位为
被除数的绝对值
divisor_work <= divisor_abs; //锁存除数的绝对值，即使后续变化也不影响除数
end else if (busy) begin //正式开始
if (count < 6'd32) begin //每轮生成 1 位商，共执行 32 轮
if (work[62:31] >= divisor_work) //work 整体左移一位后的候选余数够减除数
work <= {work[62:31] - divisor_work, work[30:0], 1'b1}; //候选余数减去除数，同时商左移并
补 1
else
work <= {work[62:0], 1'b0}; //候选余数不够减，不执行减法，同时商左移并补 0
count <= count + 6'd1; //本轮生成 1 位商，计数器加 1
end else begin //32 位商全部生成，整理最终结果
result <= {remainder_value, quotient_value}; //高 32 位保存余数，低 32 位保存商
done <= 1'b1; //结果有效，CPU 将在下一上升沿把结果写入 HI 和 LO
end
end
end
endmodule

```

sccomp_dataflow.v

```

`timescale 1ns / 1ps

```

```

module sccomp_dataflow(
input clk_in,
input reset,
output [31:0] inst,
output [31:0] pc
);
wire [31:0] cpu_pc; //取指令
wire [31:0] cpu_instr; //取指令
wire DM_R;
wire DM_W;
wire [31:0] DM_Addr;
wire [31:0] DM_Data_in;
wire [31:0] DM_Data_out;
wire [1:0] DM_Size;
wire DM_Unsigned;
wire DM_CS = DM_R | DM_W;

wire [31:0] imem_byte_addr = cpu_pc - 32'h0040_0000; //计算相对地址，便于下一行转化 ROM
下标
wire [10:0] imem_word_addr = imem_byte_addr[12:2]; //取出相对地址的低 11 位，也即除以 4

```

后的结果，IMEM_ROM 共 2048 位故有 11 位即可
wire [31:0] dmem_byte_addr = DM_Addr - 32'h1001_0000;
wire [12:0] dmem_addr = dmem_byte_addr[12:0];

```
assign pc = cpu_pc;  
assign inst = cpu_instr;
```

```
CPU sccpu(  
  .clk(clk_in),  
  .rst(reset),  
  .IM(cpu_instr),  
  .DM_Data_out(DM_Data_out),  
  .pc(cpu_pc),  
  .DM_R(DM_R),  
  .DM_W(DM_W),  
  .DM_Addr(DM_Addr),  
  .DM_Data_in(DM_Data_in),  
  .DM_Size(DM_Size),  
  .DM_Unsigned(DM_Unsigned)  
);
```

```
IMEM imem(  
  .im_addr_in(imem_word_addr), // 输入的地址  
  .im_instr_out(cpu_instr) // 根据地址从 COE 中拿出的指令  
);
```

```
DMEM dmem(  
  .dm_clk(clk_in),  
  .dm_ena(DM_CS),  
  .dm_r(DM_R),  
  .dm_w(DM_W),  
  .dm_addr(dmem_addr),  
  .dm_data_in(DM_Data_in),  
  .dm_size(DM_Size),  
  .dm_unsigned(DM_Unsigned),  
  .dm_data_out(DM_Data_out)  
);  
endmodule
```

board_scomp_dataflow.v

```
`timescale 1ns / 1ps
```

```
module board_scomp_dataflow(  
  input clk_in,
```

```

input reset,
output [7:0] o_seg,
output [7:0] o_sel
);
wire cpu_clk;
wire [31:0] inst;
wire [31:0] pc;
reg [31:0] pc_sync_1;
reg [31:0] pc_sync_2;

cpu_clock_divider #(
.HALF_PERIOD_CYCLES(500000)
) cpu_clock_divider_inst(
.clk_in(clk_in),
.reset(reset),
.clk_out(cpu_clk)
);

sccomp_dataflow cpu_core(
.clk_in(cpu_clk),
.reset(reset),
.inst(inst),
.pc(pc)
);

always @(posedge clk_in or posedge reset) begin
if (reset) begin
pc_sync_1 <= 32'h0;
pc_sync_2 <= 32'h0;
end else begin
pc_sync_1 <= pc;
pc_sync_2 <= pc_sync_1;
end
end

seg7x16 display_inst(
.clk(clk_in),
.reset(reset),
.cs(1'b1),
.i_data(pc_sync_2),
.o_seg(o_seg),
.o_sel(o_sel)
);
endmodule

```

cpu_clock_divider.v

```
`timescale 1ns / 1ps

module cpu_clock_divider #(
parameter integer HALF_PERIOD_CYCLES = 500000
)(
input clk_in,
input reset,
output clk_out
);
reg [31:0] counter;
reg divided_clk;

always @(posedge clk_in or posedge reset) begin
if (reset) begin
counter <= 32'd0;
divided_clk <= 1'b0;
end else if (counter == HALF_PERIOD_CYCLES - 1) begin
counter <= 32'd0;
divided_clk <= ~divided_clk;
end else begin
counter <= counter + 32'd1;
end
end

BUFG cpu_clk_buf(
.I(divided_clk),
.O(clk_out)
);
endmodule
```

post_sim_tb.v

```
`timescale 1ns / 1ps

module post_sim_tb;
reg clk_in;
reg reset;
wire [31:0] inst;
wire [31:0] pc;

integer cycles;
integer errors;
```

```

sccomp_dataflow dut(
.clk_in(clk_in),
.reset(reset),
.inst(inst),
.pc(pc)
);

initial begin
clk_in = 1'b0;
reset = 1'b1;
cycles = 0;
errors = 0;

#225 reset = 1'b0;
#120000;

if (errors == 0)
$display("POST_SIM_PASS cycles=%0d pc=%08h inst=%08h", cycles, pc, inst);
else
$display("POST_SIM_FAIL errors=%0d cycles=%0d pc=%08h inst=%08h", errors, cycles, pc,
inst);
$finish;
end

always #50 clk_in = ~clk_in;

always @(negedge clk_in) begin
if (!reset) begin
cycles = cycles + 1;
#20;

if ((^pc === 1'bx) || (^inst === 1'bx)) begin
errors = errors + 1;
$display("POST_SIM_ERROR unknown signal cycle=%0d pc=%h inst=%h", cycles, pc, inst);
end

if (cycles == 1 && pc !== 32'h0040_0010) begin
errors = errors + 1;
$display("POST_SIM_ERROR first jump pc=%h expected=00400010", pc);
end

if (cycles == 1 || cycles == 10 || cycles == 100 || cycles == 300 ||
cycles == 1000 || cycles == 1100 || cycles == 1200)
$display("POST_SIM_TRACE cycle=%0d pc=%08h inst=%08h", cycles, pc, inst);

```

```
end
end
endmodule
```